CSCI-GA 2110

Programming Languages

Course Notes

Fall 2026

Instructor: Prof. Joseph Tassarotti

Recitation leaders: Seong-Heon Jung and Vishnu Satish

Notes by You Li

Acknowledgements

These notes follow the Fall 2026 offering of CSCI-GA 2110 (Programming Languages) at the Courant Institute, taught by Prof. Joseph Tassarotti, with recitations led by Seong-Heon Jung and Vishnu Satish. The course studies programming languages by *implementing* them: each new language feature is added to an interpreter written in Racket, following Krishnamurthi's *Programming Languages: Application and Interpretation* [1].

Each chapter covers one week: first the lecture, split into one section per topic, then that week's recitation. The code is the code shown in lecture and the recitation handouts; recitation solutions are the ones posted by the teaching assistants.

Notation. Code is Racket (`#lang racket`) in Chapters 1 and 2 and `#lang plai-typed` from Chapter 3 on. Interactions at the REPL are shown with a `>` prompt; in `plai-typed` the line starting with `-` is the type the checker reports, and the line after it is the value. A trace such as

```
(fact 2)
---> (* 2 (fact 1))
```

shows one evaluation step per line. In the interpreter chapters, e ranges over expressions of the language being implemented, $e[v/x]$ is e with v substituted for the free occurrences of x, and $\mathrm{FV}(e)$ is the set of free variables of e. Type names such as ArithE are set in sans serif in prose.

Contents

1 Week 1: Course Introduction and Racket Basics **8**

1.1 Studying Programming Languages . . . 10
1.1.1 Classification Is Not Understanding . . . 10
1.1.2 Studying by Implementing . . . 10
1.1.3 Course Logistics . . . 11
1.2 Expressions, Functions, and Recursion . . . 12
1.2.1 Racket Is an Expression Language . . . 12
1.2.2 Defining Functions . . . 12
1.2.3 Conditionals . . . 12
1.2.4 Recursion . . . 12
1.2.5 Tail Recursion . . . 13
1.3 Pairs and Lists . . . 14
1.3.1 Pairs . . . 14
1.3.2 Lists Are Nested Pairs . . . 14
1.3.3 Reversing a List . . . 15
1.4 Recitation 1: Functional Programming in Racket . . . 16
1.4.1 Expressions and Functions . . . 16
1.4.2 Specifications . . . 17
1.4.3 Recursion: Scrolling Through Lights . . . 18
1.4.4 Lists and Higher-Order Programming . . . 19

2 Week 2: Higher-Order Functions, Scope, and State **22**

2.1 Lists and Accumulators . . . 23
2.1.1 `snoc`: Adding to the End . . . 23
2.1.2 Reversing, Correctly and Incorrectly . . . 23
2.1.3 Aggregating a List . . . 24
2.1.4 Named `let`: Hiding the Helper . . . 25
2.2 Structs and Trees . . . 26
2.2.1 `struct` . . . 26
2.2.2 Binary Trees . . . 26
2.2.3 `match` . . . 27
2.3 Higher-Order Functions . . . 28
2.3.1 Functions Are Values . . . 28
2.3.2 `map` and `filter` . . . 28
2.3.3 Folds . . . 29
2.4 Scope and State . . . 32

2.4.1 `let` and `let*` . . . 32
2.4.2 Lexical Scope . . . 32
2.4.3 Pure Code and Side Effects . . . 33
2.4.4 Boxes: State You Can See . . . 34
2.5 Recitation 2: Boolean Expressions . . . 35
2.5.1 The Language and Its Representation . . . 35
2.5.2 An Interpreter for bexp . . . 36
2.5.3 An Optimization Pass: De Morgan . . . 36
2.5.4 Testing the Optimizer Two Ways . . . 39

3 Week 3: Types and a First Interpreter 40
3.1 Types in plai-typed . . . 42
3.1.1 Every Value Has a Type . . . 42
3.1.2 Type Inference and Annotations . . . 42
3.1.3 Function Types . . . 43
3.1.4 Lists Are Homogeneous . . . 44
3.1.5 Pairs . . . 44
3.1.6 Polymorphic Types . . . 45
3.1.7 Why Insist on Homogeneous Lists? . . . 45
3.2 `define-type` and `type-case` . . . 46
3.2.1 Defining a Type with Variants . . . 46
3.2.2 `struct` vs. `define-type` . . . 46
3.2.3 `type-case`: Taking a Value Apart . . . 47
3.2.4 Exhaustiveness . . . 47
3.3 S-Expressions and the Calculator . . . 49
3.3.1 Quote and Symbols . . . 49
3.3.2 S-Expressions . . . 49
3.3.3 Concrete and Abstract Syntax . . . 50
3.3.4 The Calculator . . . 50
3.3.5 What Malformed Input Does . . . 51
3.4 Recitation 3: Typed Boolean Expressions and Pattern Matching . . . 53
3.4.1 Part A: Boolean Expressions, Typed . . . 53
3.4.2 Part B: Pattern Matching with `match` . . . 55
3.4.3 `match` vs. `type-case` . . . 57

4 Week 4: Names and Functions 58
4.1 Names: `let` and Substitution . . . 59
4.1.1 The Plan . . . 59
4.1.2 Two New Variants . . . 59
4.1.3 Eager Substitution . . . 60
4.1.4 Shadowing . . . 60
4.1.5 `eval-eager` . . . 61
4.2 Lazy Substitution and Variable Capture . . . 62
4.2.1 Lazy Substitution . . . 62
4.2.2 Is Lazy More Efficient? . . . 62
4.2.3 Where Eager and Lazy Differ . . . 63
4.2.4 Variable Capture . . . 63
4.3 Capture-Avoiding Substitution . . . 65

4.3.1 Rename, Then Substitute . . . 65
4.3.2 `gensym`: Fresh Names . . . 65
4.3.3 `subst-lazy-v2` . . . 66
4.4 Functions: `lambda` and Application . . . 68
4.4.1 Syntax . . . 68
4.4.2 `lambda` in Substitution . . . 69
4.4.3 A New Result Type . . . 69
4.4.4 Eager Application . . . 70
4.4.5 Lazy Application: `eval-lazy-v2` . . . 71
4.4.6 Capture, Again . . . 72
4.5 Recitation 4: Pattern Matching in the Interpreter . . . 73
4.5.1 The New Form . . . 73
4.5.2 Types . . . 73
4.5.3 Parsing . . . 74
4.5.4 Substitution . . . 75
4.5.5 Evaluation . . . 76

5 Week 5: From Substitution to Environments **78**
5.1 Substitution Revisited . . . 79
5.1.1 The Starting Point . . . 79
5.1.2 `let` Is Function Application . . . 79
5.1.3 Capture, Once More . . . 81
5.1.4 The Lambda Calculus . . . 82
5.1.5 α-Equivalence . . . 83
5.1.6 Lexical Addresses . . . 85
5.2 Environments . . . 88
5.2.1 The Plan . . . 88
5.2.2 The Data Structure . . . 88
5.2.3 The Interpreter . . . 89
5.2.4 Traces . . . 90
5.2.5 A Default Environment . . . 91
5.2.6 Checking Against Substitution . . . 91
5.3 Lazy `let` and Dynamic Scope . . . 93
5.3.1 Attempt 1: Store the Expression . . . 93
5.3.2 Dynamic vs. Lexical Scope . . . 94
5.3.3 Attempt 2: Store the Environment Too . . . 95
5.4 Functions with Environments: Closures . . . 96
5.4.1 Attempt 1: A Function Is Its Parameter and Body . . . 96
5.4.2 Attempt 2: Closures . . . 97
5.4.3 The Five Versions Side by Side . . . 99
5.5 Recitation 5: Environments as Functions, Scope, and Macros . . . 100
5.5.1 The Language . . . 100
5.5.2 Environments as Functions . . . 100
5.5.3 One Interpreter, Two Scopes . . . 103
5.5.4 Predict, Then Check . . . 104
5.5.5 Scope with Assignment . . . 107
5.5.6 Scope and Macros . . . 109
5.5.7 A Look at Lean . . . 111

6 Week 6: Mutable State **113**
6.1 State and Boxes . . . 115
6.1.1 Four New Constructs . . . 115
6.1.2 The Examples . . . 116
6.1.3 Attempt 1: A Box Holds Its Value . . . 116
6.2 Store-Passing Style . . . 118
6.2.1 Locations and the Store . . . 118
6.2.2 Results Carry the Store . . . 119
6.2.3 Fresh Locations . . . 119
6.2.4 Threading the Store Through Arithmetic . . . 121
6.2.5 `let` Is Application, Again . . . 122
6.2.6 Application . . . 122
6.3 Boxes in the Store-Passing Interpreter . . . 123
6.3.1 The Box Clauses . . . 123
6.3.2 Running the Examples . . . 124
6.3.3 Tracing `ex4` . . . 124
6.3.4 Tracing `ex5`: Two Tables, Two Paths . . . 125
6.3.5 The Store Only Grows . . . 126
6.4 The Meta-Circular Interpreter . . . 127
6.4.1 Boxes Implemented by Boxes . . . 127
6.4.2 Functions Implemented by Functions . . . 128
6.4.3 Side by Side . . . 129
6.4.4 What Each Version Explains . . . 129
6.5 Recitation 6: Mutable State and Tables . . . 131
6.5.1 The Language . . . 131
6.5.2 Environments and the Store . . . 132
6.5.3 Warm-up . . . 134
6.5.4 Evaluation Order . . . 135
6.5.5 Table Helpers . . . 137
6.5.6 Tables in the Interpreter . . . 139
6.5.7 Tables vs. Boxes . . . 141
6.5.8 Challenge: Passing the Store with Monads . . . 143
6.5.9 Summary . . . 146

7 Week 7: Recursion and Objects **147**
7.1 The Problem of Recursion . . . 149
7.1.1 Two New Forms . . . 149
7.1.2 Why `let` Is Not Enough . . . 149
7.1.3 A First Solution: `recV` . . . 150
7.2 Cycles from Boxes . . . 152
7.2.1 A Box That Contains Itself . . . 152
7.2.2 Backpatching . . . 152
7.2.3 A Recursive Function in Racket . . . 153
7.2.4 `fix`: Backpatching Once and for All . . . 153
7.3 `rec` with Boxes . . . 155
7.3.1 The Starting Point . . . 155
7.3.2 Desugaring `rec` into `fix` . . . 155
7.3.3 Inlining `fix` . . . 157

7.3.4 Why a Forwarding Function? . . . 157
7.4 Objects from Closures . . . 159
7.4.1 An Object Is a Function on Messages . . . 159
7.4.2 Constructors and Fields . . . 159
7.4.3 State . . . 160
7.4.4 `self` . . . 161
7.4.5 Objects with Structure: Trees . . . 163
7.4.6 Inheritance by Delegation . . . 163
7.5 Recitation 7: Recursion . . . 165
7.5.1 Tying the Knot with Boxes . . . 165
7.5.2 Recursion without Mutation . . . 168
7.5.3 Challenge: `rec` via `Z` . . . 171

8 Week 8: Macros **174**
8.1 Why `or` Is Not a Function . . . 176
8.2 A First Macro: `my-or` . . . 177
8.2.1 `my-if` . . . 178
8.2.2 `define-syntax-rule` . . . 178
8.2.3 `unless` . . . 178
8.3 A Loop: `my-while` . . . 179
8.3.1 A Macro That Never Stops Expanding . . . 179
8.3.2 Generating a Recursive Function . . . 179
8.4 Macros See the Code: `my-assert` . . . 181
8.5 Any Number of Arguments: `multi-or` . . . 182
8.5.1 Ellipses . . . 182
8.5.2 Recursion That Terminates . . . 182
8.5.3 Why the `let` . . . 182
8.5.4 A Name Introduced by the Macro . . . 183
8.6 Hygiene . . . 184
8.6.1 The Problem: Capture . . . 184
8.6.2 Hygienic Expansion . . . 184
8.6.3 Two Directions of Capture . . . 184
8.6.4 Connections . . . 185
8.7 Keywords in Patterns: `my-if2` . . . 186
8.8 `let` as a Macro . . . 187
8.8.1 `my-let*`: Recursion over the Bindings . . . 187
8.8.2 `my-let`: Many Bindings at Once . . . 188
8.9 `swap`: A Macro That Mutates Variables . . . 189
8.9.1 As a Function: It Does Nothing . . . 189
8.9.2 With Boxes: Pass the Locations . . . 189
8.9.3 As a Macro: Rewrite the Caller's Code . . . 189
8.10 Breaking Hygiene on Purpose: `aif` . . . 191
8.10.1 The Hygienic Version Fails . . . 191
8.10.2 Choosing the Context of a Name . . . 191
8.10.3 The Price . . . 192
8.11 Recitation 8: The Lambda Calculus . . . 193
8.11.1 Syntax . . . 193
8.11.2 Given: Renaming, Alpha-Equivalence, Substitution . . . 193

8.11.3 Evaluation: Normal Order 195
8.11.4 Church Booleans 197
8.11.5 Church Numerals 198
8.11.6 Challenge: Recursion with `Y` 199

Chapter 1

Week 1: Course Introduction and Racket Basics

Lecture 1: September 8, 2026. Recitation 1: September 11, 2026.

Assigned reading and resources (course page):

- Slides: `lecture01.pdf`.
- [8], Part 1, which the lecture worked through.
- [9], the essay behind the lecture's analogy between classifying languages and classifying species (Section 1.1).
- The Racket language site, `https://racket-lang.org`, with the installer and links to textbooks and tutorials.

Further reading:

- [1] Chapter 1 (Introduction).
- [4] Chapters 2 (Racket Essentials) and 3.8–3.9 (Pairs and Lists).
- [2] Part I, especially the design recipe (Section 3).
- [3] Sections 1.1 and 1.2.1 (the substitution model; linear recursion and iteration).

The first week explains how this course approaches programming languages, by building interpreters for small languages, and teaches enough Racket to start. This chapter covers:

- Section 1.1: why study programming languages, why classifying them by paradigm is the wrong place to start, and why we study languages by implementing interpreters;
- Section 1.2: Racket as an expression language, `define`, `cond`, recursion, and tail recursion with an accumulator;
- Section 1.3: pairs, lists, and the two-case shape of list functions;

- Section 1.4: Recitation 1, practicing function composition, specifications, recursion, and list functions on a traffic-light example.

1.1 Studying Programming Languages

1.1.1 Classification Is Not Understanding

The obvious way to organize the subject is by classification. Languages are sorted by *paradigm* (functional, imperative, object-oriented), by *abstraction* (high-level or low-level), by *representation* (text or graphical), and so on. A common course design follows from it: survey the categories, pick three or four languages from different ones, learn their basics, and write programs that highlight each one's features. That is not a bad idea, and you would learn a lot, but it is not the approach of this course.

Biological taxonomy makes the point. "Ape" sounds like a natural category: large, tailless primates such as the orangutan, the gorilla, and the chimpanzee. But when species are grouped by common descent, the group that contains all three also contains humans, and chimpanzees are closer to humans than to orangutans (Figure 1.1). A classification by surface features does not match the underlying structure. Sensible classification has to come *after* understanding the principles.

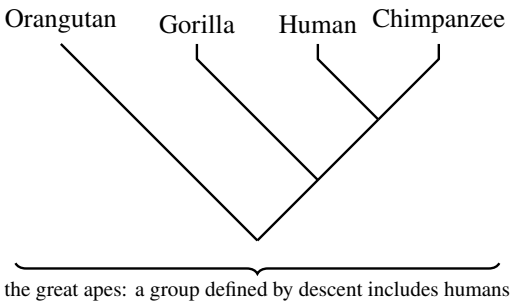


Figure 1.1: Grouping by common descent. Humans sit inside the great apes, not beside them (after slide 7).

The course's claim is that the same *building blocks* appear in many programming languages: functions, names and scope, data structures, state, types, control. It is better to understand these building blocks directly than to memorize which languages have which features.

Key Point 1 (Two false dichotomies)

- *"Functional" vs. "imperative" languages.* There is no dichotomy. A single language can have functional features, such as first-class functions, and imperative ones, such as mutable state. Racket, OCaml, Python, and JavaScript all have both.
- *"Interpreted" vs. "compiled" languages.* Interpreting and compiling describe an *implementation*, not a language. Any language can have both interpreters and compilers, so neither is a property of the language.

1.1.2 Studying by Implementing

To study the building blocks, we want to be precise and formal. One way is mathematical: give a theory of what programs mean and how they behave (formal semantics). This course takes a second way: it *implements* programming languages. To implement a feature you must decide exactly what it does in every case, so writing the implementation forces the precision that a mathematical definition would.

There are two strategies for implementing a language (Figure 1.2):

- A *compiler* translates a program in one language into another language that can be executed.
- An *interpreter* is a program that takes a program as input and directly executes it.

This course builds interpreters. Compilers are the subject of Compiler Construction (CSCI-GA 2130).

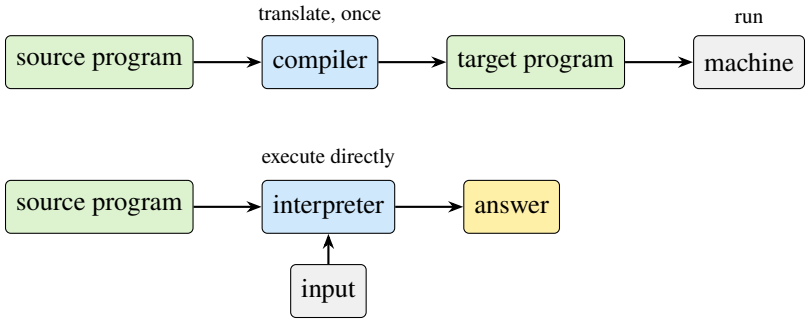


Figure 1.2: The two implementation strategies. This course writes interpreters.

1.1.3 Course Logistics

- **Staff.** Instructor: Joseph Tassarotti. Recitation leaders: Seong-Heon Jung and Vishnu Satish.
- **Textbook.** Krishnamurthi, *Programming Languages: Application and Interpretation*, 2nd edition [1], free online and linked from the syllabus.
- **Language.** The interpreters are written in Racket, for two reasons: Racket has many of the core features we want to study, so writing it is practice in them; and Racket was designed for implementing programming languages.
- **Grading.** Final exam 50%, homework 50%.
- **Homework.** Five assignments of two to three weeks each, done individually. Assignment 1 is a Racket warm-up, pure programming, auto-graded. Assignments 2–5 combine programming with conceptual questions. Up to 72 hours of lateness are allowed in total across all assignments; see the syllabus for details.

1.2 Expressions, Functions, and Recursion

1.2.1 Racket Is an Expression Language

A Racket program is not a list of steps to perform; it is an expression, and running it means simplifying that expression until only a value is left. Every construct below, including conditionals and function bodies, is an expression that produces a value.

Every call is written the same way: the function first, then its arguments, all inside parentheses. So addition is `(+ 3 x)`, not `3 + x`, and calling `foo` on 4 is `(foo 4)`.

1.2.2 Defining Functions

`(define (name argument ...) body)` gives a name to a function.

```
(define (foo x)
  (+ 3 x))
```

There is no `return` keyword, because there is nothing to return *from*: the body is a single expression, and its value is the answer. `(foo 4)` evaluates to `7`.

1.2.3 Conditionals

`cond` chooses between answers. It takes clauses written `[test result]`, tries the tests top to bottom, and returns the result of the first test that is true. `else` is the catch-all at the bottom.

```
(define (my-abs x)
  (cond
    [(> x 0) x]
    [(< x 0) (- x)]
    [else 0]))
```

`cond` is an *expression*, not a statement. It produces a value, which is why it can be the entire body of `my-abs`: there is no assignment and nothing to return, and the value of the `cond` is the value of the function. With one argument, `(- x)` is negation. Only the chosen branch is evaluated, so a branch that would divide by zero is harmless as long as its test is false.

1.2.4 Recursion

Recursion is how repetition is done. A recursive function needs two things:

- a *base case* that answers directly, without recurring, and
- a *recursive case* that makes the problem smaller, in a way that eventually reaches the base case, and trusts the function itself with the rest.

```
(define (fact x)
  (cond
    [(= x 0) 1]
    [else (* x (fact (- x 1)))]))
```

The way to check a recursive function by hand is to rewrite the expression one step at a time:

```
(fact 3)
---> (* 3 (fact 2))
---> (* 3 (* 2 (fact 1)))
---> (* 3 (* 2 (* 1 (fact 0))))
---> (* 3 (* 2 (* 1 1)))
---> 6
```

The `*` sits *outside* the recursive call, so no multiplication can happen on the way down. Each level waits for the one below it, and every level stays on the stack until the recursion bottoms out. The trace grows a tower of pending multiplications.

1.2.5 Tail Recursion

The work can be rearranged so that the multiplying happens on the way *down*. Pass the answer-so-far along as an extra argument, an *accumulator*:

```
(define (fact-helper x acc)
  (cond
    [(= x 0) acc]
    [else (fact-helper (- x 1) (* acc x))]))

(define (fact-tail x)
  (fact-helper x 1))
```

The accumulator starts at 1, the value that leaves a product unchanged; a sum would start at 0.

Definition 2 (Tail call)

A call is a *tail call* if it is the last thing the calling function does: its result is returned unchanged, with no work left pending. Racket guarantees that a tail call reuses the caller's stack frame instead of pushing a new one.

The recursive call in `fact-helper` is a tail call, so the function runs in constant stack space, like a loop:

```
(fact-helper 3 1)
---> (fact-helper 2 3)
---> (fact-helper 1 6)
---> (fact-helper 0 6)
---> 6
```

Nothing is ever pending, because `acc` is a finished number at every step. Since `fact-helper` takes two arguments and `fact` one, the wrapper `fact-tail` keeps the interface the same. Hiding the helper is a recurring theme; Section 2.1 does it with a named `let`.

1.3 Pairs and Lists

1.3.1 Pairs

`cons` glues any two values into a *pair*, and `car` and `cdr` take the pair apart:

```
> (cons 3 4)
'(3 . 4)
> (car (cons 1 2))
1
> (cdr (cons 1 2))
2
```

The names `car` and `cdr` are historical, from the registers of the IBM 704 on which Lisp was first implemented. `first` and `rest` are the same operations on lists, with readable names (the names [2] uses). Both work; pick one style and stay with it.

1.3.2 Lists Are Nested Pairs

A list is not a separate kind of thing. It is a chain of pairs in which the right component is always another list, ending in `empty` (also written `'()`):

Definition 3 (List)

A list is either `empty`, or `(cons x l)` where `x` is any value and `l` is a list.

```
(define l (cons 1 (cons 2 (cons 3 (cons 4 empty)))))
(define l-short (list 1 2 3 4))
```

`l` and `l-short` are the same value; `list` is shorthand for the chain of `cons`es (Figure 1.3). Both print as `'(1 2 3 4)`. Then `(first l)` is `1` and `(rest l)` is `'(2 3 4)`.

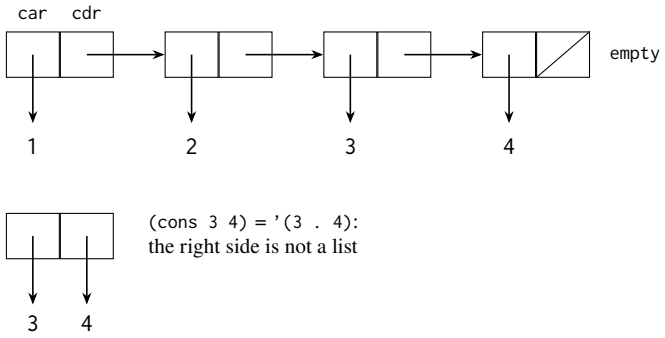


Figure 1.3: Box-and-pointer diagrams. Top: `'(1 2 3 4)`, four pairs whose `cdr`s chain to `empty` (drawn as a slash). Bottom: a bare pair.

Pitfall 4 (The dot)

The dot in `'(3 . 4)` means the right side of that pair is not a list, so the chain does not end in `empty`. Whenever a dot appears in output where you expected a list, you built a bare pair somewhere.

The two-case definition is why list functions keep having two `cond` clauses: one for `empty`, one for a `cons`.

1.3.3 Reversing a List

Key Point 5 (The shape of the function follows the shape of the data)

A list is empty or a `cons`, so a function on lists has those two cases. In the `cons` case, the recursive call is on the `rest`, the part of the data that is itself a list.

The empty list reversed is the empty list. Otherwise, reverse everything after the first element, then put the first element at the end:

```
(define (my-reverse l)
  (cond
    [(empty? l) empty]
    [else (append (my-reverse (rest l)) (list (first l)))]))
```

`append` joins two lists, so both arguments must be lists. That is why the first element is wrapped as `(list (first l))`, the one-element list. Rewriting by hand:

```
(my-reverse '(1 2 3))
---> (append (my-reverse '(2 3)) '(1))
---> (append (append (my-reverse '(3)) '(2)) '(1))
---> (append (append '(3) '(2)) '(1))
---> (append '(3 2) '(1))
---> '(3 2 1)
```

This works, but `append` itself walks its whole left argument, and it is called once per element, so `my-reverse` takes $O(n^2)$ time. Racket's built-in `reverse` uses an accumulator instead, the same trick as `fact-tail`; Recitation 1 writes that version (Section 1.4).

1.4 Recitation 1: Functional Programming in Racket

This section practices the basics of the chapter in DrRacket, on a running example about traffic lights. The code is written as a literate program: it is written in `#lang scribble/lp2`, a literate-programming language in which code sits in named `@chunk` blocks inside the prose. "Scribble HTML/PDF" renders the document; "Run" runs every chunk listed in the top-level `<*>` chunk. Tests are written with `check-expect` from `test-engine/racket-tests`, and the final `(test)` reports the results.

1.4.1 Expressions and Functions

```
(define (is-even x)
  (= 0 (modulo x 2)))
```

Exercise 6 (Predict at the REPL)

What do `is-even`, `(is-even)`, `(is-even 2)`, `(is-even 3)`, `(is-even #t)`, and `(is-even 'four)` evaluate to?

Solution. `is-even` alone is the function value, printed `#<procedure:is-even>`; a function is a value like any other. `(is-even)` is an *arity mismatch*: the function expects one argument. `(is-even 2)` is `#t` and `(is-even 3)` is `#f`. `(is-even #t)` and `(is-even 'four)` are *contract violations* raised inside `modulo`, which expects integers. Racket checks this only when the call runs: nothing stops you from writing the call.

Building programs by composition. In functional style, programs are built by composing functions. A traffic light is one of `'red`, `'yellow`, `'green`. Two functions are given:

- `can-go` EXPECTS a traffic light and PRODUCES a boolean: `#t` when a vehicle approaching the light may safely enter the intersection.
- `next-light` EXPECTS a traffic light and PRODUCES the light that comes after it.

```
(define (can-go l) (equal? l 'green))

(define (next-light l)
  (match l
    ['red    'green]
    ['yellow 'red]
    ['green  'yellow]))
```

`match` compares `l` against each pattern in turn; a quoted symbol pattern matches that symbol. (Section 3.4 looks at `match` in detail.)

Exercise 7 (Pedal to the metal)

Write `can-floor-it`: it EXPECTS a traffic light and PRODUCES `#t` when a vehicle can safely enter the intersection at either the current light or the next one.

Solution. Compose the two given functions:

```
(define (can-floor-it l)
  (or (can-go l) (can-go (next-light l))))
(check-expect (can-floor-it 'green) #t)
(check-expect (can-floor-it 'red) #t)
(check-expect (can-floor-it 'yellow) #f)
```

Red is safe because the next light is green; yellow is not, because the next light is red.

1.4.2 Specifications

Before writing a function, state which inputs are valid and what output to expect. The format is a list of EXPECTS and PRODUCES lines plus one sentence of meaning. For `next-light-iter`:

- EXPECTS a non-negative integer `n` and a traffic light `l`;
- PRODUCES a traffic light: the color of `l` after it changes `n` times.

The specification answers the two questions the code needs. When `n` is zero, the light has not changed, so the answer is `l`. When `n` is positive, the answer is the light after changing $n - 1$ times starting from `(next-light l)`.

```
(define (next-light-iter n l)
  (if (eq? n 0)
      l
      (next-light-iter (- n 1) (next-light l))))

(check-expect (next-light-iter 0 'green) 'green)
(check-expect (next-light-iter 1 'green) 'yellow)
(check-expect (next-light-iter 2 'yellow) 'green)
```

The tests come straight from the specification: one for the base case, one for a single step, and one that wraps around the cycle (yellow, red, green). The recursive call is a tail call, with the light itself playing the role of an accumulator.

1.4.3 Recursion: Scrolling Through Lights

A vehicle is one of `'sports-car`, `'suv`, `'prius`. `speed` gives its speed in meters per second and `time-to-change` gives how many seconds a light stays on:

```
(define (speed v)
  (match v
    ['sports-car 136]
    ['suv        48]
    ['prius      27]))

(define (time-to-change l)
  (match l
    ['red    10]
    ['yellow 2]
    ['green  23]))
```

Exercise 8 (Scroll)

`safe-to-scroll` EXPECTS a vehicle `v`, a non-negative distance `d` in meters, and a light `l` that has just turned on. It PRODUCES `#t` if the vehicle, driving at constant speed, enters the intersection without a violation. Hint: write a helper that decides whether the vehicle enters before the light changes.

Solution. If the vehicle reaches the intersection before the light changes, the answer is whether *this* light allows entry. Otherwise the problem shrinks: the vehicle has driven for one full light, so it is closer, and the light is now the next one.

```
(define (will-enter v d l)
  (<= d (* (time-to-change l) (speed v))))

(define (safe-to-scroll v d l)
  (if (will-enter v d l)
      (can-go l)
      (safe-to-scroll
       v
       (- d (* (time-to-change l) (speed v)))
       (next-light l))))

(check-expect (safe-to-scroll 'sports-car 137 'green) #t)
```

Two traces show both outcomes. A Prius covers $23 \times 27 = 621$ m during a green light:

```
(safe-to-scroll 'prius 1000 'green)    ; 1000 > 23*27 = 621
---> (safe-to-scroll 'prius 379 'yellow)  ; 379 > 2*27 = 54
---> (safe-to-scroll 'prius 325 'red)     ; 325 > 10*27 = 270
---> (safe-to-scroll 'prius 55 'green)    ; 55 <= 621: enters on green
---> #t

(safe-to-scroll 'suv 1200 'green)      ; 1200 > 23*48 = 1104
---> (safe-to-scroll 'suv 96 'yellow)     ; 96 <= 2*48 = 96: enters on yellow
---> #f
```

The recursion terminates because every recursive call has a strictly smaller `d`: each light lasts a positive time and every vehicle has positive speed.

Remark 9. The problem statement writes the call as `(safe-to-scroll v l d)`, while the code takes `(v d l)`. The code and its test are consistent, so the order is `(v d l)`. This is the kind of mismatch a specification is meant to catch.

1.4.4 Lists and Higher-Order Programming

Warm-up: `cons` vs. `append`.

expression	result
`(cons 3 4)`	`'(3 . 4)`, a bare pair
`(cons 3 '(4))`	`'(3 4)`
`(cons '(4) 3)`	`'((4) . 3)`: a pair whose `car` is a list
`(append 3 4)`	error: `append: contract violation`
`(append 3 '(4))`	error: the first argument must be a list
`(append '(3) 4)`	`'(3 . 4)`: the last argument is not checked

`cons` puts one element in front of a list; `append` joins lists. Neither complains about every misuse, so watch for dots in the output.

Exercise 10 (All even)

Write `all-even-rec`: it EXPECTS a list of integers and PRODUCES `#t` when every number in it is even.

Solution. The solution uses `match` with a `cons` pattern, which tests for a non-empty list and names its two parts in one step:

```
(define (all-even-rec l)
  (match l
    ['()              #t]
    [(cons head tail) (and (is-even head) (all-even-rec tail))]))
(check-expect (all-even-rec '()) #t)
(check-expect (all-even-rec '(2 4 6)) #t)
(check-expect (all-even-rec '(2 4 6 7)) #f)
```

The empty list is all even (there is no counterexample), which is also what makes `#t` the right base value for `and`.

Exercise 11 (Map)

Given `all`, which checks that a list of booleans is all `#t`, write `all-even-map` without explicit recursion.

```
(define (all l) (andmap identity l))
```

Solution. The intended answer is `(all (map is-even l))`: `map` turns the numbers into booleans and `all` combines them. The solution skips the intermediate list and passes `is-even` to `andmap` directly:

```
(define (all-even-map l) (andmap is-even l))
```

`(andmap f l)` is `#t` when `f` returns true on every element. It stops at the first element that fails, whereas `map` would first compute all the booleans.

Exercise 12 (Reverse, and reverse with a tail call)

Write a function that reverses a list. Then write a version that uses tail-call optimization.

Solution. The direct version is `my-reverse` from Section 1.3, written with `match`:

```
(define (reverse-list l)
  (match l
    ['() '()]
    [(cons head tail) (append (reverse-list tail) (list head))]))
```

Its calls pile up pending appends:

```
(reverse-list '(1 2 3 4 5 6 7))
---> (append (reverse-list '(2 3 4 5 6 7)) (list 1))
---> (append (append (reverse-list '(3 4 5 6 7)) (list 2)) (list 1))
---> ...
```

On a list with millions of elements, that is a call stack millions deep. The tail-recursive version carries the reversed-so-far list in an accumulator. Moving elements one at a time from the front of `l` to the front of `acc`

reverses their order, and cons is the right tool, since it adds to the front:

```
(define (reverse-helper l acc)
  (match l
    ['() acc]
    [(cons head tail) (reverse-helper tail (cons head acc))]))

(define (reverse-list-tail l)
  (reverse-helper l '()))
```

```
(reverse-helper '(1 2 3) '())
---> (reverse-helper '(2 3) '(1))
---> (reverse-helper '(3) '(2 1))
---> (reverse-helper '() '(3 2 1))
---> '(3 2 1)
```

This is also faster: $O(n)$ instead of $O(n^2)$, since there is no append.

Remark 13 (Tail-call *elimination*)**.** When a language *guarantees* the optimization, it is often called tail-call elimination. Racket guarantees it; many languages, including Rust and Java, do not.

Chapter 2

Week 2: Higher-Order Functions, Scope, and State

Lecture 2: September 15, 2026. Recitation 2: September 18, 2026.

Assigned reading and resources (course page):

- [8], Sections 2 and 3, with a focus on higher-order functions.

Further reading:

- [4] Sections 4.4–4.6 (functions, local binding), 4.8 (sequencing), 5 (programmer-defined datatypes), and 12 (pattern matching).
- [2] Part II (self-referential data) and Part III (abstraction: `map`, `filter`, folds).
- [3] Sections 1.3 (higher-order procedures) and 3.1 (assignment and local state).

This chapter finishes the tour of Racket: more list recursion and accumulators, structs for tree-shaped data, functions as values, scope, and finally state. It ends with the first interpreter of these notes, for Boolean expressions, together with an optimization pass. This chapter covers:

- Section 2.1: adding to the end of a list, a quadratic `reverse` and a broken one, and accumulators packaged as named `let`;
- Section 2.2: `struct`, binary trees, and `match`;
- Section 2.3: `lambda`, closures, `map`, `filter`, and the two folds;
- Section 2.4: `let` and `let*`, lexical scope, pure code and side effects, and boxes;
- Section 2.5: Recitation 2, an interpreter and a De Morgan optimizer for Boolean expressions, and how to test an optimization.

2.1 Lists and Accumulators

Throughout this chapter we use the names `empty?`, `first`, and `rest` rather than `null?`, `car`, and `cdr`. They mean the same thing.

2.1.1 snoc: Adding to the End

`(cons x l)` makes one new pair whose head is `x` and whose tail is the old list `l`. Nothing is copied, so adding to the *front* costs $O(1)$ however long the list is.

Adding to the *back* is different. The old list ends in `'()`, and that `'()` cannot be replaced without rebuilding every pair on the way to it. So a back-insert walks the whole list, in $O(n)$. Racket does not provide one; we write it and call it `snoc`, "cons" backwards:

```
(define (snoc l x)
  (cond
    [(empty? l) (list x)]
    [else (cons (first l) (snoc (rest l) x))]))
```

Two shapes of input, two branches: the skeleton of almost every list function in this course. The empty case answers "the list containing just `x`"; the other keeps the first element in place and hands the rest of the problem to `snoc`. So `(snoc '() 'x)` is `'(x)` and `(snoc '(1 2) 'x)` is `'(1 2 x)`.

2.1.2 Reversing, Correctly and Incorrectly

To reverse a list, reverse everything after the first element, then put the first element at the very end, which is exactly `snoc`:

```
(define (rev l)
  (cond
    [(empty? l) '()]
    [else (snoc (rev (rest l)) (first l))]))
```

```
(rev '(1 2 3))
---> (snoc (rev '(2 3)) 1)
---> (snoc (snoc (rev '(3)) 2) 1)
---> (snoc (snoc '(3) 2) 1)
---> (snoc '(3 2) 1)
---> '(3 2 1)
```

This is correct but slow: one `snoc` per element, each $O(n)$, so $O(n^2)$ in total. It also needs $O(n)$ stack, because the `snoc` is wrapped *around* the recursive call, which therefore is not a tail call.

Here is a tempting rewrite with cons instead of snoc:

```
(define (rev-alt l)
  (cond
    [(empty? l) '()]
    [else (cons (rev-alt (rest l)) (first l))]))
```

```
> (rev-alt '(1 2 3))
'((() . 3) . 2) . 1)
```

Pitfall 14 (cons is not symmetric)

cons expects an element on the left and a list on the right. rev-alt passes them the other way round, so it builds a nest of pairs, not a list (Figure 2.1). There is no error, because a pair may hold any two values: the wrong argument order is a *silent* bug, and the dots in the output are the only sign. Swapping the arguments does not rescue this approach: cons can only ever add to the front, and reversing needs to add to the back.

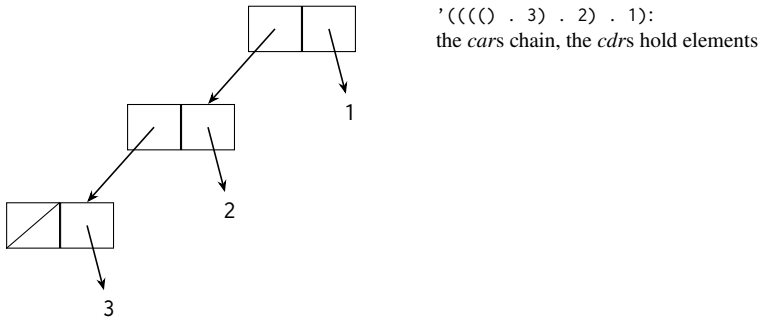


Figure 2.1: What rev-alt builds: every pair has a non-list on its right, hence the dots.

2.1.3 Aggregating a List

Summing a list follows the same skeleton:

```
(define (sum-list l)
  (cond
    [(empty? l) 0]
    [else (+ (first l) (sum-list (rest l)))]))
```

The + sits *outside* the recursive call. It cannot happen until the recursion returns, so every level must be remembered, and a long enough list runs out of stack.

As with fact in Section 1.2, the fix is to carry the running total as an extra argument, making the recursive

call a tail call:

```
(define (sum-list-helper l acc)
  (cond
    [(empty? l) acc]
    [else (sum-list-helper (rest l) (+ (first l) acc))]))

(define (sum-list-acc l)
  (sum-list-helper l 0))
```

```
(sum-list-helper '(1 2 3) 0)
---> (sum-list-helper '(2 3) 1)
---> (sum-list-helper '(3) 3)
---> (sum-list-helper '() 6)
---> 6
```

2.1.4 Named `let`: Hiding the Helper

A top-level helper with an extra argument is noise. A *named* `let` packages the same thing as a loop: the name after `let` becomes a local function, and the bindings give its parameters and their starting values.

```
(define (sum-list-loop l)
  (let loop ([l l]
             [acc 0])
    (cond
      [(empty? l) acc]
      [else (loop (rest l) (+ (first l) acc))])))
```

This is the idiomatic way to write a loop in Racket; it runs exactly like the helper version.

2.2 Structs and Trees

2.2.1 `struct`

`struct` defines a new kind of compound value. One line

```
(struct bt (label left right) #:transparent)
```

generates five functions:

- `bt`, the constructor: `(bt 1 empty empty)`;
- `bt?`, a predicate that answers `#t` for these values;
- `bt-label`, `bt-left`, `bt-right`, the accessors for the three fields.

Pitfall 15 (Forgetting `#:transparent`)

Without `#:transparent`, a struct value prints as `#<bt>` with its contents hidden, and `equal?` is true only when the two values are the very same object. Two trees that look identical then compare as different, and every test that compares trees fails. With the flag, values print readably and `equal?` compares field by field.

`struct` does *not* check the types of its fields. `(bt 1 2 3)` is accepted, even though 2 and 3 are not trees. Nothing goes wrong until some traversal calls `bt-label` on the number 2, possibly far from the line that built the bad value. Section 3.2 returns to this point: `define-type` in `plai-typed` rejects such a value where it is built.

2.2.2 Binary Trees

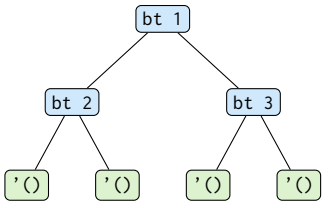
The empty tree is represented by `'()`, so a binary tree is either `empty` or a `bt` with a label and two subtrees.

```
(define tree
  (bt 1
      (bt 2 empty empty)
      (bt 3 empty empty)))
```

The definition of a tree mentions trees in *two* places, left and right, so a function over trees has two recursive calls. Lists mentioned themselves once, and list functions had one. The shape of the code follows the shape of the data (Figure 2.2).

```
(define (sum-bt t)
  (cond
    [(empty? t) 0]
    [(bt? t) (+ (bt-label t)
                (sum-bt (bt-left t))
                (sum-bt (bt-right t)))]
    [else (error 'sum-bt "not a tree: ~a" t)]))
```

Tree recursion is usually not made tail-recursive: with two recursive calls, at most one of them can be the last thing the function does, and the other has to be remembered.



(sum-bt tree)
$= 1 +$ (sum-bt (bt 2 ...)) $+$ (sum-bt (bt 3 ...))
$= 1 + (2 + 0 + 0) + (3 + 0 + 0)$
$= 6$

Figure 2.2: The value `tree`. Each `bt` node causes two recursive calls, one per subtree.

2.2.3 `match`

`match` checks the shape and extracts the fields in one step, which reads better once all the fields are needed anyway:

```
(define (sum-bt/match t)
  (match t
    ['() 0]
    [(bt label left right)
     (+ label (sum-bt/match left) (sum-bt/match right))]))
```

The pattern `(bt label left right)` matches only `bt` values and binds the three names to the fields in order.

2.3 Higher-Order Functions

2.3.1 Functions Are Values

A function does not need a name. `lambda` builds one on the spot, and it can be called immediately: `((lambda (x) (* x x)) 4)` is `16`. So `define` for functions is shorthand, and these two definitions are the same:

```
(define (square x)
  (* x x))

(define square-alt
  (lambda (x) (* x x)))
```

Use the short form. Do not nest one form inside the other: `(define (f x) (lambda ...))` defines a function that *returns* a function, which is a different thing.

Since functions are values, a function can return one:

```
(define (make-adder x)
  (lambda (y) (+ x y)))

(define add2 (make-adder 2))
```

Definition 16 (Closure)

A *closure* is a function value together with the bindings of the variables it refers to from the surrounding code.

`(make-adder 2)` returns a function that has not been called yet but already remembers that `x` is 2. Then `(add2 5)` is 7, and `((make-adder 3) 4)` is 7 as well.

2.3.2 `map` and `filter`

Once functions can be arguments, the two most common list patterns become ordinary functions. `map` applies a function to every element; the result has the same length as the input.

```
(define (my-map f l)
  (cond
    [(empty? l) '()]
    [else (cons (f (first l)) (my-map f (rest l)))]))
```

So

```
(map f (list a b c))                    ; = (list (f a) (f b) (f c))
(map (lambda (x) (+ 1 x)) (list 1 2 3))    ; = '(2 3 4)
```

`filter` keeps the elements that pass a test; the contents are unchanged, only the length shrinks. To drop an

element, return the filtered rest without consing it back on:

```
(define (my-filter f l)
  (cond
    [(empty? l) '()]
    [else (if (f (first l))
              (cons (first l) (my-filter f (rest l)))
              (my-filter f (rest l)))]))
```

`(filter odd? (list 1 2 3 4 5))` is `'(1 3 5)`.

Key Point 17 (Three shapes of list code)

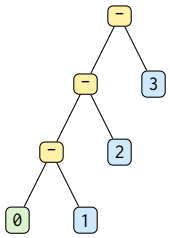
`map`: list to list, same length. `filter`: list to list, shorter. `fold`: list to a single value. Most list code is a combination of these three.

2.3.3 Folds

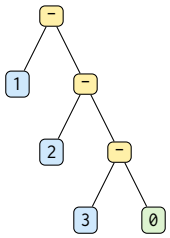
`map` and `filter` each fix what happens at every step. A *fold* takes the step as an argument: a two-argument `combine`, a `base` value for the empty list, and the list. It walks the list once and collapses it into one value. There are two folds, and they differ in which way the parentheses group (Figure 2.3):

$$\texttt{(my-foldl - 0 '(1 2 3))} = ((0-1)-2)-3 = -6,$$
$$\texttt{(my-foldr - 0 '(1 2 3))} = 1-(2-(3-0)) = 2.$$

With an associative and commutative `combine` such as `+`, `*`, or `max`, the two agree, which is why the difference is easy to miss until it bites.



`foldl`: $((0-1)-2)-3$



`foldr`: $1-(2-(3-0))$

Figure 2.3: The two folds of `-` over `'(1 2 3)` with base 0 (green). `foldl` leans left, starting from the base; `foldr` leans right, ending at the base.

`foldl` combines on the way down. In `my-foldl`, `base` doubles as the running accumulator. Each step folds one more element into it and calls itself, and that call is a tail call, so this is a loop in constant stack space:

```
(define (my-foldl combine base l)
  (cond
    [(empty? l) base]
    [else (my-foldl combine (combine base (first l)) (rest l))]))
```

```
(my-foldl + 0 '(1 2 3 4 5))
---> (my-foldl + 1 '(2 3 4 5))
---> (my-foldl + (+ 1 2) '(3 4 5))
---> (my-foldl + (+ 3 3) '(4 5))
---> (my-foldl + (+ 6 4) '(5))
---> (my-foldl + 15 empty)
---> 15
```

Nothing is pending; at every step the accumulator is a value and the list is shorter.

`foldr` combines on the way back up. In `my-foldr`, `combine` wraps the recursive call. No arithmetic can happen until the recursion reaches `'()` and the base value comes back:

```
(define (my-foldr combine base l)
  (cond
    [(empty? l) base]
    [else (combine (first l) (my-foldr combine base (rest l)))]))
```

```
(my-foldr + 0 '(1 2 3 4 5))
---> (+ 1 (my-foldr + 0 '(2 3 4 5)))
---> (+ 1 (+ 2 (my-foldr + 0 '(3 4 5))))
---> (+ 1 (+ 2 (+ 3 (+ 4 (+ 5 (my-foldr + 0 empty))))))
---> (+ 1 (+ 2 (+ 3 (+ 4 (+ 5 0)))))
---> 15
```

That growing pile of pending additions *is* the stack: it is the concrete reason `foldr` uses $O(n)$ space and `foldl` $O(1)$.

Key Point 18 (`foldr` rebuilds structure)

`(my-foldr cons '() l)` copies `l`, because `foldr` replaces each `cons` of the list by `combine` and the final `'()` by `base`. Hence `map` and `filter` are both folds in disguise:

```
(define (map-as-fold f l)
  (my-foldr (lambda (x acc) (cons (f x) acc)) '() l))

(define (filter-as-fold f l)
  (my-foldr (lambda (x acc) (if (f x) (cons x acc) acc)) '() l))
```

Pitfall 19 (Racket's `foldl` argument order)

The `combine` of `my-foldl` takes `(acc elem)`. Racket's built-in `foldl` calls `(combine elem acc)`, element first, the opposite order. So `(foldl - 0 '(1 2 3))` in Racket is $3 - (2 - (1 - 0)) = 2$, not -6. Check the order before reusing a `combine`.

2.4 Scope and State

2.4.1 `let` and `let*`

`let` binds names to values for the extent of its body. The clauses go in one pair of brackets; square and round brackets are interchangeable in Racket, so both of these are the same form:

```
(let ([x (+ 1 2)])
  (+ x x))            ; 6

(let [(x (+ 1 2))
      (y (+ 4 5))]
  (+ x y))            ; 12
```

The catch is *when* each name starts to exist. `let` evaluates all of its right-hand sides first, in the scope *outside* the `let`, and only then binds the names. So a later clause cannot use an earlier one:

```
(let ([x (+ 1 2)]
      [y (+ 1 x)])     ; x is not bound here
  (+ x y))
;; x: undefined; cannot reference an identifier before its definition
```

Read the message as "that name is not in scope here".

`let*` binds the clauses one at a time, each in the scope of the previous ones, so this version works and gives 7:

```
(define let-example
  (let* ([x (+ 1 2)]
         [y (+ 1 x)])
    (+ x y)))
```

There is no magic in `let*`; it is nested `let`s:

```
(let* ([x a] [y b]) body)   means   (let ([x a]) (let ([y b]) body)).
```

Use `let` by default, since it tells the reader the bindings are independent, and `let*` only when a clause really needs the one above it. Neither lets a clause refer to *itself*: a local recursive function needs `letrec`, or the named `let` of Section 2.1.

2.4.2 Lexical Scope

A `let` is an expression, so it can appear anywhere a value can, including in another `let`'s right-hand side:

```
(define nested-let-example
  (let ([x (let ([y 3]) (+ y y))])
    (+ x x)))           ; x = 6, so 12
```

But `y` exists only inside the inner `let`. Writing `(+ x y)` in the outer body is the same undefined-identifier error, even though `y` was "used" a moment earlier (Figure 2.4).

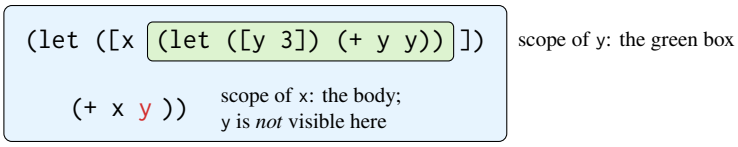


Figure 2.4: Scopes nest like the text. The body of the outer `let` is outside the inner `let`, so `y` is unbound there.

Definition 20 (Lexical scope)

Under *lexical* (static) scope, which binding a use of a name refers to is determined by where the binding is *written* in the program text, not by what happened to run before.

This is the same rule that makes the closure in `make-adder` work: the `x` in `(lambda (y) (+ x y))` refers to the `x` of the enclosing `define`, wherever the returned function is later called.

2.4.3 Pure Code and Side Effects

Everything so far is *pure*: a call depends only on its arguments and does nothing but return a value. Run the same expression twice and you get the same answer, so an expression can always be replaced by its value without changing the program. That is what makes the step-by-step traces of `rev` and the folds valid reasoning, not just illustration.

A *side effect* is anything else a call does: printing, mutating a variable, reading input, raising an error. In a language built on statements, `i++` is exactly this. Running it twice is not the same as running it once, so it cannot be replaced by its value.

- `"Hello World"` is a value. Evaluating it does nothing else.
- `(print "Hello World")` writes to the console and returns `(void)`. The output is the point; the return value is not.

Racket does not forbid effects; it makes them ordinary expressions, so staying pure is a choice made function by function.

Sequencing. A function body may hold several expressions. They run in order, the earlier ones only for their effects, and the value of the *last* one is returned:

```
(define (foo x)
  (print "Hello")
  (+ x 1))
```

`(foo 1)` returns 2 and prints along the way. The returned number is perfectly predictable; the printing is what makes `foo` impure. `begin` is the same sequencing as a single expression, for places where only one expression is allowed, such as a `cond` branch or an arm of `if`:

```
(begin
  (print "Hello")
  (print "World")
  (+ 1 2))          ; returns 3, after printing
```

Three ways to print. All return `(void)`; they differ only in what appears on the console. `(print "Hello")` shows `"Hello"`, the value as Racket would write it, quotes included. `(display "Hello")` shows `Hello`, the human-readable form. `(printf "The number foo is: ~a !" (* 3 3))` shows `The number foo is: 9 !`: `~a` splices in the display form of the next argument.

2.4.4 Boxes: State You Can See

A *box* is a mutable container with exactly one slot, the smallest possible piece of state. It is a value like any other, so it can be passed to a function that changes it.

```
(define bar (box 3))

(define (increment-box b)
  (set-box! b (+ 1 (unbox b))))
```

`(box 3)` makes a box, `(unbox bar)` reads it, and `(set-box! bar 10)` overwrites it and returns `(void)`. `increment-box` is called for its effect and returns nothing useful, so the caller has to `unbox` afterwards to see the new contents:

```
(let ([b (box 3)])
  (increment-box b)
  (unbox b))          ; 4
```

The point of a box is that mutability is *visible*. A plain 3 can never change; a box holding 3 announces that someone may change it, so a reader can tell where the state lives. By convention, a name ending in `!` mutates something: a warning that calling it twice is not the same as calling it once.

2.5 Recitation 2: Boolean Expressions

This section builds the first interpreter and first program transformation, for a tiny language of Boolean expressions. Everything is in plain Racket, with structs and `match`.

2.5.1 The Language and Its Representation

The grammar of *bexp*:

$$\langle bexp \rangle \ ::= \ \langle boolean \rangle \ \mid \ \langle bexp \rangle \wedge \langle bexp \rangle \ \mid \ \langle bexp \rangle \vee \langle bexp \rangle \ \mid \ \neg \langle bexp \rangle$$

One struct per alternative of the grammar represents it in Racket:

```
(struct leaf-node (arg))
(struct or-node (left right))
(struct and-node (left right))
(struct not-node (arg))

(define bexp-bool-1 (and-node (leaf-node #t) (leaf-node #f)))
(define bexp-sym-1 (and-node (leaf-node 'x) (leaf-node 'y)))
```

A value built from these structs is an *abstract syntax tree*: the tree structure replaces parentheses and precedence. Note that the structs are *not* transparent and their fields are unchecked: `bexp-sym-1` has symbols, not booleans, at its leaves, and Racket accepts it.

Exercise 21 (Syntactic equality)

Write `syn-equal?`, which checks whether two bexps are the same tree.

Solution. Because the structs are opaque, `equal?` would compare them by identity (Section 2.2), so we write structural equality ourselves. `match*` matches several values at once, one pattern per value:

```
(define (syn-equal? b1 b2)
  (match* (b1 b2)
    [((and-node l1 r1) (and-node l2 r2)) (and (syn-equal? l1 l2) (syn-equal? r1 r2))]
    [((or-node l1 r1) (or-node l2 r2))   (and (syn-equal? l1 l2) (syn-equal? r1 r2))]
    [((not-node a1) (not-node a2))       (syn-equal? a1 a2)]
    [((leaf-node v1) (leaf-node v2))     (equal? v1 v2)]
    [(_ _) #f]))
```

One clause per variant where both sides agree, and a final catch-all for mismatched shapes.

Pitfall 22 (A missing variant hides behind a catch-all)

If the `not-node` clause were left out, two `not-nodes` would fall through to `[(_ _) #f]`, and `syn-equal?` would say a negation is not equal to itself. Nothing would complain, and every later test comparing expressions with a negation would fail even with a correct optimizer. With a catch-all clause present, check the list of variants, not just the clauses you were given.

2.5.2 An Interpreter for bexp

Exercise 23 (Interpreter)

Write `evaluate-bexp`, which computes the Boolean value an expression represents.

Solution. One clause per variant, and a recursive call wherever a variant contains a sub-expression:

```
(define (evaluate-bexp b)
  (match b
    [(leaf-node v)    v]
    [(or-node b1 b2)  (or (evaluate-bexp b1) (evaluate-bexp b2))]
    [(and-node b1 b2) (and (evaluate-bexp b1) (evaluate-bexp b2))]
    [(not-node b1)    (not (evaluate-bexp b1))]))

(check-expect (evaluate-bexp (leaf-node #t)) #t)
(check-expect (evaluate-bexp (and-node (leaf-node #t) (leaf-node #f))) #f)
(check-expect (evaluate-bexp (and-node (leaf-node #f) (leaf-node #t))) #f)
```

The third test swaps the arguments of the second; testing both orders is cheap and catches an implementation that looks at only one side.

This is the pattern of every interpreter in these notes: our language's $\wedge$ is implemented by Racket's `and`, its $\vee$ by `or`, its $\neg$ by `not`. The *meaning* of each construct is given by the clause that handles it.

Missing cases. What happens with an incomplete `match`?

```
(define (evaluate-bexp-m b)
  (match b
    [(leaf-node v) v]))
```

Nothing complains when it is defined. The error appears only when it is called on a value that no clause matches: `(evaluate-bexp-m bexp-bool-1)` raises `match: no matching clause`. Racket checks patterns at run time, so a missing case is found only if some test happens to reach it. Section 3.4 redoes this in `plai-typed`, where the same mistake is rejected before the program runs.

2.5.3 An Optimization Pass: De Morgan

De Morgan's laws and double-negation elimination state the equivalences

$$\neg(P \wedge Q) = \neg P \vee \neg Q, \qquad \neg(P \vee Q) = \neg P \wedge \neg Q, \qquad \neg\neg P = P.$$

Using them left to right, an optimizer can eliminate redundant negations and push every remaining negation down to the leaves. The examples use symbolic leaves:

name	before	after
`bexp-1`	$\neg(\neg P \vee Q)$	$P \wedge \neg Q$
`bexp-2`	$\neg\neg P \vee \neg Q$	$P \vee \neg Q$
`bexp-3`	$P \vee \neg Q$	$P \vee \neg Q$ (already optimal)

```
(define bexp-1 (not-node (or-node (not-node (leaf-node 'P)) (leaf-node 'Q))))
(define bexp-2 (or-node (not-node (not-node (leaf-node 'P))) (not-node (leaf-node 'Q))))
(define bexp-3 (or-node (leaf-node 'P) (not-node (leaf-node 'Q))))

(define bexp-1-opt (and-node (leaf-node 'P) (not-node (leaf-node 'Q))))
(define bexp-2-opt (or-node (leaf-node 'P) (not-node (leaf-node 'Q))))
(define bexp-3-opt (or-node (leaf-node 'P) (not-node (leaf-node 'Q))))
```

What does it mean for an optimization to be correct?. Before writing the optimizer, decide how to check it. An optimization is correct when it does two things:

1. it *preserves semantics*: the output evaluates to the same value as the input;
2. it actually *optimizes*: the output has the promised shape, here every negation sitting directly on a leaf.

Both become executable checks:

```
(define (preserves-sem? unopt)
  (equal? (evaluate-bexp unopt) (evaluate-bexp (demorgan-opt unopt))))

(define (is-optimizing? opt)
  (define (only-not-at-leaves? bexp)
    (match bexp
      [(leaf-node v) #t]
      [(or-node b1 b2) (and (only-not-at-leaves? b1) (only-not-at-leaves? b2))]
      [(and-node b1 b2) (and (only-not-at-leaves? b1) (only-not-at-leaves? b2))]
      [(not-node (leaf-node v)) #t]
      [(not-node _) #f]))
  (only-not-at-leaves? opt))

(define (is-correct? unopt)
  (and (preserves-sem? unopt) (is-optimizing? (demorgan-opt unopt))))
```

`is-optimizing?` relies on `match` trying clauses in order: `(not-node (leaf-node v))` is tried first, so the catch-all `(not-node _)` sees only negations of compound expressions, which mean the pass left work undone. Note also that `preserves-sem?` takes only the input and calls `demorgan-opt` itself, so it tests the optimizer rather than a hand-written answer.

The optimizer. What to do with a negation depends entirely on what is *inside* it, so a single `(not-node b1)` clause cannot work. Nested patterns split it into the four shapes the argument can have, one per variant. This

is why the optimizer needs more cases than `evaluate-bexp`:

```
(define (demorgan-opt b)
  (match b
    ; Standard Stuff
    [(leaf-node v)    (leaf-node v)]
    [(or-node b1 b2)  (or-node (demorgan-opt b1) (demorgan-opt b2))]
    [(and-node b1 b2) (and-node (demorgan-opt b1) (demorgan-opt b2))]
    [(not-node (leaf-node l)) (not-node (leaf-node l))]
    ; De-Morgan Stuff
    [(not-node (and-node b1 b2))
     (or-node (demorgan-opt (not-node b1)) (demorgan-opt (not-node b2)))]
    [(not-node (or-node b1 b2))
     (and-node (demorgan-opt (not-node b1)) (demorgan-opt (not-node b2)))]
    [(not-node (not-node b1)) (demorgan-opt b1)]))
```

- **A negated leaf** is already as far down as a negation can go; keep it.
- **De Morgan.** After rewriting $\neg(b_1 \wedge b_2)$ to $\neg b_1 \vee \neg b_2$, the new negations may themselves sit on top of an `and`, an `or`, or a `not`, so they are optimized again: the recursive calls are on `(not-node b1)`, not on `b1`. Calling `(demorgan-opt b1)` would drop the negation and change the meaning.
- **Double negation.** The two negations cancel, and the result is `(demorgan-opt b1)`, not just `b1`, because `b1` may still contain negations to push down.

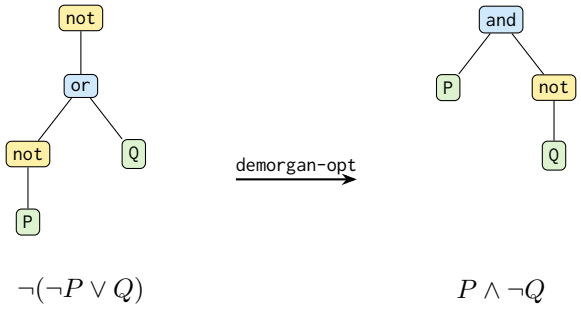


Figure 2.5: `bexp-1` before and after the pass. Negations (yellow) end up only directly above leaves.

For `bexp-1` (Figure 2.5):

```
(demorgan-opt (not (or (not P) Q)))
---> (and-node (demorgan-opt (not (not P))) (demorgan-opt (not Q)))   ; not over or
---> (and-node (demorgan-opt P) (not Q))                              ; not-not cancels
---> (and-node P (not Q))
```

Why it terminates. Every recursive call is on a smaller tree. In the De Morgan cases the argument `(not-node b1)` is new, but it is smaller than the input `(not-node (and-node b1 b2))`, which has one more node and all of `b2` besides.

2.5.4 Testing the Optimizer Two Ways

The syntactic tests compare against the exact trees written by hand:

```
(check-expect (syn-equal? (demorgan-opt bexp-1) bexp-1-opt) #t)
(check-expect (syn-equal? (demorgan-opt bexp-2) bexp-2-opt) #t)
(check-expect (syn-equal? (demorgan-opt bexp-3) bexp-3-opt) #t)
```

The semantic tests check the two correctness properties without writing the expected tree at all:

```
(define bexp-4 (not-node (or-node (not-node (leaf-node #t)) (leaf-node #f))))
(define bexp-5 (or-node (not-node (not-node (leaf-node #t))) (not-node (leaf-node #f))))
(define bexp-6 (or-node (leaf-node #t) (not-node (leaf-node #t))))
(check-expect (is-correct? bexp-4) #t)
(check-expect (is-correct? bexp-5) #t)
(check-expect (is-correct? bexp-6) #t)
```

Remark 24 (Why booleans at the leaves)**.** `bexp-4` to `bexp-6` are `bexp-1` to `bexp-3` with `#t` and `#f` in place of `'P` and `'Q`. With symbols at the leaves, `evaluate-bexp` is meaningless: `(not 'P)` is `#f` and `(or 'P #f)` is `'P`, because Racket treats every value except `#f` as true. So a semantic check only means something on Boolean leaves.

Remark 25 (Checking the table)**.** It is easy to write the first example as $\neg(\neg P \wedge Q) \mapsto P \wedge Q$. That contradicts both the code (`bexp-1` is built with `or-node`) and De Morgan's law; the correct row is $\neg(\neg P \vee Q) \mapsto P \wedge \neg Q$. Trust the definitions over the prose, and check the prose by working one example by hand.

Chapter 3

Week 3: Types and a First Interpreter

Lecture 3: September 22, 2026. Recitation 3: September 25, 2026.

Assigned reading and resources (course page):

- [1] Chapters 2 (Everything (We Will Say) About Parsing) and 3 (A First Look at Interpretation): the parser and interpreter for the arithmetic language.
- [1] Chapter 4 (A First Taste of Desugaring): extending the arithmetic language with operations such as subtraction while keeping a minimal core language. Not discussed in lecture, but the course page calls it an important idea; it returns as a language feature in Chapter 8.
- The `plai-typed` reference documentation, https://docs.racket-lang.org/plai-typed/index.html.
- Lecture code: `lecture3a.rkt` (examples of `plai-typed`) and `lecture3b.rkt` (interpreter and parser for the arithmetic language).

Further reading:

- [1] Chapter 15 (Checking Program Invariants Statically: Types), first sections.
- [5] Chapter 1 (what type systems are for), optional.

From this chapter on, code is written in `#lang plai-typed`, the language of [1]: Racket with a type checker that reads the whole program before anything runs. We introduce its types, define new types with `define-type`, and use them to write a first interpreter for a language of our own, a calculator. The last section redoes the Boolean-expression interpreter of Section 2.5 with types and then studies Racket's `match` in more depth. This chapter covers:

- Section 3.1: types, type inference and annotations, function types, homogeneous lists, pairs, and polymorphism;
- Section 3.2: `define-type`, `type-case`, and exhaustiveness checking;
- Section 3.3: quote and s-expressions, concrete vs. abstract syntax, and the calculator's parser and interpreter;

- Section 3.4: Recitation 3, typed Boolean expressions and pattern matching with `match`.

3.1 Types in plai-typed

`plai-typed` is installed with `raco pkg install plai-typed`, and a file starts with `#lang plai-typed`. It looks like Racket, but a type checker reads the whole program first, and a program that fails the check does not run at all.

3.1.1 Every Value Has a Type

At the REPL, every answer comes with two lines: the type the checker worked out, starting with `-`, and then the value.

```
> 3
- number
3
> (+ 3 4)
- number
7
> "hello"
- string
"hello"
> (+ 3 "hello")
typecheck failed: number vs. string
```

This is the idea of static typing in miniature: the type is known *before* the value is computed. The checker does not run `(+ 3 4)` to find out it is a number; it knows because `+` takes numbers and returns a number.

3.1.2 Type Inference and Annotations

No types need to be written. Here the checker sees `(+ 3 x)`, concludes that `x` must be a number, and that the result is one too:

```
(define (add3 x)
  (+ 3 x))
```

Definition 26 (Type inference)

Type inference is the checker working out the types that the programmer did not write down.

Types can also be written, as many or as few as we like. An argument becomes `(x : number)`, and the return type goes after the argument list. Whatever is left out is inferred:

```
(define (add3-v2 (x : number))            ; return type inferred
  (+ 3 x))

(define (add3-v3 (x : number)) : number   ; both annotated
  (+ 3 x))
```

Pitfall 27 (Spaces around the colon)

`(x: number)` is read as one identifier named `x:`, followed by `number`, and is rejected with `expected either <id> or '[<id> : <type>]'`. Write `(x : number)`.

An annotation is a claim the checker verifies, not a hint it trusts. This one is wrong, since `(+ 3 x)` is a number, so the definition is rejected:

```
(define (add3-v4 (x : number)) : string
  (+ 3 x))
;; typecheck failed: number vs. string
```

Either way the program is checked before it runs: a call like `(add3-v3 "hello")` never gets the chance to fail at run time, because the whole file is rejected.

If the checker can infer the types, why write them? There are two reasons:

- **Documentation.** The reader sees what the function takes and returns without reading its body.
- **Mental model.** Writing the type first makes you decide what the function is supposed to do before you write how it does it.

3.1.3 Function Types

Functions are values, so they have types. Read the arrow as "takes . . . and returns . . . ":

```
> add3
- (number -> number)
```

With two arguments, both go to the left of the arrow. Annotations can be mixed: in `myadd-v2`, `y` is unannotated but added to a number, so it is inferred to be one, and the two functions have the same type.

```
(define (myadd x y)
  (+ x y))
(define (myadd-v2 (x : number) y) : number
  (+ x y))
```

```
> myadd
- (number number -> number)
> myadd-v2
- (number number -> number)
```

A function that returns a function, `make-adder` from Section 2.3, now has a type that says so:

```
(define (myadd-v3 (x : number))
  (lambda (y) (+ x y)))
```

```
> myadd-v3
- (number -> (number -> number))
> (myadd-v3 2)
- (number -> number)
> ((myadd-v3 2) 5)
- number
7
```

Compare the two types. `myadd` takes two numbers at once. `myadd-v3` takes *one* number and returns a function that waits for the second, so `(myadd-v3 2)` is not a number yet but a function.

3.1.4 Lists Are Homogeneous

A list type says what its elements are, and every element must have that type:

```
> (list 1 2 3 4)
- (listof number)
'(1 2 3 4)
> (cons 3 (cons 4 empty))
- (listof number)
'(3 4)
> (list 1 "hello")
typecheck failed: number vs. string
> (cons 3 4)
typecheck failed: (listof number) vs. number
```

Plain Racket accepts both of the last two. In `plai-typed` the second argument of `cons` must be a list, so bare pairs (Section 1.3) are impossible. In exchange, list operations have precise types: the checker knows `(first (list 1 2 3 4))` is a `number` and `(rest (list 1 2 3 4))` a `(listof number)`, and likewise for `second` and `third`, without running anything.

3.1.5 Pairs

When two values of different types belong together, use a *pair* (a tuple). Its type is written with `*`:

```
> (pair 1 "hello")
- (number * string)
'#(1 "hello")
> (fst (pair 1 2))
- number
1
> (snd (pair 1 2))
- number
2
> (car (pair 1 2))
car: unbound identifier
```

A list has any length but one element type; a pair has exactly two slots, each with its own type. A pair is not a cons cell, and `plai-typed` provides no `car` or `cdr` at all: use `fst`/`snd` for pairs and `first`/`rest` for lists.

3.1.6 Polymorphic Types

Asking for the type of `first` shows something new:

```
> first
- ((listof 'a) -> 'a)
> rest
- ((listof 'a) -> (listof 'a))
> cons
- ('a (listof 'a) -> (listof 'a))
> fst
- (('a * 'b) -> 'a)
> map
- (('a -> 'b) (listof 'a) -> (listof 'b))
```

Definition 28 (Type variable, parametric polymorphism)

`'a` (read "alpha") is a *type variable*: any type may stand in its place, but every `'a` in one type must stand for the *same* type. A single definition that works uniformly at every type in this way is *parametrically polymorphic*; other languages call it generics.

So `first` takes a `(listof number)` to a `number` and a `(listof string)` to a `string`, but can never take a list of numbers to a string. Different letters allow different types: `fst` takes a pair of an `'a` and a `'b` and returns the `'a`. And the type of `map` alone says most of what `map` does: given a function from `'a` to `'b` and a list of `'a`, it produces a list of `'b`.

3.1.7 Why Insist on Homogeneous Lists?

Suppose `(list 1 "hello")` were allowed; call it `l`. What is the type of `(first l)`? It depends on which element happens to come first, and the checker cannot know that before the program runs. Then

- every use of `first` would need a check at run time, which is exactly what a type checker is supposed to spare us, and
- `(map f l)` is only safe if `f` accepts *both* numbers and strings, which `('a -> 'b)` cannot express.

One could design a type that lists every position, something like `(listof number string number)` for `(list 1 "hello" 2)`, but then every list function (`map`, `filter`, the folds) would need a far more complicated type. The simpler fix keeps lists homogeneous and instead makes *one* type that covers all the kinds of things to be mixed. That is what `define-type` is for.

3.2 define-type and type-case

3.2.1 Defining a Type with Variants

A shape is either a circle or a rectangle:

```
(define-type Shape
  [circle (radius : number)]                       ; variant 1
  [rectangle (width : number) (height : number)])  ; variant 2
```

This defines one new type, Shape, with two *variants*. Each variant has a constructor and typed fields:

```
> (circle 3)
- Shape
(circle 3)
```

Now `(list (circle 1) (rectangle 2 3))` is fine: the elements are different kinds of shapes, but they have one type, so the list is a `(listof Shape)`.

Like `struct`, `define-type` also generates

- predicates `circle?` and `rectangle?`, of type `(Shape -> boolean)`, and
- accessors `circle-radius`, `rectangle-width`, and `rectangle-height`, of type `(Shape -> number)`.

The predicates accept only shapes. In plain Racket `(circle? 5)` would return `#f`; here it never runs:

```
> (circle? 5)
typecheck failed: Shape vs. number
```

A recursive type. A type may mention itself. The binary tree of Section 2.2, now with types:

```
(define-type Tree
  [emp]
  [node (n : number) (lt : Tree) (rt : Tree)])
```

A variant with no fields is still a constructor, so the empty tree is `(emp)`, with parentheses. `(node 3 (emp) (emp))` has type Tree.

3.2.2 struct vs. define-type

Both make compound values, but they promise different things:

`struct` (`#lang racket`)	`define-type` (`plai-typed`)
one kind of value: a `bt` is always a `bt`	one *type* with several variants: `circle` or `rectangle`
fields unchecked: `(bt 1 2 3)` is accepted, and fails later	every field typed: `(node 3 5 (emp))` is rejected before the program runs, at the line that is wrong
different structs cannot share a homogeneous list type	all variants fit in one `(listof Shape)`

Why only one `circle`?. No other type in the program may also have a variant named `circle`. The immediate reason is that `circle`, `circle?`, and `circle-radius` are ordinary definitions, so a second `define-type` would define them again (`identifier already defined`). The deeper reason is inference: when the checker sees `(circle 3)`, the constructor's name is all it has to go on. If two types both had a `circle` variant, `(circle 3)` could belong to either, and inference would be stuck.

3.2.3 `type-case`: Taking a Value Apart

A `define-type` value is built with a constructor; `type-case` takes it apart again. There is one clause per variant, and the names in parentheses are bound to that variant's fields, in order (the names need not match those in `define-type`; only the order matters):

```
(define (area s)
  (type-case Shape s
    [circle (r) (* 3.14159 (* r r))]
    [rectangle (w h) (* w h)]))
```

```
> (area (circle 3))
- number
28.27431
> area
- (Shape -> number)
```

The same function can be written with the generated predicates and accessors:

```
(define (area-v2 s)
  (cond
    [(circle? s) (* 3.14159 (* (circle-radius s) (circle-radius s)))]
    [(rectangle? s) (* (rectangle-width s) (rectangle-height s))]))
```

It works, but `type-case` is shorter, names the fields, and lines the code up with the type: one clause per variant, just as list functions had one `cond` branch per kind of list.

3.2.4 Exhaustiveness

The real difference shows when a case is forgotten. `type-case` knows every variant of `Shape`, so leaving one out is rejected before the program runs:

```
(define (area-v3 s)
  (type-case Shape s
    [circle (r) (* 3.14159 (* r r))]))
;; type-case: syntax error; probable cause: you did not include a case
;; for the rectangle variant, or no else-branch was present
```

`cond` knows nothing about `Shape`; it tries its tests in order. The same mistake with `cond` is accepted, works on

circles, and fails only at run time when it meets a rectangle:

```
(define (area-v4 s)
  (cond
    [(circle? s) (* 3.14159 (* (circle-radius s) (circle-radius s)))]))
```

```
> (area-v4 (circle 1))
- number
3.14159
> (area-v4 (rectangle 2 3))
cond: no matching clause
```

Key Point 29 (What type-case buys)

type-case checks before the program runs that every variant is handled, and it gives each clause the variant's fields with their types. A missing case is a static error, not a bug waiting for the right input.

3.3 S-Expressions and the Calculator

3.3.1 Quote and Symbols

The apostrophe `'`, read "quote", means "do not evaluate this; keep it as data". Without it, `x` is a variable to look up. With it, `'x` is the name itself, a *symbol*:

```
> x
x: free variable while typechecking in: x
> 'x
- symbol
'x
```

Quoting a parenthesized expression keeps the whole thing as data, without doing the addition:

```
> '(1 (+ 2 3) 4)
- s-expression
'(1 (+ 2 3) 4)
```

3.3.2 S-Expressions

Look at what `'(+ 2 3)` contains: a symbol and two numbers. That is exactly the kind of mixed list that Section 3.1 ruled out. So `plai-typed` gives all quoted data one type, `s-expression`, and provides functions to ask what a piece is and to convert it back out:

```
> (s-exp->list '(+ 2 3))
- (listof s-expression)
'(+ 2 3)
> (first (s-exp->list '(+ 2 3)))
- s-expression
'+
> (s-exp->symbol (first (s-exp->list '(+ 2 3))))
- symbol
'+
> (s-exp->number (second (s-exp->list '(+ 2 3))))
- number
2
```

Key Point 30 (Ask, then convert)

Ask with `s-exp-number?`, `s-exp-symbol?`, or `s-exp-list?`, then convert with the matching `s-exp->number`, `s-exp->symbol`, or `s-exp->list`. A conversion fails at run time if the guess was wrong.

Pitfall 31 (A lone quoted symbol is not an s-expression)

`'x` has type `symbol`, not `s-expression`, so `(parse 'x)` is a type error. To get an s-expression for one symbol, write `` `x `` (backquote) or `(symbol->s-exp 'x)`.

3.3.3 Concrete and Abstract Syntax

The first interpreter is a calculator:

```
'(+ 2 3)                    ===> 5
'(* 3 4)                    ===> 12
'(* (+ 2 3) (* 5 (+ 6 7)))  ===> 325
```

Following [1, Chapters 2–3], the work is split into separate steps (Figure 3.1):

- **Concrete syntax** is what the programmer types, the text `(+ 2 3)`. Racket's *reader* already turns text into an s-expression, so we get that step for free; quote is how we hand an s-expression over directly.
- **Abstract syntax** is a tree that keeps only the meaning. `parse` turns an s-expression into this tree, rejecting anything that is not a valid program.
- The **interpreter** walks the tree and computes the answer. [1] calls it `interp`; here it is `eval`.

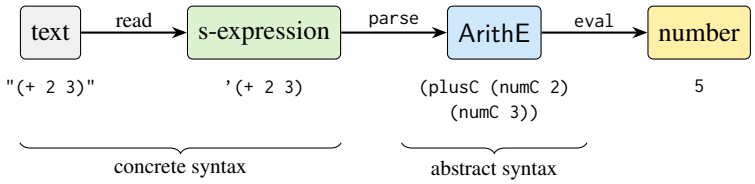


Figure 3.1: The interpreter pipeline. Racket's reader provides the first step.

3.3.4 The Calculator

Abstract syntax. One variant per kind of expression:

```
(define-type ArithE
  [numC (n : number)]
  [plusC (e1 : ArithE) (e2 : ArithE)]
  [timesC (e1 : ArithE) (e2 : ArithE)])
```

The operands of `plusC` are ArithE, not `number`. That is what allows `'(* (+ 2 3) 4)`: the left operand of the multiplication is itself an addition, the same trick as Tree having subtrees (Figure 3.2).

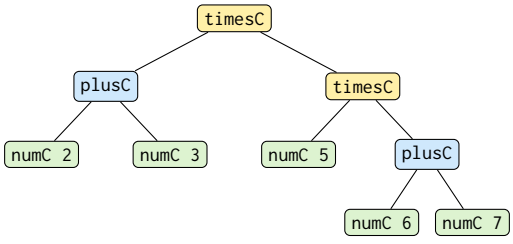


Figure 3.2: The abstract syntax of `'(* (+ 2 3) (* 5 (+ 6 7)))`. Parentheses are gone; the nesting is the tree.

The interpreter. One clause per variant:

```
(define (eval (e : ArithE)) : number
  (type-case ArithE e
    [numC (n) n]
    [plusC (e1 e2) (+ (eval e1) (eval e2))]
    [timesC (e1 e2) (* (eval e1) (eval e2))]))
```

Three variants, three clauses, and a recursive call wherever the type mentions ArithE. `eval` never checks whether its input makes sense, and it does not have to: a value of type ArithE can only be built from the three constructors, so every tree it receives is well formed.

The parser. That guarantee has to come from somewhere, and it comes from `parse`. A minimal version:

```
(define (parse (s : s-expression)) : ArithE
  (cond
    [(s-exp-number? s) (numC (s-exp->number s))]
    [(s-exp-list? s)
     (let [(l (s-exp->list s))]
       (case (s-exp->symbol (first l))
         [(+) (plusC (parse (second l)) (parse (third l)))]
         [(*) (timesC (parse (second l)) (parse (third l)))]))]))
```

A number becomes a `numC`. A list is dispatched on its first element with `case`, which compares a symbol against literal symbols, and each operand is parsed recursively. Putting the steps together, `(eval (parse '(* (+ 2 3) (* 5 (+ 6 7)))))` is 325.

3.3.5 What Malformed Input Does

This parser is right on well-formed input, but on bad input it either crashes with a message about something else, or quietly builds a wrong tree:

input	result
`(parse '(- 1 2))`	`case: no matching clause`
`(parse '(+ 1))`	`third: list contains too few elements`
`(parse '(+ 1 2 3))`	`(plusC (numC 1) (numC 2))`: the 3 is silently dropped

The last one is the worst kind of bug: no error, just a program that means something other than what was written. A more careful parser checks the shape of each form and reports what was expected. This version is

not from lecture, but every check in it addresses one row of the table:

```
(define (parse (s : s-expression)) : ArithE
  (cond
    [(s-exp-number? s) (numC (s-exp->number s))]
    [(s-exp-list? s)
     (let [(l (s-exp->list s))]
       (if (and (= (length l) 3) (s-exp-symbol? (first l)))
           (case (s-exp->symbol (first l))
             [(+) (plusC (parse (second l)) (parse (third l)))]
             [(*) (timesC (parse (second l)) (parse (third l)))]
             [else (error 'parse "unknown operator")])
           (error 'parse "expected (op e1 e2)")))]
    [else (error 'parse "expected a number or a list")]))
```

- The length check rejects `'(+ 1)` and `'(+ 1 2 3)`.
- `s-exp-symbol?` rejects `'(1 2 3)`, whose head is not an operator at all.
- The `else` of `case` rejects operators the language does not have, such as `-`.
- The final `else` rejects anything that is neither a number nor a list, such as a lone symbol.

3.4 Recitation 3: Typed Boolean Expressions and Pattern Matching

This section has two halves. Part A redoes the interpreter and De Morgan optimizer of Section 2.5 in `plai-typed`, to see what the type checker buys. Part B returns to plain Racket and studies `match`: literal patterns, wildcards, and taking lists apart.

3.4.1 Part A: Boolean Expressions, Typed

From four structs to one type. In Section 2.5, a Boolean expression was built from four unrelated structs; "a bexp" existed only in our heads, and no field had a type. With `define-type`, the four become variants of *one* type, and every field says what it holds:

```
(define-type bexp-b
  [leaf-node-b (b : boolean)]
  [and-node-b (b1 : bexp-b) (b2 : bexp-b)]
  [or-node-b (b1 : bexp-b) (b2 : bexp-b)]
  [not-node-b (b : bexp-b)])
```

Exercise 32 (Typed evaluator)

Implement `evaluate-bexp-b` with `type-case`.

Solution. `type-case` plays the role `match` played in Section 2.5:

```
(define (evaluate-bexp-b [b : bexp-b]) : boolean
  (type-case bexp-b b
    [leaf-node-b (v) v]
    [and-node-b (b1 b2) (and (evaluate-bexp-b b1) (evaluate-bexp-b b2))]
    [or-node-b (b1 b2) (or (evaluate-bexp-b b1) (evaluate-bexp-b b2))]
    [not-node-b (b1) (not (evaluate-bexp-b b1))]))

(test (evaluate-bexp-b (leaf-node-b #t)) #t)
(test (evaluate-bexp-b (and-node-b (leaf-node-b #t) (leaf-node-b #f))) #f)
(test (evaluate-bexp-b (not-node-b (or-node-b (leaf-node-b #f) (leaf-node-b #f)))) #t)
```

`test` is `plai-typed`'s `check-expect`, and `(print-only-errors true)` at the top of a program makes passing tests silent. This is the same shape as `eval` for ArithE in Section 3.3.

What do the types buy us?.

(a) **Bad leaves are rejected before anything runs.** Last week's `bexp-sym-1` was

```
(and-node (leaf-node 'x) (leaf-node 'y))
```

and `evaluate-bexp` happily returned `'y`: Racket treats every value except `#f` as true, so `(and 'x 'y)` is `'y`. No error, just an answer of the wrong kind. The typed version

```
(evaluate-bexp-b (and-node-b (leaf-node-b 'x) (leaf-node-b 'y)))
```

is rejected (`typecheck failed: boolean vs. symbol`) before anything runs, and the error points at the leaf that is wrong, not at some later function that trips over it.

(b) **Missing cases are rejected too.** Last week's `evaluate-bexp-m` handled only leaves; it was accepted and failed only when called on something else. The typed version

```
(define (evaluate-bexp-b-m [b : bexp-b]) : boolean
  (type-case bexp-b b
    [leaf-node-b (v) v]))
```

stops the program from starting: `you did not include a case for the and-node-b variant`. This is the `area-v3`/`area-v4` contrast of Section 3.2 again.

De Morgan, typed. Now the leaves are variables:

```
(define-type bexp
  [leaf-node (s : symbol)]
  [and-node (b1 : bexp) (b2 : bexp)]
  [or-node (b1 : bexp) (b2 : bexp)]
  [not-node (b1 : bexp)])
```

Exercise 33 (Port `demorgan-opt` to `plai-typed`)

Rewrite the optimizer of Section 2.5 for the type `bexp`.

Solution. Last week's `match` could name a nested pattern directly, such as `(not-node (and-node b1 b2))`. A `type-case` clause only takes apart the *outermost* constructor, so the `not-node` clause needs a second `type-case`, on the thing being negated:

```
(define (demorgan-opt [b : bexp]) : bexp
  (type-case bexp b
    [leaf-node (v) (leaf-node v)]
    [and-node (b1 b2) (and-node (demorgan-opt b1) (demorgan-opt b2))]
    [or-node (b1 b2) (or-node (demorgan-opt b1) (demorgan-opt b2))]
    [not-node (b1)
      (type-case bexp b1
        [leaf-node (v) (not-node (leaf-node v))]
        [and-node (b2 b3) (or-node (demorgan-opt (not-node b2))
                                   (demorgan-opt (not-node b3)))]
        [or-node (b2 b3) (and-node (demorgan-opt (not-node b2))
                                   (demorgan-opt (not-node b3)))]
        [not-node (b2) (demorgan-opt b2)])]))
```

The inner clauses are the four nested patterns of Section 2.5, one per variant of the negated expression: a negated leaf stays; $\neg(b_2 \wedge b_3)$ becomes $\neg b_2 \vee \neg b_3$ with both new negations optimized again; $\neg(b_2 \vee b_3)$ symmetrically; and $\neg\neg b_2$ becomes the optimized b_2. Both `type-case`s are checked for exhaustiveness, so a forgotten shape of negation is now a static error.

The tests no longer need `syn-equal?`: `test` compares `define-type` values structurally.

```
(define bexp-1 (not-node (or-node (not-node (leaf-node 'P)) (leaf-node 'Q))))
(define bexp-2 (or-node (not-node (not-node (leaf-node 'P))) (not-node (leaf-node 'Q))))
(define bexp-3 (or-node (leaf-node 'P) (not-node (leaf-node 'Q))))

(test (demorgan-opt bexp-1) (and-node (leaf-node 'P) (not-node (leaf-node 'Q))))
(test (demorgan-opt bexp-2) (or-node (leaf-node 'P) (not-node (leaf-node 'Q))))
(test (demorgan-opt bexp-3) (or-node (leaf-node 'P) (not-node (leaf-node 'Q))))
```

3.4.2 Part B: Pattern Matching with `match`

So far `match` has only taken structs apart. It can also match literal values, ignore parts of the input with wildcards, and take lists apart.

Matching on values. A pattern can be a literal, which makes `match` a compact conditional:

```
(define (factorial x)
  (match x
    [0 1]
    [_ (* x (factorial (- x 1)))]))

(define (factorial-if x)
  (if (= x 0)
      1
      (* x (factorial-if (- x 1)))))
```

_ is the *wildcard*: it matches anything and binds nothing. Clauses are tried top to bottom, so _ goes last as the catch-all. With two cases `match` and `if` are about the same size; with more, nested `if`s pile up:

```
(define (fibonacci-if x)
  (if (= x 0)
      0
      (if (= x 1)
          1
          (+ (fibonacci-if (- x 1)) (fibonacci-if (- x 2))))))
```

Exercise 34 (Fibonacci with `match`)

Rewrite `fibonacci-if` using `match`.

Solution. One line per case, and the structure stays flat:

```
(define (fibonacci x)
  (match x
    [0 0]
    [1 1]
    [_ (+ (fibonacci (- x 1)) (fibonacci (- x 2)))]))

(check-expect (fibonacci 1) 1)
(check-expect (fibonacci 10) 55)
(check-expect (fibonacci 10) (fibonacci-if 10))
```

Exercise 35 (What is `else`?)

`[else ...]` often appears as the last clause of a `match`. Predict `(match 5 [else else])`.

Solution. It is `5`. Inside `match`, `else` is not a keyword but a *pattern variable*, like `x`: it matches anything and is bound to what it matched. So here `else` is bound to 5 and the clause returns it. That is why `[else ...]` happens to work as a catch-all; but it is an accident, and it quietly binds a variable named `else` in the body. Use _ instead.

Destructuring lists. Two attempts at "the third element of a list":

```
(define (my-third x)
  (match x
    [(list a b c d) c]
    [_ x]))

(define (other-third lst)
  (match lst
    [(cons _ (cons _ (cons third _))) third]
    [_ (error "List too short")]))
```

Exercise 36 (Predict)

On which inputs do the two functions behave differently, and why?

Solution. `(list a b c d)` matches a list of *exactly* four elements. `(cons _ (cons _ (cons third _)))` matches any list of *at least* three: the last _ matches whatever rest is left, including `'()`.

input	`my-third`	`other-third`
`(list 2 3)`	`'(2 3)`	error `"List too short"`
`0`	`0`	error
`(list '(2 3) 2 4 5 6 7)`	the whole list	`4`
`(list 1 2 3 4)`	`3`	`3`

```
(check-expect (my-third (list 2 3)) (list 2 3))
(check-expect (my-third 0) 0)
(check-expect (my-third (list '(2 3) 2 4 5 6 7)) (list '(2 3) 2 4 5 6 7))
(check-expect (my-third (list 1 2 3 4)) 3)

(check-error (other-third (list 2 3)) "List too short")
(check-error (other-third 0) "List too short")
(check-expect (other-third (list '(2 3) 2 4 5 6 7)) 4)
(check-expect (other-third (list 1 2 3 4)) 3)
```

`my-third` is right only on four-element lists, and when it is wrong it does not fail: the fallback `[_ x]` hands back its input, so the caller gets a list or a number where it expected the third element. That is the same kind of silent wrong answer as the calculator parser turning `'(+ 1 2 3)` into $1 + 2$ (Section 3.3). Raising an error, as `other-third` does, is better.

Exercise 37 (Fix `my-third`)

Make `my-third` return the third element of any list with at least three elements and raise an error otherwise, using a `list` pattern. Hint: `_ ...` matches zero or more elements.

Solution.

```
(define (my-third-fixed x)
  (match x
    [(list _ _ c _ ...) c]
    [_ (error "List too short")]))

(check-error (my-third-fixed (list 2 3)) "List too short")
(check-expect (my-third-fixed (list 1 2 3)) 3)
(check-expect (my-third-fixed (list '(2 3) 2 4 5 6 7)) 4)
```

This is equivalent to `other-third`'s nested `cons` pattern, and easier to read.

3.4.3 `match` vs. `type-case`

The two halves used two tools for taking values apart. Each is strong where the other is weak:

	`match`	`type-case`
language	`racket`	`plai-typed`
nested patterns	yes: `(not-node (and-node b1 b2))`	no: one level; nest another `type-case`
literals, wildcards, lists	yes	no
missing case	run-time error, only if reached	rejected before the program runs

`match` is more expressive. `type-case` is more limited, and in exchange it can check, before anything runs, that every variant is handled. Recitation 4 (Section 4.5) implements a small `match` in our own interpreter.

Chapter 4

Week 4: Names and Functions

Lecture 4: September 29, 2026. Recitation 4: October 2, 2026.

Assigned reading and resources (course page):

- The course page lists no reading for this lecture.
- Lecture code: `lecture4a.rkt` to `lecture4d.rkt`. The course page notes that these files already contain the capture-avoiding substitution for `lambda` (Section 4.4), which was discussed at the start of Lecture 5, and that application can also be made lazy by substituting the unevaluated argument.

Further reading:

- [1] Chapter 5 (Adding Functions to the Language), which implements functions by substitution as this chapter does, and Chapter 6 (From Substitution to Environments) for where the notes go next.
- [5] Sections 5.2–5.3 (the untyped lambda calculus: free variables, substitution, and α-conversion), optional.

This chapter adds *names* to the calculator, and then *functions*. Both are implemented by substitution. The real subject, though, is how subtle substitution turns out to be. Three questions come up again and again: when to evaluate the expression being substituted, how inner bindings shadow outer ones, and how a careless substitution can *capture* a variable and change what a program means. The last section adds a `match` form to the same interpreter. This chapter covers:

- Section 4.1: `idC` and `letC`, eager substitution, and shadowing;
- Section 4.2: lazy substitution, its costs and benefits, and the variable-capture bug;
- Section 4.3: capture-avoiding substitution with fresh names from `gensym`;
- Section 4.4: `lambda` and application, a new `Value` type, eager and lazy application, and capture again, now also under eager evaluation;
- Section 4.5: Recitation 4, pattern matching in the interpreter.

4.1 Names: `let` and Substitution

4.1.1 The Plan

At the end of Chapter 3, the calculator had three forms: `numC`, `plusC`, and `timesC`. This section adds names, through a new form `(let x e1 e2)`. It means: compute `e1`, replace every `x` in `e2` with the result, and then compute `e2`.

```
(let x 5 (+ x x))
---> (+ 5 5)                 replace every x in the body with 5
---> 10

(let x (+ 1 5) (+ x x))
---> (let x 6 (+ x x))       e1 is evaluated first
---> (+ 6 6)
---> 12
```

4.1.2 Two New Variants

```
(define-type ArithE
  [numC (n : number)]
  [idC (x : symbol)]
  [plusC (e1 : ArithE) (e2 : ArithE)]
  [timesC (e1 : ArithE) (e2 : ArithE)]
  [letC (x : symbol) (e1 : ArithE) (e2 : ArithE)])
```

Section 4.4 will add two more variants for functions, and from then on every function on ArithE needs a case for them.

What is `idC` for?. An `idC` represents a *use* of a name, such as the `x` in `(+ x 1)`. We need it because of types. The operands of `plusC` must be ArithEs, and a bare symbol like `'x` is not one. Wrapping the symbol in `idC` turns it into an ArithE, so the name can sit in the tree until substitution replaces it. In this sense, `idC` does for symbols what `numC` does for numbers.

A name can appear in two roles, and only one of them becomes an `idC`:

```
;; (let x 5 (+ x 1))
(letC 'x (numC 5) (plusC (idC 'x) (numC 1)))
;;    ^ binding: a plain symbol    ^ use: an idC
```

The name right after `let` is the one being *bound*. It is stored as a plain symbol in `letC`, because it is never evaluated or replaced. Its only job is to tell substitution which name to look for.

Keywords like `+` and `let` never become `idC`s either, because `parse` uses them up when it decides which variant to build. Note also that `(plusC 'x (numC 1))` is a type error: `'x` is a symbol, not an ArithE.

Parsing. A lone symbol becomes an `idC`. In `(let x e1 e2)`, the `x` is kept as a plain symbol, and `e1` and `e2` are parsed recursively:

```
(define (parse (s : s-expression)) : ArithE
  (cond
    [(s-exp-number? s) (numC (s-exp->number s))]
    [(s-exp-symbol? s) (idC (s-exp->symbol s))]
    [(s-exp-list? s)
     (let [(l (s-exp->list s))]
       (case (s-exp->symbol (first l))
         [(+) (plusC (parse (second l)) (parse (third l)))]
         [(*) (timesC (parse (second l)) (parse (third l)))]
         [(let) (letC (s-exp->symbol (second l)) (parse (third l)) (parse (fourth l)))]))]))
```

As in Section 3.3, this parser does no error checking. For example, `(parse '(let x 1))` fails inside `fourth`, with the message `list contains too few elements`.

4.1.3 Eager Substitution

`(subst-eager v x e)` replaces every `x` in `e` with the number `v`. Most cases simply recurse into the children. The interesting cases are the two that involve names, `idC` and `letC`.

```
(define (subst-eager (v : number) (x : symbol) (e : ArithE)) : ArithE
  (type-case ArithE e
    [numC (n) (numC n)]
    [idC (y) (if (equal? x y) (numC v) (idC y))]
    [plusC (e1 e2) (plusC (subst-eager v x e1) (subst-eager v x e2))]
    [timesC (e1 e2) (timesC (subst-eager v x e1) (subst-eager v x e2))]
    [letC (y e1 e2)
          (letC y
                (subst-eager v x e1)
                (if (equal? x y) e2 (subst-eager v x e2)))]))
```

Why `(numC v)` and `(idC y)`?. `subst-eager` must return an ArithE, but `v` is a number. So when it finds the name, it wraps `v` as `(numC v)` before putting it back into the tree. When it finds a different name, it leaves that name alone: `(idC y)` rebuilds the same node, which is the same as returning `e`.

```
(subst-eager 5 'x (plusC (idC 'x) (idC 'y)))   ; = (plusC (numC 5) (idC 'y))
```

4.1.4 Shadowing

What is `(let x 5 (let x 6 (+ x x)))`: 12 or 10? The answer is 12. The inner `let` binds its own `x`, so inside its body, `x` means 6. We say the inner `x` *shadows* the outer one (Figure 4.1).

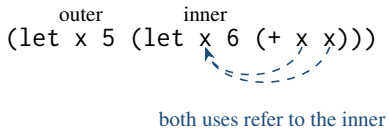


Figure 4.1: Each use of a name refers to the nearest enclosing binder of that name.

This is what the `(equal? x y)` check in the `letC` case is for. If an inner `let` binds the same name we are substituting for, substitution does not go into that `let`'s body.

```
(let x 5 (let x 6 (+ x x)))
---> (let x 6 (+ x x))       the outer subst leaves the inner body alone
---> (+ 6 6)
---> 12
```

Without the check, substitution would produce `(let x 6 (+ 5 5))`, which is 10.

The check only protects the inner *body*. The inner `e1` is still substituted, because it lies *outside* the inner binding: the inner `x` is visible only in the inner body.

```
(let x 1 (let x (+ x 1) x))
---> (let x (+ 1 1) x)       the x in e1 is the outer x
---> 2
```

4.1.5 `eval-eager`

```
(define (eval-eager (e : ArithE)) : number
  (type-case ArithE e
    [numC (n) n]
    [idC (x) (error 'eval "unbound variable")]
    [plusC (e1 e2) (+ (eval-eager e1) (eval-eager e2))]
    [timesC (e1 e2) (* (eval-eager e1) (eval-eager e2))]
    [letC (y e1 e2) (eval-eager (subst-eager (eval-eager e1) y e2))]))
```

The `letC` line follows the plan exactly: evaluate `e1`, substitute the resulting number for `y` in `e2`, and evaluate what comes out.

Why is the `idC` case an error? If a name is bound by some `let`, that `let` substitutes it away before `eval-eager` ever reaches it. So any `idC` that `eval-eager` does see was never bound, as in `(+ x 1)`, and that is an error.

```
(let x 5 (let y (+ x 1) (* x y)))
letC x: eval e1 = 5, subst 5 for x in the body
---> (let y (+ 5 1) (* 5 y))
letC y: eval e1 = 6, subst 6 for y in the body
---> (* 5 6)
---> 30
```

Each `let` removes the `idC`s for its own name. Once every `let` has been processed, no `idC` is left.

4.2 Lazy Substitution and Variable Capture

4.2.1 Lazy Substitution

There is another way to implement `let`: do not evaluate `e1` first. Instead, substitute the unevaluated tree into the body. It is then evaluated each time the body uses it.

```
(let x (+ 1 1) (+ x 2))
eager:  ---> (let x 2 (+ x 2))  ---> (+ 2 2)  ---> 4
lazy:   ---> (+ (+ 1 1) 2)                    ---> 4
```

`subst-lazy` is almost the same as `subst-eager`. The only difference is that `v` is now an `ArithE`, namely the unevaluated `e1`. Since it is already a tree, it goes into the body as is, with no `numC` wrapping:

```
(define (subst-lazy (v : ArithE) (x : symbol) (e : ArithE)) : ArithE
  (type-case ArithE e
    [numC (n) e]
    [idC (y) (if (equal? x y) v e)]
    [plusC (e1 e2) (plusC (subst-lazy v x e1) (subst-lazy v x e2))]
    [timesC (e1 e2) (timesC (subst-lazy v x e1) (subst-lazy v x e2))]
    [letC (y e1 e2)
          (letC y
                (subst-lazy v x e1)
                (if (equal? x y) e2 (subst-lazy v x e2)))]))

(define (eval-lazy (e : ArithE)) : number
  (type-case ArithE e
    [numC (n) n]
    [idC (x) (error 'eval "unbound variable")]
    [plusC (e1 e2) (+ (eval-lazy e1) (eval-lazy e2))]
    [timesC (e1 e2) (* (eval-lazy e1) (eval-lazy e2))]
    [letC (y e1 e2) (eval-lazy (subst-lazy e1 y e2))]))
```

Pitfall 38 (Mixing evaluators)

Every recursive call in `eval-lazy` must be to `eval-lazy`. Suppose one of them called `eval-eager` instead. The type checker would not complain, because both functions return a number. But inner `let`s would then silently become eager.

4.2.2 Is Lazy More Efficient?

It is when `x` is unused, because then `e1` is never evaluated.

```
(let x (+ 5 6) 10)
eager:  (let x 11 10) ---> 10      (+ 5 6) computed once, for nothing
lazy:   10                         (+ 5 6) never computed
```

This also changes which programs fail. If the language had `/`, then `(let x (/ 5 0) 10)` would not be an error in the lazy version. The same goes for an unbound name in `e1`: `(let x (+ y y) 10)` is an error eagerly, but 10 lazily.

When `x` is used more than once, lazy is *less* efficient. The reason is that `e1` is recomputed at every use:

```
(let x (+ 5 6) (+ (+ x x) x))
eager:  (+ 5 6) once     ---> (+ (+ 11 11) 11)
lazy:   (+ 5 6) 3 times  ---> (+ (+ (+ 5 6) (+ 5 6)) (+ 5 6))
```

4.2.3 Where Eager and Lazy Differ

program	eager	lazy
`(let x (+ y y) 10)`	`unbound variable`	`10`
`(let y x (let x 2 y))`	`unbound variable`	`2`

The first difference is harmless. Since `x` is unused, the lazy version never evaluates `e1`, so it never notices the unbound `y`. The second difference, however, is a real bug.

4.2.4 Variable Capture

```
(let x y (let y 2 (+ x y)))
lazy:
---> (let y 2 (+ y y))        the free y is captured by the inner let
---> (+ 2 2)
---> 4
```

The outer `y` is free, because nothing binds it. So the correct answer is an error. Racket agrees: the same program in `#lang racket` is rejected with `y: unbound identifier` before it runs.

What went wrong? Lazy substitution moved that free `y` underneath `(let y 2 ...)`. There, it now refers to the inner `y` (Figure 4.2).

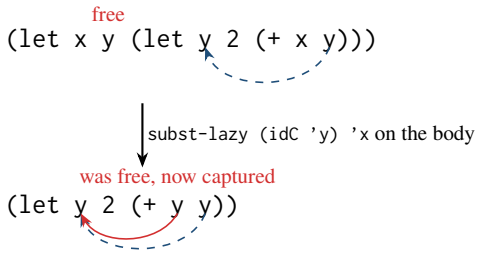


Figure 4.2: Capture. The substituted `y` (red arrow) ends up bound by an inner `let` it had nothing to do with.

To see that this is a bug, rename the inner `y` to `z` and run the program again:

```
(let x y (let z 2 (+ x z)))
lazy:
---> (let z 2 (+ y z))
---> (+ y 2)
---> eval: unbound variable
```

Key Point 39 (Names of bound variables should not matter)

The two programs differ only in renaming the inner `y` to `z`, so they must mean the same thing. Yet lazy substitution gives 4 for one and an error for the other. This bug is called *variable capture*, and a substitution that can cause it is called *capturing*.

`eval-eager` does not have this problem. It only ever substitutes numbers, and a number contains no names that could be captured. Lazy substitution is different: it substitutes whole expressions, and those can contain free variables. (Once values can be functions, eager evaluation substitutes code too, and capture comes back; see Section 4.4.4.)

4.3 Capture-Avoiding Substitution

4.3.1 Rename, Then Substitute

The fix is called *capture-avoiding substitution*. Before substituting into the body of an inner `let`, we rename that `let`'s bound variable to a fresh name, one that appears nowhere else. This is always allowed. Renaming a bound variable never changes what a program means, which was the point of the y/z example. So we are free to rename whenever it helps.

```
(let x y (let y 2 (+ x y)))
---> (let x y (let g3189 2 (+ x g3189)))    rename the inner y first
---> (let g3189 2 (+ y g3189))              the free y is no longer captured
---> (+ y 2)
---> eval: unbound variable                 same answer as the z version
```

The only remaining question is where the fresh name comes from.

4.3.2 `gensym`: Fresh Names

`gensym` ("generate symbol") is a Racket function. Every time it is called, it returns a brand-new symbol:

```
> (gensym)
'g3189
> (gensym)
'g3190
> (equal? (gensym) (gensym))
#f
```

A name chosen by hand, such as `'tmp`, does not work. The program might already use `tmp`. If it does, the renaming would create exactly the capture it was meant to avoid.

Definition 40 (Uninterned symbol)

An ordinary symbol is *interned*: `'y` is the same symbol wherever it is written, so `(equal? 'y 'y)` is `#t`. The symbol returned by `gensym` is *uninterned*: it is different from every other symbol, including one spelled the same way.

So even if a program literally contained the name `g3189`, that name would not be `equal?` to the `g3189` returned by `gensym`:

```
(define g (gensym))                              ; prints as 'g3189
(equal? g (string->symbol (symbol->string g)))   ; #f
```

This is why a gensym can never be captured by a name the user wrote, and can never capture one either.

Importing it into `plai-typed`. `plai-typed` does not provide `gensym`. We cannot simply import it from Racket, because the type checker cannot check a plain Racket import without knowing its type. `typed-in`

solves this. It imports a function from a Racket module and declares its type at the same time:

```
(require (typed-in racket/base [gensym : (-> symbol)]))
```

The type `(-> symbol)` means "takes no arguments and returns a symbol". The checker trusts this declaration. It does not check Racket's code.

4.3.3 `subst-lazy-v2`

Only the binding case changes. If the inner `let` binds the same name as the one being replaced, we stop, as before. Otherwise, we do three steps:

1. Let `z` be `(gensym)`, a name that appears nowhere else.
2. Rename `y` to `z` in the body, with `(subst-lazy (idC z) y e2)`. This step cannot cause capture, because no `let` inside `e2` binds the fresh `z`.
3. Substitute `v` for `x` in the renamed body.

```
(define (subst-lazy-v2 (v : ArithE) (x : symbol) (e : ArithE)) : ArithE
  (type-case ArithE e
    [numC (n) e]
    [idC (y) (if (equal? x y) v e)]
    [plusC (e1 e2) (plusC (subst-lazy-v2 v x e1) (subst-lazy-v2 v x e2))]
    [timesC (e1 e2) (timesC (subst-lazy-v2 v x e1) (subst-lazy-v2 v x e2))]
    [letC (y e1 e2)
          (if (equal? x y)
              (letC y (subst-lazy-v2 v x e1) e2)
              (let [(z (gensym))]
                (letC z
                      (subst-lazy-v2 v x e1)
                      (subst-lazy-v2 v x (subst-lazy (idC z) y e2)))))]))
```

```
(subst-lazy-v2 (idC 'y) 'x (parse '(let y 2 (+ x y))))
;; = (letC 'g3445 (numC 2) (plusC (idC 'y) (idC 'g3445)))
```

The inner `y` became `g3445`. The `y` that came from outside is still free, so evaluation reports it as unbound. Now the `y` version and the `z` version of the program agree.

The accompanying code, `lec4c.rkt`, splits the `letC` case three ways instead of two:

- `x` equals `y`: stop;
- `y` is `possibly-free?` in `v`: rename;
- otherwise: substitute without renaming.

However, its `possibly-free?` is a stub that always returns `#t`. So in practice it behaves exactly like the two-way code above. Implementing `possibly-free?` properly, so that renaming happens only when it is needed, is left as an exercise (see the definition of free variables, Definition 50).

Remark 41. Step 2 uses the old `subst-lazy`, which can capture in general. Here it is safe. The expression being substituted, `(idC z)`, contains only the fresh name `z`, and nothing in `e2` can bind that name.

4.4 Functions: `lambda` and Application

4.4.1 Syntax

This section adds functions to the language, through two new forms:

syntax	meaning	abstract syntax
`(lambda x e)`	a function with parameter `x` and body `e`	`lambdaC`
`(e1 e2)`	call the function `e1` on the argument `e2`	`appC`

```
(define-type ArithE
  [numC (n : number)]
  [idC (x : symbol)]
  [plusC (e1 : ArithE) (e2 : ArithE)]
  [timesC (e1 : ArithE) (e2 : ArithE)]
  [letC (x : symbol) (e1 : ArithE) (e2 : ArithE)]
  [lambdaC (x : symbol) (e : ArithE)]
  [appC (e1 : ArithE) (e2 : ArithE)])
```

`lambdaC` *defines* a function, and `appC` *uses* one. The pair works much like `letC` and `idC`: one introduces a name, the other uses it. Every function takes exactly one argument. Several arguments can be simulated by currying (Section 2.3).

Parsing. A list whose head is not `+`, `*`, `let`, or `lambda` is an application:

```
(define (parse (s : s-expression)) : ArithE
  (cond
    [(s-exp-number? s) (numC (s-exp->number s))]
    [(s-exp-symbol? s) (idC (s-exp->symbol s))]
    [(s-exp-list? s)
     (let [(l (s-exp->list s))]
       (if (s-exp-symbol? (first l))
           (case (s-exp->symbol (first l))
             [(+) (plusC (parse (second l)) (parse (third l)))]
             [(*) (timesC (parse (second l)) (parse (third l)))]
             [(let) (letC (s-exp->symbol (second l)) (parse (third l)) (parse (fourth l)))]
             [(lambda) (lambdaC (s-exp->symbol (second l)) (parse (third l)))]
             [else (appC (parse (first l)) (parse (second l)))])
           ;; head is not a symbol, e.g. ((lambda x (+ x 1)) 5)
           (appC (parse (first l)) (parse (second l)))))]))
```

```
(parse '((lambda x (+ x 1)) 5))
;; = (appC (lambdaC 'x (plusC (idC 'x) (numC 1))) (numC 5))
(parse '(f 5))
;; = (appC (idC 'f) (numC 5))
```

The outer `if` handles calls whose head is itself a list, such as `((lambda x (+ x 1)) 5)`. Without it, `parse` would call `s-exp->symbol` on that list and fail with `s-exp->symbol: not a symbol`.

Remark 42. Because applications are the fallback, `'(- 1 2)` no longer fails to parse. The `else` branch reads

it as a call of a function named -. The result is (appC (idC '-) (numC 1)), and the 2 is silently dropped. Before lambda was added, the same input failed with case: no matching clause.

4.4.2 lambda in Substitution

A lambda binds its parameter in its body, exactly as let binds x in e2. So the lambdaC case of each substitution function is its letC case without an e1. It stops at shadowing, and in subst-lazy-v2 it first renames the parameter with gensym. The application case simply recurses.

```
;; in subst-eager (subst-lazy is the same with subst-lazy)
[lambdaC (y e)
         (if (equal? x y) (lambdaC y e) (lambdaC y (subst-eager v x e)))]
[appC (e1 e2) (appC (subst-eager v x e1) (subst-eager v x e2))]

;; in subst-lazy-v2
[lambdaC (y e)
         (if (equal? x y)
             (lambdaC y e)
             (let [(z (gensym))]
               (lambdaC z (subst-lazy-v2 v x (subst-lazy (idC z) y e)))))]
[appC (e1 e2) (appC (subst-lazy-v2 v x e1) (subst-lazy-v2 v x e2))]
```

Pitfall 43 (The pattern variable shadows the argument)

In [lambdaC (y e) ...], the pattern variable e is the body. It shadows subst's own argument e, which is the whole lambda. So the shadowing branch must rebuild (lambdaC y e). Returning plain e would drop the lambda and return only its body. Shadowing bites the interpreter's author too.

4.4.3 A New Result Type

A program can now produce a function, for example (lambda x (+ x 1)) or (let f (lambda x x) f). A function is not a number, so evaluation must return a more general type. That type is Value, which is one of two things:

```
(define-type Value
  [numV (n : number)]
  [funV (x : symbol) (e : ArithE)])
```

+ and * only make sense on numbers. So they first check that both operands are numVs, unwrap them, and

report an error for anything else:

```
(define (num+ (v1 : Value) (v2 : Value)) : Value
  (if (and (numV? v1) (numV? v2))
      (numV (+ (numV-n v1) (numV-n v2)))
      (error 'eval "not a number")))

(define (num* (v1 : Value) (v2 : Value)) : Value
  (if (and (numV? v1) (numV? v2))
      (numV (* (numV-n v1) (numV-n v2)))
      (error 'eval "not a number")))
```

This is the first *dynamic* type error in our language. Whether `(+ e1 e2)` makes sense depends on what `e1` and `e2` evaluate to, and the interpreter only learns that at run time.

4.4.4 Eager Application

We first add functions to the *eager* interpreter. Here application is *call-by-value*: evaluate the function, evaluate the argument, and then substitute the argument's *value* into the body. `let` works the same way.

There is one obstacle. `subst-eager` can only substitute numbers, but a value may now be a function. So the interpreter turns the value back into an expression, and substitutes it with `subst-lazy` or `subst-lazy-v2`, which substitute expressions. Nothing about this is lazy, because the expression they receive has already been evaluated:

```
(define (value->expr (v : Value)) : ArithE
  (type-case Value v
    [numV (n) (numC n)]
    [funV (x e) (lambdaC x e)]))

(define (eval-eager-v2 (e : ArithE)) : Value
  (type-case ArithE e
    [numC (n) (numV n)]
    [idC (x) (error 'eval "unbound variable")]
    [plusC (e1 e2) (num+ (eval-eager-v2 e1) (eval-eager-v2 e2))]
    [timesC (e1 e2) (num* (eval-eager-v2 e1) (eval-eager-v2 e2))]
    [letC (y e1 e2)
          (eval-eager-v2 (subst-lazy (value->expr (eval-eager-v2 e1)) y e2))]
    [lambdaC (x e) (funV x e)]
    [appC (e1 e2)
          (type-case Value (eval-eager-v2 e1)
            [funV (x e)
                  (eval-eager-v2 (subst-lazy (value->expr (eval-eager-v2 e2)) x e))]
            [numV (n) (error 'eval "not a function")])]))
```

The accompanying code, `lec4d.rkt`, uses different names: it writes `Expr` for ArithE and `lambdaV` for `funV`. It also passes `subst` a Value and converts it at the `idC` case. The effect is the same.

With this, functions can be passed around:

```
(let do-twice (lambda f (f (f 1)))
  (let double (lambda x (* 2 x))
    (do-twice double)))
---> (do-twice double)  with both substituted
---> (double (double 1))
---> 4
```

Eager capture. Section 4.2 argued that eager substitution cannot capture, because a number contains no names. A function value, however, does contain names, and its body may contain free variables:

```
(let f (lambda x (+ x y))         y is free in f's body
  (let y 6
    (f 1)))
---> (let y 6 ((lambda x (+ x y)) 1))   substitute f's value: y is captured
---> ((lambda x (+ x 6)) 1)
---> 7
```

The right answer is an error. The `y` in `f` refers to no binding, and Racket rejects the same program with `y: unbound identifier`. `eval-eager-v2` returns 7 because `subst-lazy` carried the free `y` under `(let y 6 ...)`. So once values can contain code, eager evaluation no longer protects against capture.

The fix is the same as in Section 4.3. `eval-eager-v3` is `eval-eager-v2` with both calls to `subst-lazy` replaced by `subst-lazy-v2`. Now the program above raises `unbound variable`. Returning `f` instead of calling it shows the renaming:

```
> (eval-eager-v3 (parse '(let f (lambda x (+ x y)) (let y 6 f))))
(funV 'g3507 (plusC (idC 'g3507) (idC 'y)))
```

When `f` was substituted into the body of `(let y 6 ...)`, its parameter was renamed to a fresh name. The `lambdaC` case of `subst-lazy-v2` always does this. The free `y` stays free.

4.4.5 Lazy Application: `eval-lazy-v2`

The last version makes application lazy, just like `let` in Section 4.2. The argument is substituted without being evaluated.

```
(define (eval-lazy-v2 (e : ArithE)) : Value
  (type-case ArithE e
    [numC (n) (numV n)]
    [idC (x) (error 'eval "unbound variable")]
    [plusC (e1 e2) (num+ (eval-lazy-v2 e1) (eval-lazy-v2 e2))]
    [timesC (e1 e2) (num* (eval-lazy-v2 e1) (eval-lazy-v2 e2))]
    [letC (y e1 e2) (eval-lazy-v2 (subst-lazy-v2 e1 y e2))]
    [lambdaC (x e) (funV x e)]
    [appC (e1 e2)
          (type-case Value (eval-lazy-v2 e1)
            [funV (x e) (eval-lazy-v2 (subst-lazy-v2 e2 x e))]
            [numV (n) (error 'eval "not a function")])]))
```

- **lambdaC**: a lambda is already a value. Nothing needs to be computed until it is called.
- **appC**: first evaluate e1, which must give a function (funV x e). Then call it the way let works: substitute the argument e2 for x in the body e, and evaluate the result. Since this is the lazy version, e2 is substituted unevaluated.

```
((lambda x (+ x 1)) 5)
---> (+ 5 1)             subst 5 for x in the body
---> 6
```

Key Point 44 (let is application)

((lambda x e) e2) means the same as (let x e2 e). What lambda adds is that the function is a *value*. It can be named with let, passed as an argument, and returned as a result.

program	result
(let f (lambda x (* x 2)) (f 5))	(numV 10)
((lambda f (f 3)) (lambda x (+ x 1)))	(numV 4): a function as argument
(((lambda x (lambda y (+ x y))) 1) 2)	(numV 3): a function as result
((lambda x 5) y)	(numV 5): the unused argument is never evaluated
the do-twice example of Section 4.4.4	(numV 4)
(5 3)	not a function
(+ (lambda x x) 1)	not a number

4.4.6 Capture, Again

With functions, capture-avoiding substitution matters even more. A function can return a function, so a free name can end up under another lambda:

```
(((lambda x (lambda y (+ x y))) y) 2)
subst-lazy:     (lambda y (+ y y)),  then 2 ---> 4          captured!
subst-lazy-v2:  (lambda g (+ y g)),  then 2 ---> (+ y 2) ---> unbound variable
```

The earlier interpreters eval-eager and eval-lazy still return a number, and subst-eager can only substitute numbers. So their lambdaC and appC cases remain errors ("TODO: lambda"). The versions that support functions are eval-eager-v2/v3 and eval-lazy-v2.

program	eval-eager-v3 (call-by-value)	eval-lazy-v2 (call-by-name)
((lambda x 5) y)	unbound variable	(numV 5)
(let x (+ y y) 10)	unbound variable	(numV 10)
the f/y example of Section 4.4.4	unbound variable	unbound variable

With capture-avoiding substitution, the two strategies differ only when an argument is never used, just as for let in Section 4.2. Racket, like most languages, is call-by-value.

4.5 Recitation 4: Pattern Matching in the Interpreter

In Section 3.4, we *used* Racket's `match` to write `factorial` and `fibonacci`. This section *implements* a small version of it. We add a `match` form to the calculator with `let`, and extend each part of the interpreter in turn: the types, `parse`, `subst`, and `eval`. The code starts with the `gensym` import of Section 4.3, because its `let` uses capture-avoiding lazy substitution.

4.5.1 The New Form

A pattern-matching expression looks like this:

```
(match e (n1 e1) (n2 e2) ... (_ en))
```

Each `ni` is a number literal, and _ is the wildcard. To evaluate the expression:

1. evaluate `e` to a number, once;
2. try the clauses from top to bottom: `ni` matches if it equals that number, and _ matches anything;
3. the *first* matching clause wins, and only its expression is evaluated;
4. if no clause matches, it is an error.

This is Racket's `match`, restricted to literals and _. That is just enough for definitions in the style of `factorial` and `fibonacci`. There are no pattern variables, so a clause never binds a name. This fact will matter for substitution.

4.5.2 Types

A clause pairs a pattern with an expression. The two kinds of clause carry different data: a number pattern has a number, and the wildcard has nothing. So each kind gets its own variant:

```
(define-type PatternE
  [numP (n : number) (e : ArithE)]
  [anyP (e : ArithE)])

(define-type ArithE
  [numC (n : number)]
  [plusC (e1 : ArithE) (e2 : ArithE)]
  [timesC (e1 : ArithE) (e2 : ArithE)]
  [letC (x : symbol) (e1 : ArithE) (e2 : ArithE)]
  [idC (x : symbol)]
  [patternC (e : ArithE) (patterns : (listof PatternE))])
```

`patternC` holds two things: the expression being matched, called the *scrutinee*, and the clauses in order.

The two types are *mutually recursive*. An ArithE can contain PatternEs, and each PatternE contains an ArithE. `define-type` allows this, and the two definitions can come in either order.

Examples. Here are four examples, in ML-like notation:

	program	expected
1	`match 5 with | 5 -> 0 | 6 -> 1`	0
2	`match (1 + 2) with | 3 -> 1 | _ -> x`	1
3	`match x with | 0 -> z | _ -> (match y with | 0 -> x | 1 -> y)`	unbound variable
4	`let x = 1 in let y = 1 in` (the `match` of 3, last clause `y + 1`)	2

```
(define example-2 : ArithE
  (patternC (plusC (numC 1) (numC 2))
            (list (numP 3 (numC 1))
                  (anyP (idC 'x)))))

(define example-4 : ArithE
  (letC 'x (numC 1)
        (letC 'y (numC 1)
              (patternC (idC 'x)
                        (list (numP 0 (idC 'z))
                              (anyP (patternC (idC 'y)
                                              (list (numP 0 (idC 'x))
                                                    (numP 1 (plusC (numC 1) (idC 'y))))))))))
```

Examples 2 and 4 both contain a name that nothing binds: `x` in example 2 and `z` in example 4. Yet neither program is an error, because the clause containing that name is never chosen. Example 3 is different. Its error comes from the scrutinee `x`, and the scrutinee is always evaluated.

4.5.3 Parsing

The concrete syntax is `(match e clause ...)`. So after the `match` keyword, `(second l)` is the scrutinee, and `(rest (rest l))` is the list of clauses. A helper turns that list of s-expressions into a list of PatternEs. Each

clause is a two-element list `(pattern expr)`, and the pattern decides which variant to build:

```
(define (parse-patterns (s-pats : (listof s-expression))) : (listof PatternE)
  (cond
    [(empty? s-pats) empty]
    [else
     (let [(s-pat (s-exp->list (first s-pats)))]
       (cond
         [(s-exp-number? (first s-pat))
          (cons (numP (s-exp->number (first s-pat)) (parse (second s-pat)))
                (parse-patterns (rest s-pats)))]
         [(equal? (s-exp->symbol (first s-pat)) '_)
          (list (anyP (parse (second s-pat))))]))]))

(define (parse (s : s-expression)) : ArithE
  (cond
    [(s-exp-number? s) (numC (s-exp->number s))]
    [(s-exp-symbol? s) (idC (s-exp->symbol s))]
    [(s-exp-list? s)
     (let [(l (s-exp->list s))]
       (case (s-exp->symbol (first l))
         [(+) (plusC (parse (second l)) (parse (third l)))]
         [(*) (timesC (parse (second l)) (parse (third l)))]
         [(let) (letC (s-exp->symbol (second l)) (parse (third l)) (parse (fourth l)))]
         [(match) (patternC (parse (second l)) (parse-patterns (rest (rest l))))]))]))
```

Three details of this parser are worth noting:

- Parsing *stops* at the first wildcard. A number clause is consed onto the rest of the clauses, but a `_` clause returns just `(list (anyP ...))`. Any clause after a wildcard could never be reached, so dropping it does not change the meaning of the program. For example, `(parse '(match 1 (_ 0) (2 3)))` has only one clause.
- The pattern is *not* parsed as an expression. It must be a number literal or `_`, and anything else is an error. This function does not raise that error itself, though. Instead, `(match 3 ((+ 1 2) 0))` fails inside `s-exp->symbol` (`not a symbol`), and `(match 3 (n 0))` falls off the end of the inner `cond` (`no matching clause`).
- `parse-patterns` and `parse` call each other, just as the two types refer to each other. Note that `match` is a symbol in *our* language, and `case` matches it the same way it matches `+` and `let`.

4.5.4 Substitution

Exercise 45 (Why substitute into `match`?)

Why do we need to define substitution for pattern-matching expressions?

Solution. Because `let` is implemented by substitution, and the body of a `let` can contain a `match`. In example 4, the `x` and `y` bound by the `let`s are used both in the scrutinees and in the clause bodies. If `subst` stopped at a `patternC`, those `idC`s would never be replaced, and `eval` would report them as unbound. (In `plai-typed` the question does not really arise: `type-case` rejects a `subst` that has no `patternC` case.)

Substituting into a clause means substituting into its expression. The pattern itself is a number or _, so it contains no names:

```
(define (subst-pattern (erep : ArithE) (x : symbol)) : (PatternE -> PatternE)
  (lambda (p)
    (type-case PatternE p
      [numP (n e) (numP n (subst erep x e))]
      [anyP (e) (anyP (subst erep x e))])))
```

`subst-pattern` is *curried*. It takes `erep` and `x`, and returns a function from one clause to one clause. That is exactly the shape `map` wants, so the `patternC` case fits on one line:

```
(define (subst (erep : ArithE) (x : symbol) (e : ArithE)) : ArithE
  (type-case ArithE e
    [idC (y) (if (equal? x y) erep (idC y))]
    [numC (n) (numC n)]
    [plusC (e1 e2) (plusC (subst erep x e1) (subst erep x e2))]
    [timesC (e1 e2) (timesC (subst erep x e1) (subst erep x e2))]
    [patternC (e pats) (patternC (subst erep x e) (map (subst-pattern erep x) pats))]
    [letC (y e1 e2)
          (cond
            [(equal? x y) (letC y (subst erep x e1) e2)]
            [else
             (let [(z (gensym))]
               (letC z (subst erep x e1) (subst erep x (subst (idC z) y e2))))])]))
```

The `letC` case is the capture-avoiding `subst-lazy-v2` of Section 4.3.

Note what the `patternC` case does *not* need: a shadowing check or renaming. `letC` needs those because a `let` binds a name. Our patterns bind nothing, so a clause body sees exactly the same names as the `match` around it. In this respect, clauses behave like the children of `plusC`.

4.5.5 Evaluation

The helper `get-pat-expr` takes two arguments: the number the scrutinee evaluated to, and the clauses. It returns the *expression* of the first clause that matches:

```
(define (get-pat-expr (num : number) (pats : (listof PatternE))) : ArithE
  (cond
    [(empty? pats) (error 'get-pat-expr "unmatched expression")]
    [else (type-case PatternE (first pats)
            [numP (n e) (cond [(equal? num n) e] [else (get-pat-expr num (rest pats))])]
            [anyP (e) e])]))

(define (eval (e : ArithE)) : number
  (type-case ArithE e
    [numC (n) n]
    [plusC (e1 e2) (+ (eval e1) (eval e2))]
    [timesC (e1 e2) (* (eval e1) (eval e2))]
    [patternC (e pats) (eval (get-pat-expr (eval e) pats))]
    [letC (x e1 e2) (eval (subst e1 x e2))]
    [idC (x) (error 'eval "unbound variable!")]))
```

`eval` evaluates the scrutinee once, picks the clause, and then evaluates only that clause's expression.

Why return the expression, not its value?. `get-pat-expr` could instead evaluate every clause and return numbers. That would be wrong, not just slow. Example 4 has a clause `(0 z)`, and evaluating it would raise "unbound variable", even though the scrutinee is 1 and that clause is never chosen. Returning the unevaluated ArithE means that only the chosen clause is evaluated. This is the same reason only one branch of an `if` is evaluated.

Tracing example 4. In the trace, `g1` stands for whatever name `gensym` returns.

```
(let x 1 (let y 1 (match x (0 z) (_ (match y (0 x) (1 (+ 1 y)))))))
letC x:  subst (numC 1) for x in the body.
         The inner let binds y, not x, so y is renamed to g1 first.
---> (let g1 1 (match 1 (0 z) (_ (match g1 (0 1) (1 (+ 1 g1))))))
letC g1: subst (numC 1) for g1 in the body
---> (match 1 (0 z) (_ (match 1 (0 1) (1 (+ 1 1)))))
patternC: scrutinee = 1; (0 z) fails; _ matches
---> (match 1 (0 1) (1 (+ 1 1)))
patternC: scrutinee = 1; (0 1) fails; (1 ...) matches
---> (+ 1 1)
---> 2
```

The clause `(0 z)` is skipped without being evaluated. This is why the free `z` is harmless.

Example 3, in contrast, fails at its very first step. Its scrutinee is `(idC 'x)`, no `let` has replaced it, and so `eval` reports it as unbound.

Clause order. `get-pat-expr` walks the list from the front and stops at the first match. So the order of clauses matters, and a clause after _ is dead code:

```
(test (eval (parse '(match 5 (_ 0) (5 1)))) 0)
(test (eval (parse '(match 5 (5 1) (_ 0)))) 1)
(test (eval (parse '(match 5 (5 1) (5 2)))) 1)
(test (eval (parse '(match (* 2 3) (5 0) (6 1) (_ 2)))) 1)
(test/exn (eval (parse '(match 7 (5 0) (6 1)))) "unmatched expression")
```

The last test is the run-time "no matching clause" error of Section 3.4, now in our own language. Nothing can check for this error ahead of time, because whether a clause matches depends on a number that is known only when the program runs.

Chapter 5

Week 5: From Substitution to Environments

Lecture 5: October 6, 2026. Recitation 5: October 9, 2026.

Assigned reading and resources (course page):

- [1] Chapter 6 (From Substitution to Environments) and Chapter 7 (Functions Anywhere), which make exactly this chapter's move and hit the same dynamic-scope bug on the way.
- Lecture code: `lecture05.zip` (see the descriptions at the tops of the files).

Further reading:

- [3] Section 3.2 (The Environment Model of Evaluation), for the same idea drawn as frames and pointers.
- [7] Chapter 3 (Names, Scopes, and Bindings), for static and dynamic scope with assignment and for macros (Section 5.5).
- [5] Sections 5.1–5.3 (the untyped lambda calculus), optional.

Chapter 4 implemented names and functions by *substitution*: rewrite the program, replacing a name by its value, before running it. This chapter first finishes that story, then keeps the language and changes the implementation. An *environment* interpreter never rewrites the program; it carries a table from names to values and looks a name up when it reaches it. We build it in five small versions, called 5a to 5e below, and twice the first attempt gets scope wrong in the same way. The last section revisits environments and scope, and carries the scope question over to assignment and macros. This chapter covers:

- Section 5.1: `let` as function application, capture once more, the lambda calculus, and α-equivalence;
- Section 5.2: an environment interpreter for `let`, checked against substitution (version 5a);
- Section 5.3: lazy `let` with environments, dynamic scope, and the fix (versions 5b and 5c);
- Section 5.4: functions with environments, and closures (versions 5d and 5e);
- Section 5.5: Recitation 5, environments as functions, one interpreter for both scopes, scope with assignment, and hygienic macros.

5.1 Substitution Revisited

5.1.1 The Starting Point

We start where Chapter 4 ended: the eager interpreter with functions, `eval-eager-v3` of Section 4.4.4. A call evaluates the function and the argument, then substitutes the argument's value into the body with the capture-avoiding `subst-lazy-v2` of Section 4.3; function values are written `(lambdaV x e)` here, `funV` in Section 4.4. The two binding forms are

```
[letC (y e1 e2)
      (eval (subst e1 y e2))]
[appC (e1 e2)
      (let ([v1 (eval e1)]
            [v2 (eval e2)])
        (eval (subst (val->expr v2) (lambdaV-x v1) (lambdaV-e v1))))]
```

where `val->expr` turns a value back into an expression, as `value->expr` did in Section 4.4.

Remark 46. The `letC` line above is a *lazy* `let` next to an *eager* application. The substitution interpreter kept for comparison in Section 5.2.6 evaluates `e1` first, and the rest of this chapter takes `let` to be eager whenever application is. Mixing the two strategies would break the equation of the next subsection.

5.1.2 `let` Is Function Application

Key Point 47 (`let` desugars to `lambda`)

`(let x e1 e2)` means `((lambda x e2) e1)`.

"Call `e1` by the name `x` while computing `e2`" is the same as "make the function with parameter `x` and body `e2`, and call it on `e1`".

Both sides evaluate `e1`, substitute it for `x` in `e2`, and evaluate the result:

```
(let x 5 (+ x x))          ((lambda x (+ x x)) 5)
---> (+ 5 5)               ---> (+ 5 5)
---> 10                    ---> 10
```

So `letC` is not needed in the core language: `parse` could turn every `let` into an `appC` of a `lambdaC`, and `eval` and `subst` would lose a case each. This is *desugaring*. The equation holds only if `let` and application use the same strategy, both eager or both lazy. With a lazy `let` and an eager call, `(let x (+ y y) 10)` is 10 but `((lambda x 10) (+ y y))` is an unbound-variable error.

Step by step. Under eager evaluation, the two sides do the same work in the same order:

```
(let x e1 e2)                  ((lambda x e2) e1)
1. evaluate e1 to a value v    1. evaluate (lambda x e2): a function value
                               2. evaluate e1 to a value v
2. bind x to v, evaluate e2    3. bind x to v, evaluate the body e2
```

Step 1 on the right does no real work. Evaluating a `lambda` only packages the parameter and the body into a value. After that, both sides compute `e1` and then compute `e2` with `x` standing for its value.

The scopes match as well. On both sides, `x` is visible in `e2` and nowhere else. A `let` is defined that way, and in the application `e1` lies outside the `lambda`, so it cannot see the parameter. So every use of a name points to the same binder on both sides, and the two programs have the same binding arrows.

Where the binding happens. A `lambda` only *declares* its parameter. The binding happens when the function is *called*, because only then is the argument's value known. In the interpreter, that is the last line of the `appC` case. In the substitution interpreter above, binding `x` to `v` means substituting `v` for `x` in the body:

```
(eval (subst (val->expr v2) (lambdaV-x v1) (lambdaV-e v1)))
;      bind: replace the parameter by the argument's value in the body
```

In the environment interpreter of Section 5.4, binding means extending the environment instead:

```
(eval-env (extend-env (bind (closV-x v1) v2) (closV-env v1))
          (closV-e v1))
;          bind: add parameter = argument's value to the saved environment
```

The two forms differ only in where the name and the value come from:

	`let`	`lambda` application
binding happens in	the `letC` case	the `appC` case
the name comes from	the `letC` field	the parameter, kept in the function value
the value comes from	evaluating `e1`	evaluating the argument

Several bindings: `let` and `let*`. The same idea extends to forms that bind several names at once, such as the `let` and `let*` of Problem Set 2 (written with brackets, as in Racket). Here the difference between the two forms becomes visible.

A multi-binding `let` is a call to a function with several parameters:

```
(let ([x1 e1] ... [xn en]) body)   means   ((lambda (x1 ... xn) body) e1 ... en)
```

All the arguments `e1 ... en` sit outside the `lambda`, so none of them can see any `xi`. Each `ei` is evaluated in the outer scope, which is exactly the rule for `let`.

A `let*` is a chain of one-binding `let`s, one inside the next, and so a chain of one-parameter `lambda`s:

```
(let* ([x1 e1] [x2 e2]) body)
  means   (let ([x1 e1]) (let ([x2 e2]) body))
  means   ((lambda (x1) ((lambda (x2) body) e2)) e1)
```

Now `e2` sits *inside* the `lambda` for `x1`, so it can see `x1`. That is exactly the rule for `let*`: each `ei` sees the bindings before it.

Example 48. The two programs below differ only in `let` versus `let*`:

```
(let ([x 5]) (let  ([x (* x 2)] [y x]) (+ x y)))    ; 15
(let ([x 5]) (let* ([x (* x 2)] [y x]) (+ x y)))    ; 20
```

Desugar the inner forms:

```
;; let: both arguments are evaluated outside the lambda, where x = 5
((lambda (x y) (+ x y)) (* x 2) x)
---> ((lambda (x y) (+ x y)) 10 5)
---> (+ 10 5) = 15

;; let*: the second binding sits inside the first lambda, where x = 10
((lambda (x) ((lambda (y) (+ x y)) x)) (* x 2))
---> ((lambda (x) ((lambda (y) (+ x y)) x)) 10)
---> ((lambda (y) (+ 10 y)) 10)
---> (+ 10 10) = 20
```

In the `let` version, the argument `x` that becomes `y` is outside the `lambda`, so it is the outer `x`, which is 5. In the `let*` version, it is inside the `lambda` for the new `x`, so it is 10.

Pitfall 49 (Nesting one-parameter `lambda`s gives `let*`, not `let`)

It is tempting to desugar a multi-binding `let` into nested one-parameter `lambda`s. That changes its meaning: every later argument moves inside an earlier `lambda` and can now see the earlier names. A `let` needs a single call with all the arguments outside, or some other way to evaluate every `ei` in the outer scope.

In an interpreter, the two rules show up as the environment each `ei` is evaluated in. For `let`, every `ei` is evaluated in the environment of the `let` itself, and the new bindings are added all at once before the body. For `let*`, the environment is extended after each binding, and the next `ei` is evaluated in the extended one.

5.1.3 Capture, Once More

A capture example:

```
(let x (+ 1 y) (let y 2 (+ x y)))
```

Nothing binds the first `y`, so the right answer is an unbound-variable error (Figure 5.1). With a lazy `let`, `(+ 1 y)` is substituted unevaluated for `x`, and the question is what happens to its `y` when it lands inside `(let y 2`

`...`):

```
without renaming (subst-lazy):
---> (let y 2 (+ (+ 1 y) y))      the free y is captured
---> (+ (+ 1 2) 2)
---> 5                            wrong

with renaming (subst-lazy-v2):
---> (let z 2 (+ (+ 1 y) z))      inner y renamed to a fresh z first
---> (+ (+ 1 y) 2)
---> unbound variable             right
```

The second trace ends in `(let z 2 (+ (+ 1 y) z))`. It is the example of Section 4.2 `(let x y (let y 2 (+ x y)))` with `y` replaced by `(+ 1 y)`: any substituted expression that has a free variable can be captured, however complicated it is. An eager `let` reports the error even earlier, while evaluating `(+ 1 y)` before any substitution.

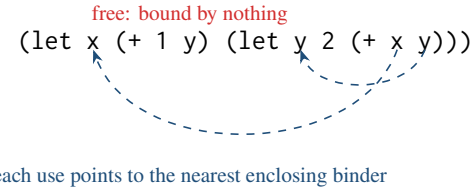


Figure 5.1: Lexical binding structure of the capture example. The `y` inside `(+ 1 y)` is free; a correct substitution must keep it free after moving it under `(let y 2 ...)`.

5.1.4 The Lambda Calculus

Delete `numC`, `plusC`, `timesC`, and `letC` from the expression type. What remains is Church's *lambda calculus* (1936), usually written with λ:

lambda calculus	our syntax	abstract syntax
x	`x`	`idC`
$\lambda x.\, e$	`(lambda x e)`	`lambdaC`
$e_1\ e_2$	`(e1 e2)`	`appC`

It has a single computation rule, *β-reduction*:

$$(\lambda x.\, e)\ e_2 \longrightarrow e[e_2/x],$$

which is the `appC` case of the lazy interpreter: substitute the argument for the parameter. Despite having nothing but functions, the lambda calculus can express every computation: numbers, Booleans, pairs, and recursion can all be encoded as functions. The `let` desugaring above is the first such encoding: a `let` is a β-redex.

5.1.5 α-Equivalence

Definition 50 (Free and bound variables)

A use of a name is *bound* if some enclosing binder (a `let`, a `lambda`, or any other form that introduces names) binds that name; it refers to the nearest such binder. A use with no enclosing binder of its name is *free*. $\mathrm{FV}(e)$ is the set of names that occur free in e.

For example, in `(let y 2 (+ x y))` the `y` in the body is bound and the `x` is free, so $\mathrm{FV} = \{\texttt{x}\}$.

Definition 51 (α-equivalence)

Two expressions are *α-equivalent* if they differ only in the names of bound variables, renamed consistently together with their uses. Free variables must stay as they are.

`(lambda x (+ x 1))`	`(lambda y (+ y 1))`	α-equivalent
`(let y 2 (+ x y))`	`(let z 2 (+ x z))`	α-equivalent
`(lambda x (+ x y))`	`(lambda x (+ x z))`	*not*: `y` and `z` are free
`(lambda x (lambda y x))`	`(lambda y (lambda y y))`	*not*: renaming `x` to `y` caused capture

To decide whether two expressions are α-equivalent, draw for each variable use an arrow to the binder it refers to, and mark the uses that have no binder as free. The two expressions are α-equivalent exactly when the free variables have the same names in the same places and every arrow points to the binder in the corresponding place. A rename is safe when the new name is *fresh*: it does not occur free in the binder's scope, so no use that pointed outward gets captured by the renamed binder.

Exercise 52 (Checking renamings)

Which of the following are α-equivalent to `(* a ((lambda (b) (+ a b)) 3))`?

(A) `(* a ((lambda (c) (+ a c)) 3))`

(B) `(* c ((lambda (b) (+ c b)) 3))`

(C) `(* a ((lambda (a) (+ a a)) 3))`

Solution. In the original, `a` is free (both uses) and `b` is bound by the `lambda`.

(A) α-equivalent. The bound `b` is renamed to `c` at the binder and at its use; `c` is fresh, and the free `a` is untouched.

(B) *Not*. Both uses of the free `a` became `c`. Renaming a free variable changes which outside value the expression depends on.

(C) *Not*. Renaming `b` to `a` is not fresh: `a` occurs free in the body `(+ a b)`. After the rename, that `a` is captured by `(lambda (a) ...)`, so the function returns twice its argument instead of adding the outer `a`.

Free variables of α-equivalent expressions. Renaming touches only bound variables, so it should leave the free ones alone. The following makes this precise.

Theorem 53

If e_1 and e_2 are α-equivalent, then $\mathrm{FV}(e_1) = \mathrm{FV}(e_2)$. Moreover, the free uses sit in the same places: wherever e_1 has a free `x`, e_2 has a free `x` at the corresponding place in the tree.

Proof sketch. It suffices to show that one legal rename keeps the free variables, since α-equivalence is built from a finite number of renames. A rename changes only a binder and the uses that point to it, so no free name is changed. The new name is fresh, so no free use is captured by the renamed binder. And the renamed uses still point to the same binder, so no bound use becomes free. Each use therefore stays free or stays bound, and free names keep their spelling. □

The theorem gives a quick way to rule out α-equivalence: if the free variables differ, the two expressions cannot be α-equivalent.

e_1	e_2	FV	α-equivalent?
`(lambda (x) (+ x y))`	`(lambda (z) (+ z y))`	{y}, {y}	yes
`(lambda (x) (+ x y))`	`(lambda (y) (+ y y))`	{y}, {}	no (y captured)

The converse is false: equal free variables do not make two expressions α-equivalent. `(+ y 1)` and `(* y 2)` both have $\mathrm{FV} = \{\mathtt{y}\}$, but their trees differ. So comparing free variables is a necessary check, not a sufficient one. When the sets agree, compare the binding arrows, or the lexical addresses of Section 5.1.6.

Subexpressions. If two expressions are α-equivalent, are their corresponding subexpressions α-equivalent too? Not always. A subexpression taken out on its own loses its enclosing binders, so the names those binders bound become free, and the renamed ones no longer match:

```
(lambda (x) x)   and   (lambda (y) y)     alpha-equivalent
x                and   y                  the bodies: not alpha-equivalent
```

Corresponding subexpressions are α-equivalent when no renamed binder encloses them, for example when they are closed. Bodies under renamed binders are compared by first giving both parameters the same fresh name `z`:

$$\texttt{(lambda (x) b1)} \equiv_\alpha \texttt{(lambda (y) b2)} \quad \text{exactly when} \quad b_1[\mathtt{x} := \mathtt{z}] \equiv_\alpha b_2[\mathtt{y} := \mathtt{z}].$$

Substitution and α-equivalence. In this subsection, $e[x := e']$ is capture-avoiding substitution. Substitution preserves α-equivalence: start from α-equivalent pieces, substitute, and the results are still α-equivalent. Going backwards, from equal results to the pieces, works only when the substitution loses no information. Table 5.1 collects the four directions.

Same context, different pieces (row 3). If x does not occur free in C, substitution does nothing, and nothing follows: with $C = \texttt{(+ 1 2)}$, both $C[x := 5]$ and $C[x := 7]$ are `(+ 1 2)`, yet 5 and 7 are not α-equivalent. If x does occur free in C, look at one of its places. After substitution, that place holds a copy of e_1 on one side and of e_2 on the other. Because the substitution avoids capture, no binder of C can catch a free name of e_1 or e_2. So every arrow starting inside e_1 ends inside e_1 or leaves the whole expression, and likewise for e_2. The two whole expressions have the same arrows, so the two copies do too, which makes $e_1 \equiv_\alpha e_2$.

if	then	holds?
$e_1 \equiv_\alpha e_2$	$C[x := e_1] \equiv_\alpha C[x := e_2]$	always
$e_1 \equiv_\alpha e_2$	$e_1[x := v] \equiv_\alpha e_2[x := v]$	always
$C[x := e_1] \equiv_\alpha C[x := e_2]$	$e_1 \equiv_\alpha e_2$	only if x occurs free in C
$e_1[x := v] \equiv_\alpha e_2[x := v]$	$e_1 \equiv_\alpha e_2$	not in general; yes if v is a fresh variable

Table 5.1: Capture-avoiding substitution and α-equivalence. The first two rows fail for capturing substitution.

Different expressions, same piece (row 4). Substitution can forget where something came from: with $e_1 = \texttt{x}$, $e_2 = \texttt{5}$, and $v = \texttt{5}$, both results are `5`, but `x` and `5` are not α-equivalent. The exception is a rename to a fresh variable `z`. Since `z` occurs nowhere else, every `z` in the result must have been an `x`, so the rename can be undone and nothing is lost. This is the case used above to compare the bodies of two `lambda`s.

Remark 54. The arguments for rows 3 and 4 are sketches, not full proofs. A complete proof goes by induction on the structure of the expression; compare [5, Section 5.3], where terms are identified up to α-equivalence.

Key Point 55 (Why α-equivalence matters for the interpreter)

α-equivalent programs must give the same answer: the names of bound variables are the programmer's choice and carry no meaning. This principle does two jobs in Chapter 4:

- It *detects* the capture bug. `(let x y (let y 2 (+ x y)))` and `(let x y (let z 2 (+ x z)))` are α-equivalent, yet capturing substitution gives 4 for one and an error for the other.
- It *licenses* the fix. Renaming a bound `y` to a fresh `gensym` produces an α-equivalent program, so `subst` may rename whenever it likes.

The same test will expose the bug in Section 5.3: renaming the inner `x` to `z` in `(let x 1 (let y x (let x 2 (+ y y))))` changes the answer of the first lazy environment interpreter, so that interpreter is wrong.

5.1.6 Lexical Addresses

α-equivalence says that only the arrows from uses to binders matter, not the names. A *lexical address* writes each arrow down as a pair of numbers, so the names can be dropped altogether.

Definition 56 (Lexical depth and lexical position)

Let a variable use refer to a binder. Its *lexical depth* d is the number of binders that lie between the use and its binder: 0 if the binder is the innermost one enclosing the use, 1 if one more binder must be crossed, and so on. Its *lexical position* p is the index of the variable in that binder's parameter list, counting from 0. The pair is written `(: d p)`.

In the nameless form, each `(lambda (x1 ... xn) body)` becomes `(lambda n body)`, keeping only the number of parameters, and each bound use becomes its address. A free variable has no binder and therefore no address; it keeps its name.

Example 57.

```
(lambda (x y)
  ((lambda (a) (x (a y)))
   x))
```

Inside the inner `lambda`, `a` is bound by the innermost binder, at position 0, so it is `(: 0 0)`. `x` and `y` are bound one binder further out, at positions 0 and 1: `(: 1 0)` and `(: 1 1)`. The last `x` sits directly in the outer `lambda`, so it is `(: 0 0)`. The nameless form is

```
(lambda 2
  ((lambda 1 ((: 1 0) ((: 0 0) (: 1 1))))
   (: 0 0)))
```

The two uses of `x` get different addresses because they sit at different depths, while `a` and the last `x` share the address `(: 0 0)` although their names differ.

Key Point 58 (Addresses and α-equivalence)

Two expressions are α-equivalent exactly when their nameless forms are identical: renaming a bound variable consistently never changes any address, and a capturing rename does. The same pair also tells an implementation where to find a value: if the environment is a chain of frames, one per binder, `(: d p)` means "go out d frames and take slot p", with no name comparison as in `lookup`. With one parameter per `lambda`, as in the lambda calculus, the position is always 0 and the depth alone is the *de Bruijn index*.

Remark 59. Conventions differ between sources: some count from 1, and some treat `let` as a binder of its own (it is a `lambda` application, Section 5.1). Check which convention a given problem uses.

Exercise 60 (Nameless form)

Write the nameless form of

```
(lambda (x y z)
  (z (lambda (x y) (z x)) y))
```

Solution. The outer binder has parameters `x`, `y`, `z` at positions 0, 1, 2; the inner one has `x`, `y` at positions 0, 1.

- The first `z` sits directly in the outer `lambda`: `(: 0 2)`.
- In the inner body `(z x)`, `z` is not a parameter of the inner binder, so it crosses one binder to reach the outer `z`: `(: 1 2)`. The `x` is the inner one, which shadows the outer `x`: `(: 0 0)`.
- The last `y` is outside the inner `lambda`, so it is the outer `y`: `(: 0 1)`.

```
(lambda 3
  ((: 0 2)
   (lambda 2 ((: 1 2) (: 0 0)))
   (: 0 1)))
```

The inner parameter `y` is never used, but it still occupies position 1, and it keeps the outer `y` out of reach only inside the inner body.

5.2 Environments

5.2.1 The Plan

Every interpreter since Section 4.1 has had to answer one question: in

```
(let x 5 (+ x x))
```

when `eval` reaches the `x` in `(+ x x)`, where does the 5 come from? There are two answers (Figure 5.2).

- **Substitution** (Chapter 4). Before evaluating the body, rewrite it: replace every `x` by the value. The body becomes `(+ 5 5)`, which contains no names at all, so evaluating it never has to ask about `x`.
- **Environments** (this chapter). Leave the body exactly as written. Evaluate it together with a table, the *environment*, that says `x = 5`. When evaluation reaches `x`, look it up in the table.

Both give 10. Version 5a goes back to the simplest language with names, numbers and eager `let`, and implements it the second way. The rest of the chapter is about making the second answer agree with the first in *every* program, not just this one.

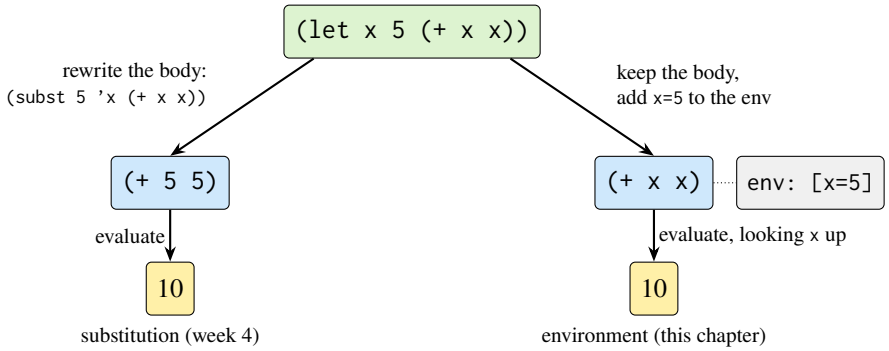


Figure 5.2: Two implementations of the same `let`. Substitution changes the program; an environment changes only what `eval` carries along.

From here on the expression type is called Expr rather than ArithE; the variants are the same. Version 5a drops `lambdaC` and `appC` again, so its only values are numbers and `eval` returns a `number`:

```
(define-type Expr
  [numC (n : number)]
  [plusC (e1 : Expr) (e2 : Expr)]
  [timesC (e1 : Expr) (e2 : Expr)]
  [letC (x : symbol) (e1 : Expr) (e2 : Expr)]
  [idC (x : symbol)])
```

5.2.2 The Data Structure

Definition 61 (Environment)

An *environment* is a mapping from names to values. Here it is a list of *bindings*, newest first; looking a name up returns the value of the *first* binding with that name.

```
(define-type Binding
  [bind (name : symbol) (val : number)])

(define-type-alias Env (listof Binding))
(define empty-env empty)
(define extend-env cons)

(define (lookup (x : symbol) (env : Env)) : number
  (cond
    [(cons? env)
     (if (equal? (bind-name (first env)) x)
         (bind-val (first env))
         (lookup x (rest env)))]
    [else (error 'lookup "No binding found")]))
```

Three small pieces of `plai-typed` appear here:

- `define-type-alias` gives a type a second name. `Env` *is* `(listof Binding)`; writing `Env` only documents intent.
- `(define extend-env cons)` names an existing function. Adding a binding is consing it onto the front: `(extend-env (bind 'x 5) env)`.
- `lookup` is ordinary list recursion, the shape of `my-reverse` in Section 1.3: an empty case (no binding, an error) and a `cons` case (check the first binding, else recur on the rest).

5.2.3 The Interpreter

`eval-env` takes the environment as an extra argument, and `eval` starts it with the empty one:

```
(define (eval-env (env : Env) (e : Expr)) : number
  (type-case Expr e
    [numC (n) n]
    [plusC (e1 e2) (+ (eval-env env e1) (eval-env env e2))]
    [timesC (e1 e2) (* (eval-env env e1) (eval-env env e2))]
    [idC (x) (lookup x env)]
    [letC (x e1 e2)
          (let ([v1 (eval-env env e1)])
            (eval-env (extend-env (bind x v1) env) e2))]))

(define (eval (e : Expr)) : number
  (eval-env empty-env e))
```

Compare it with `eval-eager` of Section 4.1 clause by clause:

case	substitution	environment
`letC`	evaluate `e1`, *rewrite* `e2` with `subst`, evaluate the new tree	evaluate `e1`, *extend the env* with `x=v1`, evaluate the unchanged `e2`
`idC`	never reached for a bound name, so it is an error	`lookup`
others	recurse	recurse, passing `env` along unchanged

The `idC` row is the heart of the change. In the substitution interpreter, reaching a name means it was never bound. In `eval-env`, reaching a name is the normal case, and the table answers it.

5.2.4 Traces

Write the environment next to each step. `eval` starts with `[]`:

```
(let x 5 (+ x x))                  env = []
  evaluate 5                       ---> 5
  evaluate (+ x x)                 env = [x=5]
    x ---> 5,  x ---> 5
---> 10
```

Shadowing comes for free. The new binding goes on the front, and `lookup` returns the first match, so an inner binding hides an outer one of the same name (Figure 5.3). When `(+ x x)` is evaluated, the environment is `'((bind x 6) (bind x 5))`:

```
(let x 5 (let x 6 (+ x x)))        env = []
  evaluate 5                       ---> 5
  (let x 6 (+ x x))                env = [x=5]
    evaluate 6                     ---> 6
    (+ x x)                        env = [x=6, x=5]
      x ---> 6 (first match),  x ---> 6
---> 12
```

Substitution needed the `(equal? x y)` test in `subst-eager` to get this right (Section 4.1); here there is no special case at all.

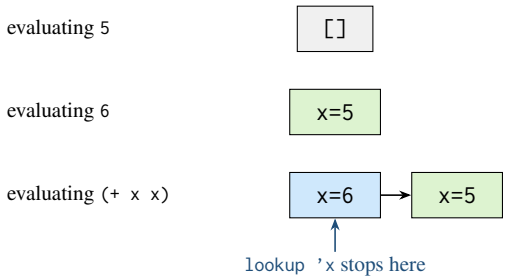


Figure 5.3: The environment while evaluating `(let x 5 (let x 6 (+ x x)))`. Each `let` pushes a binding on the front; `lookup` scans from the front, so the newest `x` wins.

Scope comes from passing the env down. `eval-env` passes the extended environment only *into* the body of the `let`; it never returns it. So a binding is visible exactly inside its own body. For example, `(+ (let x 5 (+ 3 x)) e2)`:

```
(+ (let x 5 (+ 3 x)) x)            env = []
  left:  (let x 5 (+ 3 x))         env = []
           (+ 3 x)                 env = [x=5]   ---> 8
  right: x                         env = []      ---> lookup: No binding found
```

The `x=5` binding existed only during the left operand. When the right operand is evaluated, `plusC` uses its own `env`, which never contained `x`. This matches lexical scope: the second `x` is not inside the `let`.

5.2.5 A Default Environment

Starting from a non-empty environment gives the language predefined names for free:

```
(define default-env : Env
  (extend-env (bind 'pi 3.14159) (extend-env (bind 'e 2.71828) empty-env)))

(define (eval-default-env (e : Expr)) : number
  (eval-env default-env e))
```

```
> (eval-default-env (parse '(let r 10 (* pi (* r r)))))
- number
314.159
```

A program may still shadow them: `(let pi 3 pi)` is 3. The substitution interpreter has no natural place for such names; it would have to wrap every program in `(let pi 3.14159 ...)`. Real languages work like `default-env`: Racket's `+`, `map`, and `first` are bindings in the initial environment.

5.2.6 Checking Against Substitution

For comparison we keep the old substitution evaluator, the eager interpreter of Section 4.1 under new names:

```
(define (eval-subst (e : Expr)) : number
  (type-case Expr e
    [numC (n) n]
    [plusC (e1 e2) (+ (eval-subst e1) (eval-subst e2))]
    [timesC (e1 e2) (* (eval-subst e1) (eval-subst e2))]
    [letC (x e1 e2)
          (let ([v1 (eval-subst e1)])
            (eval-subst (subst v1 x e2)))]
    [idC (x) (error 'eval-subst "unbound variable!")]))
```

where `subst` is `subst-eager`. Both interpreters are run on the same four programs:

program	`eval-subst`	`eval`
`(+ 27 (let x 5 (+ x x)))`	37	37
`(let x 5 (let y 6 (+ x y)))`	11	11
`(let x 5 (let x 6 (+ x x)))`	12	12
`(let x 5 (let x (* x 2) (+ x x)))`	20	20

The last program checks that `e1` is evaluated in the *outer* environment: `(* x 2)` sees `x=5`, and only then is `x=10` pushed for the body.

Key Point 62 (Environment vs. substitution, for `let`)

For this language the two interpreters always agree: each binding in `env` is a substitution that has been *postponed* until the name is actually reached. Instead of copying 5 into every `x` of the body up front, `eval-env` remembers `x=5` and fills it in on demand. Substitution is the reference: whenever a later environment interpreter disagrees with it, the environment interpreter has a scope bug. That is how we find both bugs of Sections 5.3 and 5.4.

5.3 Lazy `let` and Dynamic Scope

Versions 5b and 5c make the environment interpreter *lazy*, as Section 4.2 did for substitution: `let` does not evaluate `e1`, but stores it and evaluates it only when `x` is used. The language is still that of 5a, numbers and `let`. The first attempt gets scope wrong; the second fixes it.

5.3.1 Attempt 1: Store the Expression

A binding now holds an unevaluated expression, and `lookup` returns it:

```
(define-type Binding
  [bind (name : symbol) (defn : Expr)])

(define (lookup (x : symbol) (env : Env)) : Expr
  ...)   ; as in 5a, returning (bind-defn (first env))

(define (eval-env1 (env : Env) (e : Expr)) : number
  (type-case Expr e
    [numC (n) n]
    [plusC (e1 e2) (+ (eval-env1 env e1) (eval-env1 env e2))]
    [timesC (e1 e2) (* (eval-env1 env e1) (eval-env1 env e2))]
    [idC (x) (eval-env1 env (lookup x env))]
    [letC (x e1 e2) (eval-env1 (extend-env (bind x e1) env) e2)]))

(define (eval1 (e : Expr)) : number
  (eval-env1 empty-env e))
```

`letC` stores `e1` as is. `idC` looks up the stored expression and evaluates it, in `env`, the environment *at the use*. The first examples work:

```
> (eval1 (parse '(+ 27 (let x 5 (+ x x)))))
37
> (eval1 (parse '(let x 5 (let y 6 (+ x y)))))
11
> (eval1 (parse '(let x 5 (let x 6 (+ x x)))))
12
```

Now try a program where the stored expression mentions a name that is rebound before the use:

```
(let x 1 (let y x (let x 2 (+ y y))))   env = []
  store x=1                             env = [x=1]
  store y=x            (the expression x, not a value)
                                        env = [y=x, x=1]
  store x=2                             env = [x=2, y=x, x=1]
  (+ y y)
    y ---> its expression is x; evaluate x in the CURRENT env
         ---> lookup x finds x=2 ---> 2
---> 4
```

"We get 4, but under our substitution semantics this would give us 2": when `y` was defined, the only `x` in scope was `x=1`, so `y` means 1 (Figure 5.4). There is a quicker way to see that 4 is wrong. Rename the inner `x` to `z`;

the program is α-equivalent (Section 5.1), so the answer must not change:

```
> (eval1 (parse '(let x 1 (let y x (let x 2 (+ y y))))))
4
> (eval1 (parse '(let x 1 (let y x (let z 2 (+ y y))))))
2
```

It does change, so `eval-env1` is wrong. The last example is the counterpart of capture in Section 4.2:

```
> (eval1 (parse '(let x y (let y 2 (+ x x)))))
4
```

The `y` in `(let x y ...)` is free, so this should be an error, but by the time `x` is used, the environment happens to contain `y=2`.

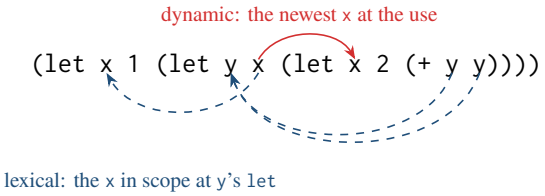


Figure 5.4: What the `x` stored in `y` refers to. Lexically (dashed blue) it is the outer `x`, giving 2; `eval-env1` evaluates it after the inner `let x 2` and finds that one instead (red), giving 4.

5.3.2 Dynamic vs. Lexical Scope

Definition 63 (Lexical and dynamic scope)

Under *lexical* (static) scope, the binding a name refers to is determined by the program text: the nearest enclosing binder of that name, which can be found without running the program. Under *dynamic* scope, it is determined by the execution history: the most recent binding of that name that is still active when the name is evaluated.

This definition is slightly informal; a more precise one could be given. The binding arrows of Figure 5.4 are the informal picture.

Substitution always gives lexical scope: it replaces names inside the binder's body at the moment the binder is processed, so a name can only ever be replaced by its own binder. `eval-env1` gives dynamic scope, because it evaluates the stored expression in whatever environment exists when the name is used, and by then other bindings may have been added.

Pitfall 64 (Dynamic scope breaks α-equivalence)

Under dynamic scope, renaming a local variable can change what *other* code means, here the definition of `y`. A function's meaning depends on every caller's local names. This is why dynamic scope is considered a design mistake for ordinary variables, and why the test "rename a bound variable and compare" is such a useful bug detector.

5.3.3 Attempt 2: Store the Environment Too

Version 5c corrects the issue by saving the environment *at the time of the let* inside the binding, and evaluates the stored expression in that environment:

```
(define-type Binding
  [bind (name : symbol) (env : Env) (defn : Expr)])

(define (lookup (x : symbol) (env : Env)) : Binding
  ...)   ; returns the whole binding, (first env)

(define (eval-env2 (env : Env) (e : Expr)) : number
  (type-case Expr e
    [numC (n) n]
    [plusC (e1 e2) (+ (eval-env2 env e1) (eval-env2 env e2))]
    [timesC (e1 e2) (* (eval-env2 env e1) (eval-env2 env e2))]
    [idC (x) (let ([res (lookup x env)])
               (eval-env2 (bind-env res) (bind-defn res)))]
    [letC (x e1 e2) (eval-env2 (extend-env (bind x env e1) env) e2)]))

(define (eval2 (e : Expr)) : number
  (eval-env2 empty-env e))
```

Two lines changed. `letC` stores `env`, the environment of the `let`, alongside `e1`. `idC` evaluates the stored expression in the stored environment, not the current one. A binding refers to an earlier environment, so the types are mutually recursive: a Binding contains an Env, which is a list of Bindings.

```
(let x 1 (let y x (let x 2 (+ y y))))   env = []
  store x=(1 in [])                     env = [x]
  store y=(x in [x=1])                  env = [y, x]
  store x=(2 in [y, x=1])               env = [x, y, x]
  (+ y y)
    y ---> evaluate x in y's saved env [x=1] ---> 1
---> 2
```

"Now these examples have behaviors like what our substitution semantics produced":

```
> (eval2 (parse '(let x 1 (let y x (let x 2 (+ y y))))))
2
> (eval2 (parse '(let x 1 (let y x (let z 2 (+ y y))))))
2
> (eval2 (parse '(let x y (let y 2 (+ x x)))))
lookup: No binding found
```

In the last one, `x`'s expression `y` is evaluated in `x`'s saved environment, which is empty. The interpreter is still lazy: `(let x (+ y y) 10)` is 10, since `x` is never used.

Key Point 65 (The fix, in one sentence)

Run stored code in the environment where it was *written*, not where it is *used*. The pair (expression, environment) is the general tool for this; Section 5.4 needs it again for functions.

5.4 Functions with Environments: Closures

Versions 5d and 5e add `lambda` to the environment interpreter, back with an eager `let`. Once again we first do something that behaves incorrectly, or at least counter-intuitively, and then switch to a fixed version: the first attempt has dynamic scope, and the fix is the one of Section 5.3.

5.4.1 Attempt 1: A Function Is Its Parameter and Body

Expr gets `lambdaC` and `appC` back, `parse` is the one of Section 4.4 (a list whose head is not a keyword is an application), and the result type is the Value of Section 4.4, with function values written `lambdaV`. A binding holds a Value:

```
(define-type Value
  [numV (n : number)]
  [lambdaV (x : symbol) (e : Expr)])

(define-type Binding
  [bind (name : symbol) (val : Value)])
;; Env, empty-env, extend-env, lookup: as in 5a, returning a Value

(define (eval-env (env : Env) (e : Expr)) : Value
  (type-case Expr e
    [numC (n) (numV n)]
    [lambdaC (x e) (lambdaV x e)]
    [plusC (e1 e2) (numV (+ (numV-n (eval-env env e1)) (numV-n (eval-env env e2))))]
    [timesC (e1 e2) (numV (* (numV-n (eval-env env e1)) (numV-n (eval-env env e2))))]
    [letC (x e1 e2)
          (let ([v1 (eval-env env e1)])
            (eval-env (extend-env (bind x v1) env) e2))]
    [appC (e1 e2)
          (let [(v1 (eval-env env e1))
                (v2 (eval-env env e2))]
            (eval-env (extend-env (bind (lambdaV-x v1) v2) env) (lambdaV-e v1)))]
    [idC (x) (lookup x env)]))

(define (eval (e : Expr))
  (eval-env empty-env e))
```

`+` and `*` unwrap their operands with `numV-n` directly, and `appC` takes the function apart with `lambdaV-x` and `lambdaV-e`, so adding a function or applying a number fails inside an accessor rather than with a custom message. A call evaluates the function and the argument (eager), then evaluates the body in `env` extended with `x=v2`: it is the `letC` clause with the body taken from the function value. The four `let` programs of Section 5.2.6 still give 37, 11, 12, 20, now as `numV`s, and simple functions work:

```
> (eval (parse '((lambda x (+ 2 x)) 1)))
(numV 3)
> (eval (parse '(let x 2 (let addx (lambda y (+ y x)) (addx 3)))))
(numV 5)
```

"But now..." `env` in the `appC` clause is the environment of the *call*. If the caller has rebound a name that the

body uses freely, the body sees the caller's binding:

```
(let x 2
  (let addx (lambda y (+ y x))
    (let x 5
      (addx 3))))
  env at the call:   [x=5, addx=(lambdaV y (+ y x)), x=2]
  body (+ y x) in    [y=3, x=5, addx=..., x=2]
    y ---> 3,  x ---> 5   (the caller's x)
---> 8
```

Lexically, the x in addx's body is the x=2 around the lambda, so the answer should be 5 (Figure 5.5). "Oops, we have the same problem of dynamic scope!" And for the same reason as eval-env1: the function body is stored code, and it is run in the environment where it is used.

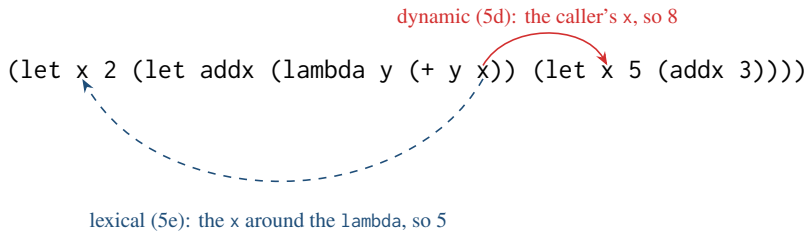


Figure 5.5: The free x in addx's body. Without closures, the body is evaluated in the caller's environment and finds the inner x.

This is also the eager-capture bug of Section 4.4.4 in a new form. In Section 4.4.4, `(let f (lambda x (+ x y)) (let y 6 (f 1)))` gave 7 under capturing substitution, because the free y of f met the y=6 at the call. The eval of 5d gives 7 for it too. Capture in a substitution interpreter and dynamic scope in an environment interpreter are the same mistake.

5.4.2 Attempt 2: Closures

The fix is the one of Section 5.3: save the environment where the code was written, here where the lambda was evaluated, together with the code.

Definition 66 (Closure)

A *closure* is a function value consisting of the parameter, the body, and the environment in which the lambda expression was evaluated. Calling it evaluates the body in that saved environment, extended with the parameter.

Version 5e replaces `lambdaV` by `closV` and changes two clauses of `eval-env`; everything else is 5d unchanged:

```
(define-type Value
  [numV (n : number)]
  [closV (env : Env) (x : symbol) (e : Expr)])

    [lambdaC (x e) (closV env x e)]
    ...
    [appC (e1 e2)
          (let [(v1 (eval-env env e1))
                (v2 (eval-env env e2))]
            ; run the body in the closure's saved env, not the current one
            (eval-env (extend-env (bind (closV-x v1) v2) (closV-env v1)) (closV-e v1)))]
```

`lambdaC` captures the current `env` in the closure. `appC` extends `(closV-env v1)`, the saved environment, instead of `env`, the caller's. Value and Env now refer to each other, as Binding and Env did in 5c. (`val->expr` is gone: nothing is substituted any more, so values never have to turn back into expressions.)

Here is the `addx` example again, with the inner `x` changed to 6 (dynamic scope would give 9):

```
(let x 2
  (let addx (lambda y (+ y x))
    (let x 6
      (addx 3))))
  evaluating the lambda in [x=2]  ---> addx = (closV [x=2] y (+ y x))
  env at the call:   [x=6, addx, x=2]          (not used for the body)
  body (+ y x) in    [y=3, x=2]                (the closure's env + y=3)
    y ---> 3,  x ---> 2
---> 5
```

"It gives 5, as we'd expect with lexical scope." The closure carries its own environment, shown in Figure 5.6. Whatever the caller binds, the body sees only the bindings that were in scope where the `lambda` was written, plus its parameter. The example `(let f (lambda x (+ x y)) (let y 6 (f 1)))` is now `No binding found`, as it should be.

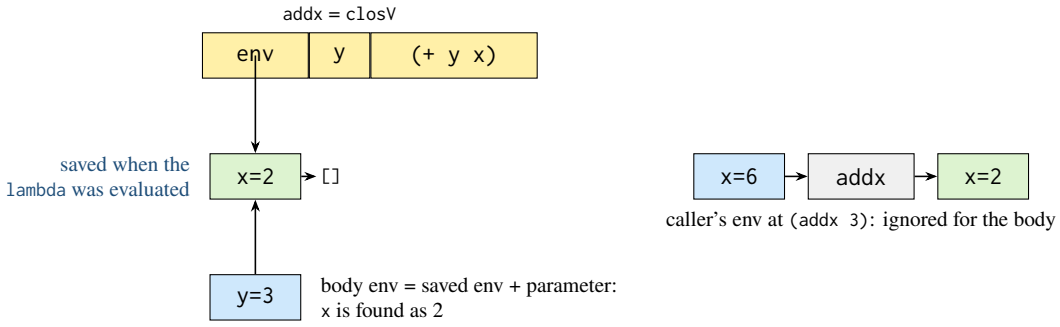


Figure 5.6: The closure for `addx` and the environments of the call `(addx 3)`. The body is evaluated in the closure's environment extended with `y=3`; the caller's `x=6` is never consulted.

Key Point 67 (Closures)

A function value must remember the environment in which it was created. This is not an optimization or a detail of this interpreter: it is the definition of lexical scope for first-class functions, and it is why `make-adder` in Section 2.3 returns a function that still knows its `x` after `make-adder` has returned.

5.4.3 The Five Versions Side by Side

Each version is a small change to the previous one, alternating between a first attempt that gets scope wrong and a fix:

version	what is added	what a binding stores	scope
5a	environments, eager `let`, numbers only	a number	lexical
5b	lazy `let`, attempt 1	an unevaluated expression	dynamic
5c	lazy `let`, attempt 2	an expression *and its env*	lexical
5d	`lambda`, attempt 1	a value; functions are `lambdaV`	dynamic
5e	`lambda`, attempt 2	a value; functions are closures	lexical

The two red rows fail for the same reason and are fixed the same way: some code is run in the environment where it is used instead of the environment where it was written. On the programs of this chapter:

	substitution (Section 4.4.4)	`eval1` (5b)	`eval2` (5c)	`eval` (5d)	`eval` (5e)
`(let x 5 (let x 6 (+ x x)))`	12	12	12	12	12
`(let x 1 (let y x (let x 2 (+ y y))))`	2	4	2	2	2
`(let x y (let y 2 (+ x x)))`	error	4	error	error	error
`(let x (+ y y) 10)`	error	10	10	error	error
addx example	5	—	—	8	5
`(let f (lambda x (+ x y)) (let y 6 (f 1)))`	error	—	—	7	error

The substitution column is the capture-avoiding eager interpreter `eval-eager-v3` (Section 4.4.4). Red entries disagree with it for scope reasons; "—" means the version has no `lambda`. The row `(let x (+ y y) 10)` is not a scope difference but the eager/lazy one of Section 4.2: lazy interpreters never evaluate an unused `e1`.

5.5 Recitation 5: Environments as Functions, Scope, and Macros

Environment interpreters were built twice in this chapter, and twice the first attempt had dynamic scope (Sections 5.3 and 5.4). This section comes back to the same material from four sides:

1. a second representation of environments, as *functions* instead of lists;
2. one interpreter with a flag `dynamic?` that runs any program under either scope;
3. small programs whose result depends on the scope, to predict and then check;
4. scope in languages with assignment, and scope in *macros*, where the static/dynamic question comes back as *hygiene*.

The section ends with a short look at Lean. Every result in this section comes from running the code shown.

5.5.1 The Language

The language is that of version 5e (Section 5.4): numbers, `+`, `*`, `let`, one-argument `lambda`, and application.

```
(define-type Expr
  [numC (n : number)]
  [plusC (e1 : Expr) (e2 : Expr)]
  [timesC (e1 : Expr) (e2 : Expr)]
  [letC (x : symbol) (e1 : Expr) (e2 : Expr)]
  [lambdaC (x : symbol) (e : Expr)]
  [appC (e1 : Expr) (e2 : Expr)]
  [idC (x : symbol)])

(define-type Value
  [numV (n : number)]
  [closV (env : Env) (x : symbol) (e : Expr)])
```

`parse` is unchanged. A list whose head is not `+`, `*`, `lambda`, or `let` is an application, whether the head is some other symbol, as in `(f 0)`, or another list, as in `((lambda x x) 3)`. Since every function takes exactly one argument, a function that needs none takes a dummy argument `_` and is called with `0`.

5.5.2 Environments as Functions

In Section 5.2 an environment was a list of bindings, and `lookup` walked the list. But the only thing the interpreter ever does with an environment is ask "what is `x` bound to?" So an environment can simply *be* the function that answers that question:

```
(define-type-alias Env (symbol -> (optionof Value)))
```

Given a name, the environment returns `(some v)` if the name is bound to `v` and `(none)` if it is unbound. `optionof` is plai-typed's built-in "maybe" type, with the two variants `some` and `none`. `Env` and `Value` refer to each other (a closure holds an environment, an environment returns values), which `define-type` allows across definitions.

The three operations. Three definitions are needed.

Exercise 68 (The empty environment)

Define `empty-env : Env`, the environment in which nothing is bound.

Solution.

```
(define empty-env : Env
  (lambda (x) (none)))
```

It answers `none` for every name. It does not raise an error itself; it only says "not here", and `lookup` decides that this is an error.

Exercise 69 (`extend-env`)

Define `(extend-env x v env)`, which binds `x` to `v` on top of `env`. Hint: return a new function. When is it asked about `x`? What should it do otherwise?

Solution.

```
(define (extend-env (x : symbol) (v : Value) (env : Env)) : Env
  (lambda (y)
    (if (equal? x y)
        (some v)
        (env y))))
```

Asked about `x`, the name it binds, the new function answers `v`. Asked about any other name, it passes the question on to the environment it extends. The old `env` is captured in the closure that `lambda` builds, so this is a closure in Racket, the very thing `closV` implements in our language. Unlike the `extend-env` of Section 5.2, which took a Binding, this one takes the name and value directly; there is no Binding type at all.

Exercise 70 (`lookup`)

Define `(lookup x env)`, which returns the value bound to `x` in `env` and signals an error if `x` is unbound, with the message `(string-append "unbound variable: " (symbol->string x))`. Hint: you can `type-case` on an `optionof`, just as on a type you defined.

Solution.

```
(define (lookup (x : symbol) (env : Env)) : Value
  (type-case (optionof Value) (env x)
    [some (v) v]
    [none () (error 'lookup (string-append "unbound variable: " (symbol->string x)))]))
```

`lookup` calls the environment on `x` and takes the answer apart: `some` carries the value, `none` means no layer recognized the name.

Pitfall 71 (type-case syntax: plai-typed is not plait)

Writing the clauses of `lookup` as `[(some v) v]` and `[(none) ...]`, and the type as `(Optionof Value)`, gives a program that does not even compile:

```
type-case: expected an identifier from a define-type
  at: (some v)
```

That is the syntax of `#lang plait`, the successor of plai-typed. In `#lang plai-typed` a clause is the variant name followed by a parenthesized list of field names, `[some (v) ...]` and `[none () ...]`, exactly as in every `type-case` on Expr so far; the type is written in lower case, `optionof`.

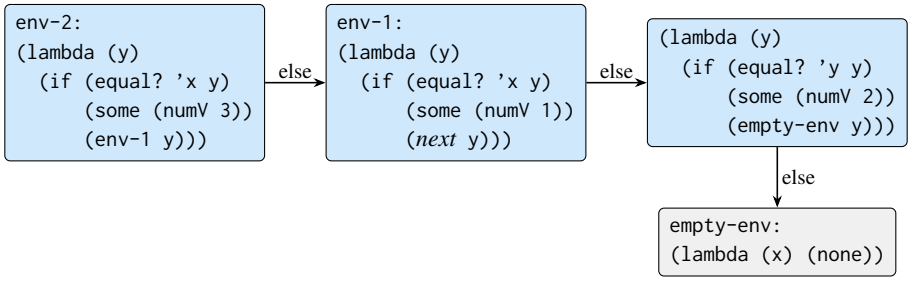


Figure 5.7: `env-1` and `env-2` as chains of closures. Each `extend-env` wraps the previous environment; a lookup enters at the left and moves right until some layer recognizes the name. `(lookup 'x env-2)` stops at the first box (3), and `env-1`, still reachable on its own, answers 1.

Tests. We build two environments (Figure 5.7):

```
(define env-1 (extend-env 'x (numV 1) (extend-env 'y (numV 2) empty-env)))
(define env-2 (extend-env 'x (numV 3) env-1))

(test (lookup 'x env-1) (numV 1))
(test (lookup 'y env-1) (numV 2))
(test/exn (lookup 'z env-1) "unbound")
(test (lookup 'x env-2) (numV 3))   ; the newer binding shadows the older one
(test (lookup 'x env-1) (numV 1))   ; env-1 is unchanged
```

Shadowing works for the same reason as with lists: the newest binding is asked first. The last test is *persistence*: extending `env-1` built a new function around it and changed nothing, which is what lets a closure keep the environment of its `lambda` while the rest of the program goes on extending.

Exercise 72 (Lookup time)

How long does `lookup` take, compared to the list version?

Solution. The same: linear in the number of bindings. A lookup calls one closure per enclosing `extend-env` until one matches, just as the list version visits one `cons` cell per binding. The function representation hides the chain inside closures, but the chain is still there (Figure 5.7). Neither version does better than linear; a faster lookup needs another data structure (a hash table, or, in a compiler, a known position for each variable).

Exercise 73 (What lists can do)

What can you do with a list environment that you can't do with a function environment?

Solution. Anything other than asking about one name. A function is opaque: the only operation on it is calling it. With a list you can

- print it, which matters for debugging: a `closV` shows its environment as `#<procedure>`;
- enumerate it: list all bound names, count them, or find the free variables a closure actually uses;
- compare two environments for equality. Two `closV`s with function environments can never be compared structurally, so a `test` whose expected value is a closure cannot be written;
- remove a binding, or save it to a file.

In exchange, the function representation commits to nothing: any function of type `(symbol -> (optionof Value))` is an environment, for example one that computes bindings on demand.

5.5.3 One Interpreter, Two Scopes

Here is the interpreter of version 5e on the new environments, with one extra argument `dynamic?`: `#f` for static scope (as in 5e), `#t` for dynamic scope (as in 5d). Every case except `appC` is written, and is the same for both scopes:

```
(define (eval-env (dynamic? : boolean) (env : Env) (e : Expr)) : Value
  (type-case Expr e
    [numC (n) (numV n)]
    [plusC (e1 e2) (numV (+ (numV-n (eval-env dynamic? env e1))
                            (numV-n (eval-env dynamic? env e2))))]
    [timesC (e1 e2) (numV (* (numV-n (eval-env dynamic? env e1))
                             (numV-n (eval-env dynamic? env e2))))]
    [letC (x e1 e2)
          (let ([v1 (eval-env dynamic? env e1)])
            (eval-env dynamic? (extend-env x v1 env) e2))]
    [idC (x) (lookup x env)]
    [lambdaC (x body) (closV env x body)]
    [appC (e1 e2) ...]))                    ; the exercise below

(define (eval-static (s : s-expression)) : Value
  (eval-env #f empty-env (parse s)))
(define (eval-dynamic (s : s-expression)) : Value
  (eval-env #t empty-env (parse s)))
```

Exercise 74 (The `appC` case)

Fill in `appC`. Which environment should the function body run in? Under static scope, the environment saved in the closure (where the `lambda` was written); under dynamic scope, the current environment (where the function is called).

Solution.

```
[appC (e1 e2)
      (let ([f (eval-env dynamic? env e1)]
            [arg (eval-env dynamic? env e2)])
        (type-case Value f
          [closV (cenv x body)
                 (eval-env dynamic?
                           (extend-env x arg (if dynamic? env cenv))
                           body)]
          [else (error 'eval-env "not a function")]))]
```

A call evaluates the function and the argument (eager), and then has to choose *which environment to extend* with the parameter:

scope	body runs in	which is
static (`dynamic? = #f`)	`cenv`, saved in the closure	where the `lambda` was written
dynamic (`dynamic? = #t`)	`env`, the current environment	where the function is called

That one `if` is the whole difference between 5d and 5e. In dynamic mode the closure still saves `cenv`, but nothing ever reads it, so a `closV` there is just the `lambdaV` of Section 5.4. The `else` clause catches a call of a number, such as `(1 2)`, which raises `eval-env: not a function` in both modes.

The `addx` example (Figure 5.5) gives one answer per scope:

```
(define lecture-example
  '(let x 2
     (let addx (lambda y (+ y x))
       (let x 6
         (addx 3)))))

(test (eval-static lecture-example) (numV 5))   ; x from where addx was written
(test (eval-dynamic lecture-example) (numV 9))  ; x from where addx was called
```

All seven tests pass.

5.5.4 Predict, Then Check

For each program, predict the result under static scope and under dynamic scope *before* running it; then check with `(eval-static ...)` and `(eval-dynamic ...)`. The result may be an error rather than a value, and then the question is which error and why. In our language every local variable becomes a `let`, and a function that needs no argument takes a dummy `_` and is called with `0`.

Exercise 75 (Predict, part a)

In C-like syntax:

```
int x = 14;
int h()  { return x; }
int f()  { int x = 13; return h(); }
int g()  { int x = 12; return h(); }
```

What do `f()` and `g()` return? In our language:

```
(define ex-a-f
  '(let x 14
     (let h (lambda _ x)
       (let f (lambda _ (let x 13 (h 0)))
         (let g (lambda _ (let x 12 (h 0)))
           (f 0))))))
;; ex-a-g: the same, ending in (g 0)
```

Solution. Static: both return `(numV 14)`. Dynamic: `f()` returns `(numV 13)` and `g()` returns `(numV 12)`. The body `x` is free in `h`. Statically it means the `x` visible where `h` is written, the global 14, whoever calls `h`. Dynamically it means the most recent `x` on the way to the call: `f`'s 13, or `g`'s 12. So under dynamic scope the same function returns different values depending on its caller, and renaming `f`'s local `x` to `z`, which should not matter, changes the result back to 14.

Exercise 76 (Predict, part b)

In C-like syntax:

```
const int b = 5;
int foo() { int a = b + 5; return a; }
int bar() { int b = 2; return foo(); }
```

What does `bar()` return? In our language:

```
(define ex-b
  '(let b 5
     (let foo (lambda _ (let a (+ b 5) a))
       (let bar (lambda _ (let b 2 (foo 0)))
         (bar 0)))))
```

Solution. Static: `(numV 10)`. Dynamic: `(numV 7)`. This is part a again with a twist: `b` is meant to be a constant, yet under dynamic scope `bar`'s local `b` replaces it inside `foo`, giving $2 + 5 = 7$ instead of $5 + 5 = 10$. A `const` declaration does not protect a name from being shadowed.

Exercise 77 (Predict, part c)

`f` uses a variable `y` that is not bound where `f` is written:

```
(define ex-c
  '(let f (lambda _ y)
     (let y 3
       (f 0))))
```

Solution. Static: `lookup: unbound variable: y`. Dynamic: `(numV 3)`. `f` is written where no `y` exists, so statically its body can never find one, and the call fails. Dynamically the call happens inside `(let y 3 ...)`, and the lookup finds that `y`. Dynamic scope "fixes" this program, but it is the kind of program a static checker would rightly reject, since whether `f` works depends on every caller.

Exercise 78 (Predict, part d)

A function that returns a function:

```
(define ex-d
  '(let make-adder (lambda n (lambda m (+ n m)))
     (let add5 (make-adder 5)
       (add5 1))))
```

Solution. Static: `(numV 6)`. Dynamic: `lookup: unbound variable: n`. This is the program that sinks dynamic scope. Under static scope, `(make-adder 5)` evaluates `(lambda m (+ n m))` in `make-adder`'s body, where `n` is 5, and the closure saves that environment. Under dynamic scope the saved environment is ignored, and `n` exists only while `make-adder` is running:

```
(add5 1), dynamic:
  env at the call = {add5 = <m-closure>, make-adder = <n-closure>}
  body (+ n m) runs in that env extended with m = 1
  lookup n  --->  unbound variable: n
```

The binding of `n` disappeared when `make-adder` returned, but the function that uses it did not. This is why closures save their environment: returning a function needs the environment of its definition.

Exercise 79 (Predict, part e)

Same as part d, but now some other `n` is around when `add5` is called:

```
(define ex-e
  '(let make-adder (lambda n (lambda m (+ n m)))
     (let add5 (make-adder 5)
       (let n 100
         (add5 1)))))
```

Solution. Static: `(numV 6)`. Dynamic: `(numV 101)`. With an unrelated `n` at the call site, dynamic scope no longer fails; it silently uses that `n`, so `add5` adds 100. That is worse than part d: a crash points at the bug, a wrong number does not. Figure 5.8 shows the two environments the body of `add5` runs in.

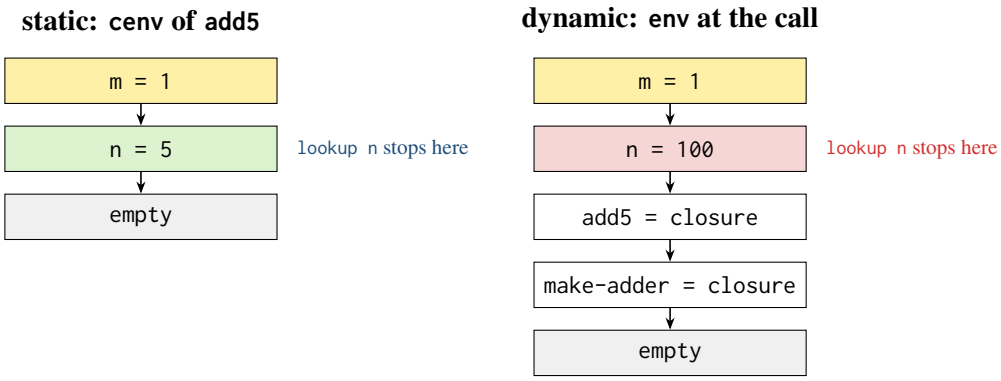


Figure 5.8: The environment in which the body `(+ n m)` of `add5` runs in `ex-e`. Statically it extends the environment saved when `(lambda m ...)` was evaluated inside `make-adder`, where `n` is 5. Dynamically it extends the caller's environment, where the nearest `n` is the unrelated 100. In `ex-d` the `n = 100` cell is missing, so the dynamic lookup reaches `empty` and fails.

Running all six confirms the predictions:

program	static	dynamic
`ex-a-f`	`(numV 14)`	`(numV 13)`
`ex-a-g`	`(numV 14)`	`(numV 12)`
`ex-b`	`(numV 10)`	`(numV 7)`
`ex-c`	`lookup: unbound variable: y`	`(numV 3)`
`ex-d`	`(numV 6)`	`lookup: unbound variable: n`
`ex-e`	`(numV 6)`	`(numV 101)`

Key Point 80

Under static scope, the meaning of a free variable in a function is fixed by the program text: you can read it off without knowing who calls the function. Under dynamic scope it depends on the chain of calls at run time, so local names are no longer private (a, b), renaming a local variable can change the result, and a function that outlives its defining call loses its free variables (d) or silently picks up someone else's (e).

5.5.5 Scope with Assignment

Our language has no assignment, but most languages do, and then a function can *change* the variable a free name refers to. The question is the same as before, which variable does the name denote, and these exercises are worked on paper: for each program, what does it print under static scope, and under dynamic scope?

Exercise 81 (Assignment, part a)

```
int x;
void h() { printf("%d\n", x); }
void f() { x = 13; h(); }
void g() { x = 12; h(); }
int main() { x = 14; f(); g(); }
```

Solution. Both scopes print `13` then `12`. Neither `f` nor `g` declares a local `x`, so there is only one `x`, the global one, and the assignments change it before `h` reads it. Compare Predict, part a, where `f` and `g` *declare* locals: static and dynamic scope can disagree only when there is more than one variable with the same name.

Exercise 82 (Assignment, part b)

This one is bash, which really is dynamically scoped, so you can run it in a terminal:

```
x=1
function g () { echo $x ; x=2 ; }
function f () { local x=3 ; g ; }
f
echo $x
```

Solution. Dynamic (what bash does): `3` then `1`. `g` is called from `f`, so its `x` is `f`'s `local x`: it prints 3, and `x=2` overwrites `f`*'s* local. When `f` returns, its local disappears and the global `x` is still 1. Static: `1` then `2`. `g`'s `x` would always be the global: it prints 1 and sets the global to 2, which the last line prints.

Exercise 83 (Assignment, part c)

From [7, Chapter 3]:

```
n : integer                 -- global
procedure first()
    n := 1
procedure second()
    n : integer             -- local
    first()
n := 2
if read_integer() > 0
    second()
else
    first()
write_integer(n)
```

Solution. Static: `1`, whatever is read. `first`'s `n` is the global one, and `first` runs on either branch. Dynamic: `2` for a positive input, `1` otherwise. Called from `second`, `first`'s `n` is `second`'s local, so `n := 1` changes that local and the global stays 2. Called from the main program, `first` sets the global. The output depends on the

input through the scope rule, and nothing in the text of `first` shows it.

5.5.6 Scope and Macros

The rest of the section moves the same question to *macros*. A macro generates code from a shorthand before the program runs. The generated code mentions names, some written in the macro's definition and some supplied at the use, and each must be resolved somewhere.

Text macros in C. The C preprocessor replaces text. `unhygenic.c`:

```c
#include <stdio.h>
#define SWAP(a, b) do { \
  int temp = (a);       \
  (a) = (b);            \
  (b) = temp;           \
} while(0)

int main(void) {
  int x = 10;
  int y = 20;
  SWAP(x, y);
  printf("temp = %d, y = %d\n", x, y);
  return 0;
}
```

Exercise 84 (Running `unhygenic.c`)

Compile and run `unhygenic.c` as it is. What does it print?

Solution. `temp = 20, y = 10`: the swap works (the label `temp` in the format string is just text; the value printed is `x`). The `do { ... } while(0)` wrapper turns the macro body into one statement, so that `SWAP(x, y);` also works as the body of an `if` without braces.

Exercise 85 (A strange renaming)

In the body of `main`, rename `x` to `temp`. Don't compile it yet.

Solution. Every `x` in `main` becomes `temp`:

```c
int main(void) {
  int temp = 10;
  int y = 20;
  SWAP(temp, y);
  printf("temp = %d, y = %d\n", temp, y);
  return 0;
}
```

Exercise 86 (Predicting the behavior)

Discuss with your neighbor what you *think* the renamed program prints.

Solution. Renaming a local variable consistently should not change what a program does, so the natural prediction is the same output as before: `temp = 20, y = 10`.

Exercise 87 (Explaining the behavior)

Now compile and run the program. Did it do what you expected? Why did it do that?

Solution. No. It prints `temp = 10, y = 20`: nothing is swapped. `cc -E` shows the expanded code:

```
int temp = 10;
int y = 20;
do { int temp = (temp); (temp) = (y); (y) = temp; } while(0);
```

The argument `temp` is pasted inside the block that declares the macro's own `temp`, so every `temp` in the block refers to the inner variable: the user's `temp` has been *captured*. The inner `temp` is initialized from itself, an uninitialized read (undefined behavior), then set to `y`, and finally `y = temp` copies that value back into `y`. The outer `temp` is never touched. This is the variable capture of Section 4.2, produced by a text substitution that knows nothing about binding.

Definition 88 (Preserving α-equivalence)

Recall (Section 5.1) that programs p and q are α-equivalent, $p \equiv_\alpha q$, if renaming bound variables in p gives q. A program transformation T, a function from programs to programs, *preserves α-equivalence* if

$$p \equiv_\alpha q \implies T(p) \equiv_\alpha T(q).$$

Exercise 89 (Reasoning about α-equivalence)

Consider the C preprocessor as a program transformation: it takes C code and produces C code. Does it preserve α-equivalence?

Solution. No. The program with `x` and the program with `temp` are α-equivalent: one renames a local variable of the other. After preprocessing they are not even equivalent in behavior, let alone α-equivalent: one swaps and the other does not. A transformation that preserves α-equivalence lets the programmer choose names freely; with C macros, the names used inside every macro body are silently reserved.

Hygienic macros in Racket. The same macro in Racket:

```
(define-syntax-rule (swap a b)
  (let ([tmp a])
    (set! a b)
    (set! b tmp)))
```

`define-syntax-rule` defines `(swap a b)` as a pattern and its expansion as a template. Using the macro's own name for one of the variables does not break it:

```
> (define tmp 10)
> (define y 20)
> (swap tmp y)
> (list tmp y)
'(20 10)
```

Racket's expander is *hygienic*: the `tmp` introduced by the template and the `tmp` the user passed in are different variables, even though they print the same. Effectively the macro's `tmp` is renamed to a fresh name, as `subst` did with `gensym` in Section 4.3, so the expansion is

```
(let ([tmp1 tmp]) (set! tmp y) (set! y tmp1))
```

Key Point 90

Hygiene is static scope for macros. A name written in the macro's template refers to the binding visible where the *macro is defined*; a name passed in by the user refers to the binding visible where the *macro is used*. A text macro like C's resolves every name at the use site, after expansion, which is the macro analogue of dynamic scope, and it has the same symptom: renaming a variable changes what a program does.

5.5.7 A Look at Lean

The last file, `bexp.lean`, revisits the Boolean expressions of Section 2.5 in Lean, a functional language whose type checker can also check *proofs*:

```
inductive BExp where
  | leaf (b : Bool)
  | and (l : BExp) (r : BExp)
  | or (l : BExp) (r : BExp)
  | not (b : BExp)

def eval (b : BExp) : Bool :=
  match b with
  | BExp.leaf b => b
  | BExp.and l r => eval l ∧ eval r
  | BExp.or l r => eval l ∨ eval r
  | BExp.not b => ¬ (eval b)

def opt (b : BExp) : BExp := b

theorem opt_correct : ∀ (b : BExp), eval (opt b) = eval b := by
  intro b
  unfold opt
  exact Bool.le_antisymm (fun a => a) fun a => a
```

`inductive` is `define-type`, and `eval` is the interpreter of Section 2.5 written with `match`. The new part is the `theorem`. In Section 2.5 the optimizer was checked by *testing* that `(eval (opt b))` equals `(eval b)` on

chosen examples. Here the same statement is written for *all* `b` as a type, and the proof after `by` is a program that Lean checks has that type. Here `opt` is still the identity, so after `unfold opt` the goal is `eval b = eval b`; the last line proves it from $\leq$ in both directions (`rfl` would also do). Replacing `opt` by the De Morgan optimizer of Section 2.5 would make the theorem say something; its proof would then need induction on `b`, one case per constructor.

The rest of the example hints at *dependent types*, types that mention values:

- `BExpProof` packages an expression, a result, and a `certificate : eval bexp = result`. A value of this type cannot be built unless the result really is the value of the expression.
- `crazy (xs : Array Int) : (if xs.size > 10 then String else Int)` has a return type that depends on the *length* of its argument. A caller can use the result only after finding out which branch holds, which is what `if h : xs.size > 10` in `crazier` does: `h` is a proof of the condition, available inside the branch.

The bodies are `sorry`, Lean's placeholder that type checks as anything; the example shows what can be *stated*, not how to prove it.

Chapter 6

Week 6: Mutable State

Lecture 6: October 13, 2026. Recitation 6: October 16, 2026.

Assigned reading and resources (course page):

- [1] Chapter 8 (Mutation: Structures and Variables), Section 8.1 in particular, which adds boxes to the closure interpreter with a store as this chapter does.
- [1] Chapter 12 (Representation Decisions), which discusses meta-circular evaluation for `lambda`, the strategy of Section 6.4.
- Lecture code: `lecture06.zip` (see the descriptions at the tops of the files).

Further reading:

- [3] Section 3.1 (Assignment and Local State), for what assignment costs: an expression can no longer be replaced by its value.
- [6], optional: the paper that names the problem of Section 6.4, an interpreter that silently inherits features of the language it is written in.

Every interpreter so far has been *pure*: evaluating an expression produced a value and nothing else, so the same expression in the same environment always gave the same answer. This chapter adds the smallest piece of mutable state, the box of Section 2.4, to the closure interpreter of Section 5.4. The first attempt stores a box's contents directly in the value and cannot express `set-box!`. The fix is a second table next to the environment, the *store*, which maps locations to values and is passed into and out of every evaluation step. We then write the same interpreter a second time using Racket's own boxes, and compare the two. This chapter covers:

- Section 6.1: the four new constructs, the examples, and a first attempt that cannot mutate;
- Section 6.2: store-passing style, with locations, `fetch`, the Result type, and threading the store through arithmetic, `let`, and application;
- Section 6.3: the box clauses of the store-passing interpreter, with traces showing where the environment and where the store come from;

- Section 6.4: the meta-circular interpreter, which implements boxes with Racket boxes and functions with Racket functions, and what each version explains;
- Section 6.5: Recitation 6, mutable variables with `set!`, evaluation order, immutable tables versus boxes, and store-passing with `mret` and `mbind`.

6.1 State and Boxes

6.1.1 Four New Constructs

We start from the closure interpreter of Section 5.4 and add four constructs to Expr: `boxC` creates a box, `unboxC` reads its contents, `setboxC` updates them, and `seqC` is Racket's `(begin e1 e2)`.

```
(define-type Expr
  [numC (n : number)]
  [plusC (e1 : Expr) (e2 : Expr)]
  [timesC (e1 : Expr) (e2 : Expr)]
  [letC (x : symbol) (e1 : Expr) (e2 : Expr)]
  [lambdaC (x : symbol) (e : Expr)]
  [appC (e1 : Expr) (e2 : Expr)]
  [idC (x : symbol)]
  [boxC (e : Expr)]
  [unboxC (e : Expr)]
  [setboxC (e1 : Expr) (e2 : Expr)]
  [seqC (e1 : Expr) (e2 : Expr)])
```

The parser gets one line per construct, in the `case` on the head symbol, with Racket's names as the concrete syntax:

```
[(begin) (seqC (parse (second l)) (parse (third l)))]
[(box) (boxC (parse (second l)))]
[(set-box!) (setboxC (parse (second l)) (parse (third l)))]
[(unbox) (unboxC (parse (second l)))]
```

concrete syntax	abstract syntax	meaning
`(box e)`	`boxC`	a new box whose contents is the value of `e`
`(unbox e)`	`unboxC`	the current contents of the box `e`
`(set-box! e1 e2)`	`setboxC`	make the value of `e2` the contents of the box `e1`
`(begin e1 e2)`	`seqC`	evaluate `e1`, then `e2`; the value is that of `e2`

`begin` takes exactly two expressions, so a longer sequence is written with nested `begin`s. At this point Value is still `numV` and `closV`, and `eval-env` is the one of Section 5.4 with an `else` clause that reports the new variants as unimplemented. The rest of the chapter writes the four clauses, twice.

6.1.2 The Examples

Five test programs:

```
(define ex1 (parse '(begin (+ 1 1) (+ 2 2))))
(define ex2 (parse '(let b (box 0) (set-box! b 1))))
(define ex3 (parse '(let b (box 0) (unbox b))))
(define ex4 (parse '(let b (box 0) (begin (begin (set-box! b 1)
                                                 (set-box! b (+ 1 (unbox b))))
                                          (unbox b)))))

(define ex5
  (parse '(let a (box 1)
            (let f (lambda x (+ x (unbox a)))
              (begin
                (set-box! a 2)
                (f 10))))))
```

Read as Racket programs, they should give:

- `ex1`: 4. The value of `(+ 1 1)` is computed and thrown away.
- `ex2`: whatever `set-box!` returns. Racket's returns `(void)`; this language will return the box (Section 6.3).
- `ex3`: 0, the initial contents.
- `ex4`: 2. The box is set to 1, then to one more than its contents, and read.
- `ex5`: 12. This is the interesting one. The closure `f` is created while `a` holds 1, and called after `a` has been set to 2. The closure saved its environment (Section 5.4), yet `(unbox a)` in its body must see the *new* contents, so $10 + 2 = 12$. Racket itself gives 12 for the same program.

6.1.3 Attempt 1: A Box Holds Its Value

The obvious first try: a box value simply contains the value it holds.

```
(define-type Value
  [numV (n : number)]
  [closV (env : Env) (x : symbol) (e : Expr)]
  [boxV (v : Value)])

    [boxC (e) (boxV (eval-env env e))]
    [unboxC (e) (boxV-v (eval-env env e))]
    [setboxC (e1 e2)
             (let ([v (eval-env env e1)])
               (boxV (eval-env env e2)))]
    [seqC (e1 e2)
          (let ([v (eval-env env e1)])
            (eval-env env e2))]
```

`boxC` and `unboxC` are fine: wrap the value, unwrap it. The trouble is `setboxC`. It evaluates the box `e1` into `v`, and then there is nothing to do with `v`: a `boxV` is an immutable piece of data, so the only option is to build a *new* `boxV` holding the new value. `seqC` evaluates `e1`, ignores the result, and evaluates `e2`. On the examples:

	Attempt 1	expected
ex1	(numV 4)	4
ex2	(boxV (numV 1))	the box, now holding 1
ex3	(numV 0)	0
ex4	(numV 0)	2
ex5	(numV 11)	12

In ex4, each set-box! builds a new box that the enclosing begin throws away, while the environment still binds b to the original (boxV (numV 0)) (Figure 6.1). The final (unbox b) reads that box and gets 0. In ex5, f's saved environment binds a to (boxV (numV 1)) forever, so the call gives 11.

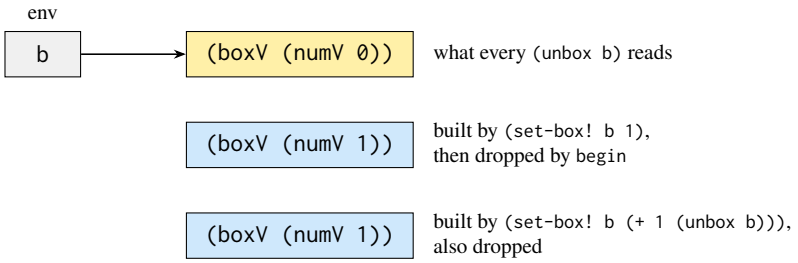


Figure 6.1: Attempt 1 on ex4. Nothing points to the boxes that set-box! builds; the name b still refers to the original, unchanged box. The second one holds 1 because (unbox b) still reads 0.

Pitfall 91 (ex2 looks right)

(eval ex2) is (boxV (numV 1)), exactly what a working set-box! would return, and (unbox (set-box! (box 0) 7)) gives (numV 7). The new contents are visible through the *return value* of set-box!, never through the name of the box. That is functional update, like cons building a new list, not mutation. A test of mutation has to read the box again through another path afterwards, as ex4 and ex5 do.

The same failure shows up whenever two parts of a program hold the same box. In (let a (box 0) (let b a (begin (set-box! b 3) (unbox a)))), a and b name one box, so the answer should be 3; Attempt 1 gives (numV 0). A binding in the environment holds a *copy* of a value, and values are immutable, so a box cannot be a value that contains its contents.

Key Point 92 (Mutation needs a level of indirection)

For a change made through one reference to be seen through another, both must refer to a shared place that holds the contents, rather than contain the contents themselves. The box value must say *where* its contents are, and the contents must be kept somewhere that every part of the evaluation can see and that can change over time.

6.2 Store-Passing Style

6.2.1 Locations and the Store

The indirection is a second table. A box value no longer contains its contents; it contains a *location*, a number that names a cell in a table called the *store*.

Definition 93 (Location, store)

A *location* is an address, here a number. The *store* maps locations to values. A box value is a location; its contents are whatever the store currently maps that location to.

So a name now goes through up to two tables: the environment maps the name to a value, and if that value is a box, the store maps the box's location to its contents. The code for the store copies the code for the environment (Section 5.2) line for line:

```
(define-type-alias Location number)

(define-type Storage
  [cell (l : Location) (val : Value)])

(define-type-alias Store (listof Storage))
(define empty-store empty)
(define override-store cons)

(define (fetch (l : Location) (sto : Store)) : Value
  (cond
    [(cons? sto)
     (if (equal? (cell-l (first sto)) l)
         (cell-val (first sto))
         (fetch l (rest sto)))]
    [else (error 'fetch "No location found")]))
```

	environment	store
maps	names (`symbol`) to values	locations (`number`) to values
entry	`(bind name val)`	`(cell l val)`
add an entry	`extend-env` = `cons`	`override-store` = `cons`
look up	`lookup`, newest first	`fetch`, newest first

`fetch` is `lookup` with the names changed. Writing to a location does not remove its old cell: `override-store` puts a new cell for the same location on the front, and `fetch` stops at the first match. The latest write wins in the same way that the innermost binding wins in `lookup` (Figure 5.3), with shadowing in time instead of shadowing in the program text.

6.2.2 Results Carry the Store

A box value is now a location:

```
(define-type Value
  [numV (n : number)]
  [closV (env : Env) (x : symbol) (e : Expr)]
  [boxV (l : Location)])
```

Evaluating an expression may now change the store, and whoever evaluates next must see the change. So `eval-env` takes the current store as an argument and returns, together with the value, the store as it is *after* the evaluation:

```
(define-type Result
  [res (v : Value) (sto : Store)])

(define (eval-env (env : Env) (sto : Store) (e : Expr)) : Result
  (type-case Expr e
    [numC (n) (res (numV n) sto)]
    [lambdaC (x e) (res (closV env x e) sto)]
    [idC (x) (res (lookup x env) sto)]
    ...))

(define (eval (e : Expr))
  (eval-env empty-env empty-store e))
```

Numbers, `lambda`, and names do not touch the store, so they return the store they were given.

Key Point 94 (Store-passing style)

Every evaluation takes the store as input and returns the updated store as output. An effect is no longer something that *happens*; it is a difference between the store that went in and the store that came out. The interpreter itself stays pure: no Racket mutation is needed to explain the language's mutation (with one exception, `new-loc` below).

6.2.3 Fresh Locations

`(box e)` needs a location that no existing box uses. A first attempt at `new-loc`, commented out because it is wrong:

```
#|
(define new-loc
  (let ([b (box 0)])
    (let ([l (unbox b)])
      (begin
        (set-box! b (+ 1 l))
        l))))
|#
```

There is no `lambda`, so the right-hand side is evaluated once, when the `define` runs: it creates the box, reads 0, sets the box to 1, and returns 0. `new-loc` is just the number 0, and the box is never seen again. Asking for

`new-loc` twice gives 0 both times, and `(new-loc)` does not even type check:

```
> new-loc
- number
0
> (new-loc)
typecheck failed: call of a non-function
```

The fix wraps the body in `(lambda () ...)` inside the `let`:

```
(define new-loc
  (let ([b (box 0)])
    (lambda ()
      (let ([l (unbox b)])
        (begin
          (set-box! b (+ 1 l))
          l)))))
```

Now the `let` runs once and creates one box, and `new-loc` is a closure over it; the body runs at every call. This is the counter pattern: `make-adder` of Section 2.3 with a box in place of a number, so the closure's private state can change.

```
> (new-loc)
- number
0
> (new-loc)
- number
1
> (new-loc)
- number
2
```

Remark 95. `new-loc` implements the language's mutation using Racket's: `set-box!` on a Racket box. It is the one place where the store-passing interpreter is not pure. Only freshness matters, so the effect is harmless, but the counter is global: it is never reset between programs. Evaluating `ex2` twice in one session gives `(res (boxV 0) ...)` the first time and `(res (boxV 1) ...)` the second, with the same contents. The traces in this chapter are each run in a fresh session, so locations start at 0.

6.2.4 Threading the Store Through Arithmetic

`plusC` has two subexpressions, either of which may change the store. The clause evaluates `e1` in the incoming store `sto`, giving `sto1`; evaluates `e2` in `sto1`, giving `sto2`; and returns `sto2`:

```
[plusC (e1 e2)
       (let ([r1 (eval-env env sto e1)])
         (type-case Result r1
           [res (v1 sto1)
                (let ([r2 (eval-env env sto1 e2)])
                  (type-case Result r2
                    [res (v2 sto2)
                         (res (numV (+ (numV-n v1) (numV-n v2))) sto2)])))])]
```

`type-case Result` is used here only to take a `res` apart into its two fields. `timesC` is the same with `*`. The store is *threaded*: each evaluation's output store is the next one's input (Figure 6.2), while `env` is passed unchanged to both operands, as before.

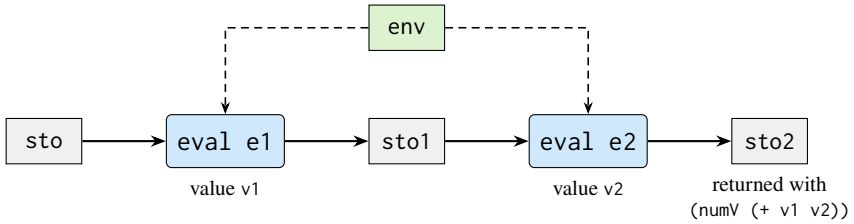


Figure 6.2: Threading the store through `(+ e1 e2)`. The store flows through the operands in sequence; the environment is handed to both unchanged.

Pitfall 96 (Passing the wrong store)

Every store has type Store, so the type checker accepts any of `sto`, `sto1`, `sto2` in any position. Two plausible slips in `plusC` are evaluating `e2` in `sto`, and returning `sto1`. Each is caught by a program that writes to a box inside one operand (these two variants are deliberately wrong):

```
(let b (box 0) (+ (begin (set-box! b 5) 0) (unbox b)))   ; P1
(let b (box 0) (begin (+ 0 (begin (set-box! b 5) 0)) (unbox b)))   ; P2
```

program	correct `plusC`	`e2` gets `sto`	returns `sto1`
P1	`(numV 5)`	`(numV 0)`	`(numV 5)`
P2	`(numV 5)`	`(numV 5)`	`(numV 0)`

Evaluating `e2` in `sto` makes `e2` blind to `e1`'s writes; returning `sto1` forgets `e2`'s writes once the addition is over. Each store must be used exactly once, by the next step.

6.2.5 `let` Is Application, Again

`letC` would need the same threading: evaluate `e1` in `sto`, then the body in the resulting store. We avoid writing it by turning `let` into application, the equation of Section 5.1:

```
[letC (x e1 e2) (eval-env env sto (appC (lambdaC x e2) e1))]
```

The rewriting happens in `eval-env` rather than in `parse`, but the effect is the same: the store is threaded through `let` by the code for `appC`, written once. The equation is valid here because both `let` and application are eager.

6.2.6 Application

`appC` evaluates the function in `sto` and the argument in `sto1`, as `plusC` did, and then runs the body:

```
[appC (e1 e2)
      (type-case Result (eval-env env sto e1)
        [res (v1 sto1)
             (type-case Result (eval-env env sto1 e2)
               [res (v2 sto2)
                    (eval-env (extend-env (bind (closV-x v1) v2) (closV-env v1))
                              sto2
                              (closV-e v1))])])]
```

The body gets its environment and its store from different places. The environment is the closure's, `(closV-env v1)`, extended with the parameter: lexical scope, unchanged from Section 5.4: the body uses and extends the environment saved in the closure, not the current one. The store is `sto2`, the *current* one. A closure saves an environment but no store, and `closV` has no field for one.

Key Point 97 (Environment vs. store)

The environment follows the program text: it is saved in closures and restored when they are called, so a name means what it meant where it was written. The store follows time: it is passed from each step to the next in execution order and is never saved or restored, so a box's contents are whatever was last written, wherever the reading code was written.

If closures saved the store too, `f` in `ex5` would read `a` in the store of the moment `f` was created, where it holds 1, and `ex5` would give 11, the wrong answer of Attempt 1. Section 6.3 traces `ex5` and draws both tables.

6.3 Boxes in the Store-Passing Interpreter

6.3.1 The Box Clauses

With the store in place, the four new clauses are short. `boxC` evaluates the initial contents, takes a fresh location, and adds a cell for it; `unboxC` evaluates the box and fetches its location:

```
[boxC (e)
      (type-case Result (eval-env env sto e)
        [res (v sto1)
             (let ([l (new-loc)])
               (res (boxV l) (override-store (cell l v) sto1)))])]
[unboxC (e)
        (type-case Result (eval-env env sto e)
          [res (v1 sto1)
               (let ([v2 (fetch (boxV-l v1) sto1)])
                 (res v2 sto1))])]
```

Note that `unboxC` fetches from `sto1`, the store *after* evaluating `e`, in case evaluating the box expression itself wrote to it. `setboxC` evaluates the box and then the new contents, threading the store as `plusC` does, and adds a cell for the box's location on top of `sto2`:

```
[setboxC (e1 e2)
         (type-case Result (eval-env env sto e1)
           [res (v1 sto1)
                (type-case Result (eval-env env sto1 e2)
                  [res (v2 sto2)
                       (res v1 (override-store (cell (boxV-l v1) v2) sto2))])])]
[seqC (e1 e2)
      (type-case Result (eval-env env sto e1)
        [res (v1 sto1)
             (eval-env env sto1 e2)])]
```

Two details:

- `setboxC` returns `v1`, the box itself, not the new contents and not a void value (the language has none). This is a design choice; Racket's `set-box!` returns `(void)`.
- `seqC` discards `v1` but *not* `sto1`: `e2` runs in the store that `e1` left behind. In Attempt 1 the `seqC` clause looked the same, but there was no store to pass on, so `e1` could only be evaluated and forgotten. Here sequencing finally has a purpose: `e1` is run for its effect on the store.

6.3.2 Running the Examples

Each in a fresh session, so that `new-loc` starts at 0:

```
> (eval ex1)
- Result
(res (numV 4) '())
> (eval ex2)
- Result
(res (boxV 0) (list (cell 0 (numV 1)) (cell 0 (numV 0))))
```

```
> (eval ex3)
- Result
(res (numV 0) (list (cell 0 (numV 0))))
> (eval ex4)
- Result
(res (numV 2) (list (cell 0 (numV 2)) (cell 0 (numV 1)) (cell 0 (numV 0))))
> (eval ex5)
- Result
(res (numV 12) (list (cell 0 (numV 2)) (cell 0 (numV 1))))
```

All five are now right. `ex1` allocates nothing, so its final store is empty. In `ex2` the value is the box, location 0, and the store holds two cells for location 0, the write of 1 in front of the initial 0.

6.3.3 Tracing ex4

Write the store newest first, with `l:v` for `(cell l v)`. The outer `let` becomes an application of `(lambda b ...)` to `(box 0)`:

```
(let b (box 0) (begin (begin (set-box! b 1)
                             (set-box! b (+ 1 (unbox b))))
                      (unbox b)))
  (lambda b ...)                          ---> closure, sto = []
  (box 0)       new location 0            sto = [0:0]
  body in env [b=(boxV 0)]
    (set-box! b 1)                        sto = [0:1, 0:0]
    (set-box! b (+ 1 (unbox b)))
      (unbox b): fetch 0 ---> 1
      (+ 1 1) ---> 2                      sto = [0:2, 0:1, 0:0]
    (unbox b): fetch 0 ---> 2
---> (numV 2) with sto = [0:2, 0:1, 0:0]
```

The environment never changes after the `let`: `b` is bound to `(boxV 0)` throughout. Every change happens in the store, and each `begin` hands the store from its first part to its second.

6.3.4 Tracing ex5: Two Tables, Two Paths

```
(let a (box 1)
  (let f (lambda x (+ x (unbox a)))
    (begin (set-box! a 2) (f 10))))
  (box 1)       new location 0            sto = [0:1]
  env [a=(boxV 0)]
  (lambda x ...) ---> f = (closV [a=(boxV 0)] x (+ x (unbox a)))
  env [f=closure, a=(boxV 0)]
  (set-box! a 2)                          sto = [0:2, 0:1]
  (f 10)
    body env:   the closure's [a=(boxV 0)] + x=10
    body store: the current one,          sto = [0:2, 0:1]
    (unbox a): fetch 0 ---> 2
    (+ 10 2) ---> 12
---> (numV 12) with sto = [0:2, 0:1]
```

The closure saved the binding `a=(boxV 0)`: a location, not a number. The location has not changed since the closure was created, but the store has, and the call reads the current store (Figure 6.3). This is how lexical scope and mutation coexist: the closure fixes *which* box `a` is, and the store says what is in it *now*.

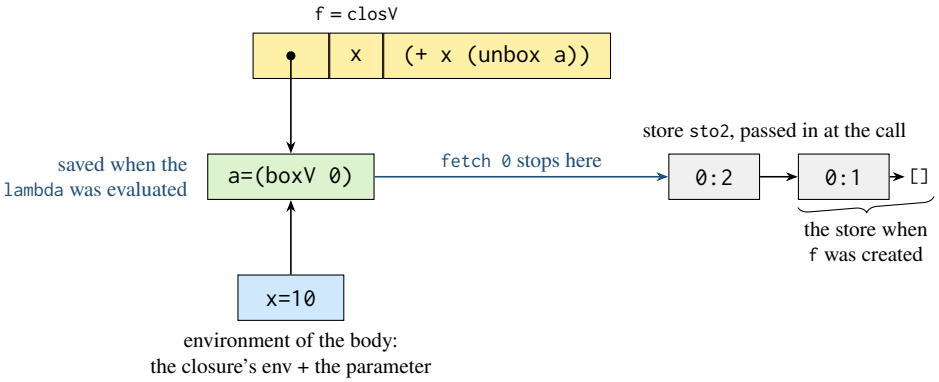


Figure 6.3: `ex5` while evaluating `(unbox a)` in `f`'s body. The environment (left) comes from the closure; the store (right) is the current one. The store at closure-creation time is still there as a suffix, but nothing reads it, since `fetch` stops at the newer cell.

Key Point 98 (Where the environment and the store come from)

	environment	store
to subexpressions	the same `env` to each	chained: `sto`, then `sto1`, ...
returned by `eval-env`	never	always, in the `res`
saved in a closure	yes	no
used for a function body	the closure's, extended	the current one, `sto2`
follows	the program text	the order of evaluation

6.3.5 The Store Only Grows

Nothing ever removes a cell. `set-box!` conses a new cell for an existing location, and the old one stays behind it, unreachable by `fetch`: after `ex4` the store holds three cells for location 0, of which only the first matters. Boxes that no part of the program can reach any more also keep their cells. A real implementation would overwrite a location in place and reclaim the memory of unreachable boxes with a garbage collector [1, Chapter 11]. For explaining what mutation *means*, a store that only grows is simpler: every past state is still visible in the trace, and the store after any step is the store before it with some cells added on the front.

6.4 The Meta-Circular Interpreter

6.4.1 Boxes Implemented by Boxes

The store-passing interpreter needed a new type, a new argument and result for `eval-env`, and a rewrite of every clause. The meta-circular interpreter starts again from the closure interpreter with the four new constructs, and takes a shortcut: the object language's boxes are implemented with Racket's boxes.

Definition 99 (Meta-circular interpreter)

An interpreter is *meta-circular* when it implements a feature of the language it interprets by the same feature of the language it is written in. SICP uses the word for an evaluator written in the language that it evaluates [3, Section 4.1]; here it applies feature by feature: first to boxes, then to functions.

A box value holds a Racket box of values, `(boxof Value)`. `eval-env` goes back to the signature of Section 5.4, `(env e) : Value`, with no store, and every old clause is unchanged:

```
(define-type Value
  [numV (n : number)]
  [closV (env : Env) (x : symbol) (e : Expr)]
  [boxV (b : (boxof Value))])

    [boxC (e)
          (boxV (box (eval-env env e)))]
    [unboxC (e)
            (unbox (boxV-b (eval-env env e)))]
    [setboxC (e1 e2)
             (let ([v1 (eval-env env e1)]
                   [v2 (eval-env env e2)])
               (begin
                 (set-box! (boxV-b v1) v2)
                 v1))]
    [seqC (e1 e2)
          (begin
            (eval-env env e1)
            (eval-env env e2))]
```

Each clause does in Racket what the construct does in the object language. `(box e)` makes a Racket box holding the value of `e` and wraps it in `boxV`. `unbox` unwraps and reads it. `set-box!` evaluates the box and the new contents and mutates the Racket box; like the store-passing version, it returns the box `v1`. `seqC` is Racket's `begin`.

Remark 100. Once all eleven variants have clauses, the `else` clause of Section 6.1 can never be reached, and `plai-typed` rejects the program: "the else branch of this type-case is unreachable". Deleting the `else` fixes it.

Compare this `setboxC` with Attempt 1's. There, `boxV` contained a `Value` directly, and `set-box!` could only build a new `boxV`. Here `boxV` contains a Racket box, a reference to a mutable cell. Copying the `boxV` into an environment or a closure copies the reference, not the cell, so every copy sees the same contents. This is the indirection of Section 6.1, supplied by Racket. The results agree with the store-passing interpreter on all five

examples:

```
> (eval ex1)
- Value
(numV 4)
> (eval ex2)
- Value
(boxV (box (numV 1)))
> (eval ex3)
- Value
(numV 0)
> (eval ex4)
- Value
(numV 2)
> (eval ex5)
- Value
(numV 12)
```

In `ex5`, the closure for `f` saves an environment in which `a` is bound to a `boxV` holding a Racket box; `(set-box! a 2)` mutates that Racket box, and the call reads it.

6.4.2 Functions Implemented by Functions

The same idea applies to functions. A function value no longer records its parameter, body, and environment; it simply *is* a Racket function from values to values. The constructor keeps the name `closV`:

```
(define-type Value
  [numV (n : number)]
  [closV (f : (Value -> Value))]
  [boxV (b : (boxof Value))])

    [lambdaC (x e)
             (closV (lambda (arg-value)
                      (eval-env (extend-env (bind x arg-value) env) e)))]
    [appC (e1 e2)
          (let [(v1 (eval-env env e1))
                (v2 (eval-env env e2))]
            ((closV-f v1) v2))]
```

`lambdaC` builds a Racket `lambda` that, given the argument's value, evaluates the body in `env` extended with `x`. `appC` evaluates the function and the argument and calls the Racket function. The three accessors `closV-env`, `closV-x`, and `closV-e` of Section 5.4 are gone.

The `env` inside the Racket `lambda` is the environment of the `lambdaC` clause, the one where the object-language `lambda` was evaluated. Racket's own closure saves it, so the object language gets lexical scope without any

code for it:

```
> (eval (parse '((lambda x (+ 2 x)) 1)))
- Value
(numV 3)
> (eval (parse '(let x 2 (let addx (lambda y (+ y x)) (let x 6 (addx 3))))))
- Value
(numV 5)
> (eval (parse '(lambda x x)))
- Value
(closV #<procedure>)
```

The `addx` example of Section 5.4 gives 5, the lexical answer, and `ex5` still gives 12. A function value now prints as an opaque Racket procedure: its parameter, body, and saved environment can no longer be inspected. In Section 5.4 it took two versions, 5d and 5e, to get closures right; here the right answer comes for free, and for the same reason as the boxes, it explains nothing: the interpreter has lexical scope because Racket does.

6.4.3 Side by Side

	store-passing	meta-circular
box value	`(boxV l)`, a location	`(boxV b)`, a Racket box
function value	`(closV env x e)`	`(closV f)`, a Racket function
contents live in	the store, an explicit list of cells	Racket's heap
`eval-env`	`(env sto e) : Result`	`(env e) : Value`
`set-box!`	adds a cell to the store it returns	Racket's `set-box!`
`begin`	passes `sto1` from `e1` to `e2`	Racket's `begin`
order of evaluation	forced by the store: `e2` needs `sto1`	whatever Racket does
Racket mutation used	only in `new-loc`	for every box operation
old clauses	all rewritten to thread the store	unchanged, except `lambdaC` and `appC` once functions are meta-circular

6.4.4 What Each Version Explains

The meta-circular interpreter is much shorter, and it is correct, but it explains mutation *by* mutation. Someone who does not already know what `set-box!` does learns nothing from `[setboxC ... (set-box! (boxV-b v1) v2) ...]`. Two things in particular are inherited from Racket without appearing in the code:

- **The store.** Where the contents of a box live, and what happens to them over time, is Racket's business. The store-passing interpreter writes the store down as a value that can be printed, as in the `(res ...)` results above.
- **The order of evaluation.** In `setboxC`, `e1` is evaluated before `e2` only because Racket's `let` evaluates its right-hand sides left to right; in `plusC`, only because Racket evaluates the arguments of `+` left to right. In the store-passing interpreter the order is forced by data dependencies: `e2` cannot be evaluated until `sto1` exists, and `sto1` is produced by evaluating `e1`.

Once there is mutation, order matters. Consider the program

```
(let b (box 1) (+ (unbox b) (begin (set-box! b 10) (unbox b))))
```

Both interpreters give `(numV 11)`: the left operand reads 1 before the right operand writes 10. Evaluating the right operand first would give 20. Without mutation the two orders could never be told apart; with it, the order is part of the meaning of `+`, and the meta-circular interpreter specifies it only by reference to Racket. Reynolds pointed out this kind of dependence: a definitional interpreter can silently inherit the evaluation order of the language it is written in [6].

Key Point 101 (What an interpreter explains)

An interpreter explains a feature only if it implements it with something simpler. Chapter 5 implemented environments with Racket lists and `plusC` with Racket's `+`; that was fine, because lists and addition were not what was being studied, and scope was implemented explicitly. This week the object of study is mutation, so implementing it with Racket's mutation is circular; implementing functions with Racket's functions likewise hides how closures capture their environment. The store-passing interpreter explains mutation with immutable lists and function arguments and results, and in doing so also makes evaluation order explicit.

There is one remaining circle in the store-passing interpreter: `new-loc` uses a Racket box. It is not essential: `new-loc` only has to produce a location not yet in the store, and a counter threaded alongside the store would do. It is the same kind of shortcut as `gensym` in Section 4.3, a source of fresh names rather than the subject of the chapter.

6.5 Recitation 6: Mutable State and Tables

This section reviews the store-passing design, makes evaluation order visible, and then adds a new kind of value, an immutable *table* (key-value map), to see which things a store makes changeable and which it does not. It ends with a rewrite of part of the interpreter in which the store-passing is done by two helper functions.

6.5.1 The Language

The language has numbers, `+`, `*`, `let`, `lambda`, application, boxes, `begin`, and mutable variables with `set!`, plus four forms for tables:

```
(make-table (k1 e1) (k2 e2) ...)   a new table mapping each key ki to the value of ei
(get-table t k)                    the value for key k in table t
(set-table t k v)                  a NEW table: like t, but with k mapped to v
(remove-table t k)                 a NEW table: like t, but without k
```

Two design decisions are fixed:

- **Keys are symbols written in the program**, like field names, not expressions. `(get-table t (+ 1 2))` is not allowed; the key is known when the program is parsed.
- **Tables never change.** `set-table` and `remove-table` leave `t` alone and return a new table, like Racket's `hash-set` and `hash-remove`. Most of Section 6.5.7 follows from this.

```
(define-type Entry
  [entry (k : symbol) (e : Expr)])

(define-type Expr
  [numC (n : number)]
  [plusC (e1 : Expr) (e2 : Expr)]
  [timesC (e1 : Expr) (e2 : Expr)]
  [letC (x : symbol) (e1 : Expr) (e2 : Expr)]
  [lambdaC (x : symbol) (e : Expr)]
  [appC (e1 : Expr) (e2 : Expr)]
  [idC (x : symbol)]
  [boxC (e : Expr)]
  [unboxC (e : Expr)]
  [setboxC (e1 : Expr) (e2 : Expr)]
  [setC (x : symbol) (e2 : Expr)]
  [seqC (e1 : Expr) (e2 : Expr)]
  [maketableC (entries : (listof Entry))]
  [gettableC (tbl : Expr) (k : symbol)]
  [settableC (tbl : Expr) (k : symbol) (val : Expr)]
  [removetableC (tbl : Expr) (k : symbol)])

(define-type-alias Table (hashof symbol Value))

(define-type Value
  [numV (n : number)]
  [closV (env : Env) (x : symbol) (e : Expr)]
  [boxV (l : Location)]
  [tableV (t : Table)])
```

Compare the two new kinds of value. A `boxV` holds a *location*: the contents live in the store, so they can change while the `boxV` itself stays the same. A `tableV` holds the whole Table directly. There is nothing in it that points into the store, so a table value, once made, is fixed.

Entry is a separate type for the same reason as PatternE in Section 4.5: one `(k e)` pair carries a symbol and an expression, and a `maketableC` has a list of them.

```
(define (parse-entry (s : s-expression)) : Entry
  (let ([l (s-exp->list s)])
    (entry (s-exp->symbol (first l)) (parse (second l)))))
```

`parse` gets one line per new form:

```
           [(set!) (setC (s-exp->symbol (second l)) (parse (third l)))]
           [(make-table) (maketableC (map parse-entry (rest l)))]
           [(get-table) (gettableC (parse (second l)) (s-exp->symbol (third l)))]
           [(set-table) (settableC (parse (second l)) (s-exp->symbol (third l))
                                   (parse (fourth l)))]
           [(remove-table) (removetableC (parse (second l)) (s-exp->symbol (third l)))]
```

The keys go through `s-exp->symbol`, not `parse`: that is where "keys are not expressions" is enforced. The two `else` branches of `parse` send everything that is not a keyword to application, whether the head is a variable like `(f 10)` or an expression like `((get-table t inc) 41)`.

6.5.2 Environments and the Store

With `set!`, the meaning of a name is split over two maps:

```
env   : symbol   -> Location      (fixed when the binding is made)
store : Location -> Value         (can be overridden later)
```

The store, `fetch`, Result, and `new-loc` are those of Section 6.2. What changes is the environment: a binding now holds a location, and `lookup` returns it.

```
(define-type Binding
  [bind (name : symbol) (loc : Location)])
(define-type-alias Env (listof Binding))

(define (lookup (x : symbol) (env : Env)) : Location
  (cond
    [(cons? env)
     (if (equal? (bind-name (first env)) x)
         (bind-loc (first env))
         (lookup x (rest env)))]
    [else (error 'lookup "No binding found")]))
```

In `eval-env`, the cases for numbers, `lambda`, `+`, `*`, `let`, `begin`, `box`, and `unbox` are as in Sections 6.2 and 6.3.

These four differ:

```
    [appC (e1 e2)
          (type-case Result (eval-env env sto e1)
            [res (v1 sto-1)
                 (type-case Result (eval-env env sto-1 e2)
                   [res (v2 sto-2)
                        (let ([l (new-loc)])
                          (eval-env
                           (extend-env (bind (closV-x v1) l) (closV-env v1))
                           (override-store (cell l v2) sto-2)
                           (closV-e v1)))])])]
    [setC (x e)
          (type-case Result (eval-env env sto e)
            [res (v sto-1)
                 (let [(l (lookup x env))]
                   (res v (override-store (cell l v) sto-1)))])]
    [idC (x) (res (fetch (lookup x env) sto) sto)]
    [setboxC (e1 e2)
             (type-case Result (eval-env env sto e1)
               [res (v1 sto-1)
                    (type-case Result (eval-env env sto-1 e2)
                      [res (v2 sto-2)
                           (res v2 (override-store (cell (boxV-l v1) v2) sto-2))])])]
```

What to notice:

- **Every application allocates.** `appC` gets a fresh location for the parameter and stores the argument there, and `letC` is application in disguise. So every variable has its own cell, and that is what lets `set!` work.
- **`set!` and `set-box!` both override a cell**, but they find it differently. `setC` looks the *name* up in the environment; `setboxC` takes the location out of the *value* it evaluated. `set!` can only reach the variable it names; a box can be passed around, and whoever holds it can change it.
- **`set-box!` returns the new contents** `v2` here, where the interpreter of Section 6.3 returns the box `v1`. Of the examples below, only `ex2` sees the difference.
- **The store is threaded, never shared.** Each case passes the store that the previous subexpression returned to the next one. Which subexpression goes first is a choice the interpreter makes (Section 6.5.4).

```
(define (eval (e : Expr)) : Result
  (eval-env empty-env empty-store e))

(define (run (s : s-expression)) : Value
  (res-v (eval (parse s))))
```

6.5.3 Warm-up

Exercise 102 (The examples of Section 6.1, and `set!`)

Predict the value of each program before running it.

```
(define ex0 '(let v (+ 1 1) (+ v v)))
(define ex1 '(begin (+ 1 1) (+ 2 2)))
(define ex2 '(let b (box 0) (set-box! b 1)))
(define ex3 '(let b (box 0) (unbox b)))
(define ex4 '(let b (box 0) (begin (begin (set-box! b 1)
                                          (set-box! b (+ 1 (unbox b))))
                                   (unbox b))))
(define ex5 '(let a (box 1)
               (let f (lambda x (+ x (unbox a)))
                 (begin
                   (set-box! a 2)
                   (f 10)))))
(define ex6 '(let v (+ 1 1) (begin (set! v 3) (+ v v))))
```

Solution.

```
(test (run ex0) (numV 4))
(test (run ex1) (numV 4))
(test (run ex2) (numV 1))
(test (run ex3) (numV 0))
(test (run ex4) (numV 2))
(test (run ex5) (numV 12))
(test (run ex6) (numV 6))
```

ex	result	why
ex1	4	`begin` throws away the first value
ex2	1	`set-box!` returns the new contents (`v2`)
ex4	2	the second `set-box!` reads the store the first one returned
ex5	12	`f` reads the box when it is *called*: $10 + 2$
ex6	6	`set!` overrides `v`'s cell, then $3 + 3$

`ex5` is the one to understand. The closure for `f` captures the environment, which maps `a` to a location. The closure does not capture the store. By the time `(f 10)` runs, the store says that `a`'s box holds 2.

6.5.4 Evaluation Order

Our interpreter evaluates `e1` before `e2` in `(+ e1 e2)`, because it has to pick one to receive the store first. With side effects, the choice is visible:

```
(define ex-order
  '(let b (box 10)
     (+ (begin (set-box! b 5) 4)
        (unbox b))))

(test (run ex-order) (numV 9))
```

```
left to right:  (begin (set-box! b 5) 4) runs first, so (unbox b) sees 5:   4 + 5  = 9
right to left:  (unbox b) runs first and sees 10:                           4 + 10 = 14
```

Real languages make different choices. The same program can be written in C, Racket, and OCaml: `first()` adds 1 to a global counter, `second()` adds 10, and each prints the counter. Both are called once as the

arguments of a function and once as the operands of +. The C version:

```c
#include <stdio.h>

int count = 0;

__attribute__((noinline))
int first(void) {
    count += 1;
    printf("[first]  count = %d\n", count);
    return 10;
}

__attribute__((noinline))
int second(void) {
    count += 10;
    printf("[second] count = %d\n", count);
    return 20;
}

__attribute__((noinline))
void print_sum(int a, int b) {
    printf("Result of a + b: %d\n", a + b);
}

int main(void) {
    printf("Demonstration 1: Function Argument Evaluation Order\n");
    print_sum(first(), second());

    count = 0; // reset count

    printf("Demonstration 2: Expression Operand Evaluation Order\n");
    int total = first() + second();
    printf("Total: %d\n", total);

    return 0;
}
```

Compiled with Apple clang 21, at `-O0` and `-O2`, it prints:

```
Demonstration 1: Function Argument Evaluation Order
[first]  count = 1
[second] count = 11
Result of a + b: 30
Demonstration 2: Expression Operand Evaluation Order
[first]  count = 1
[second] count = 11
Total: 30
```

language	order	guaranteed?
Racket	left to right	yes, by the language definition
C	Apple clang 21, `-O0` and `-O2`: left to right	no: unspecified
OCaml	right to left in practice	no: unspecified

In C the order is "arbitrary": the standard leaves it unspecified for both arguments and operands, so another compiler (GCC on x86 is the usual example) may print `[second]` first. The sum is 30 either way. Only the trace, and anything else the calls do to `count`, depends on the order. (The Racket version never resets `count`, so its second demonstration prints 12 and 22 instead of 1 and 11; the order is the same.) The OCaml version is named for OCaml's usual right-to-left order; it was not run here.

The lesson for interpreters: the language designer *chooses* the order, and in a store-passing interpreter the choice is written down explicitly by which `eval-env` call gets `sto` and which gets `sto-1`. In Section 6.5.7, `ex-f` checks the order for `set-table`.

6.5.5 Table Helpers

A Table is a Racket hash. The four operations we need:

```
(hash empty)       an empty hash
(hash-ref t k)     (some v) if k maps to v in t, (none) otherwise
(hash-set t k v)   a new hash: like t, but with k mapped to v
(hash-remove t k)  a new hash: like t, but without k
```

Exercise 103 (`expect-table`)

Write `(expect-table v)`, which returns the table inside `v`, or signals the error `"expected a table"` if `v` is not a table. Hint: `type-case` can have an `else` case, for "every other kind of value".

Solution.

```
(define (expect-table (v : Value)) : Table
  (type-case Value v
    [tableV (t) t]
    [else (error 'expect-table "expected a table")]))
```

`eval-env` gets back a Value, and only a `tableV` can be used as a table. Instead of repeating that check in three cases, it gets its own helper. Without it we would write `(tableV-t v)`, and a non-table would fail with a message about the accessor instead of about the program.

Exercise 104 (`make-table`)

Write `(make-table env sto entries acc)`, which evaluates each entry's expression, left to right, passing the store along, and adds each key and its value to `acc`. It returns the finished table, along with the final store. If a key appears twice, the later entry wins. Hint: look at how the `plusC` case passes the store from `e1` to `e2`.

Solution. This is the only helper that runs code. It is the `plusC` pattern stretched over a list, plus an accumulator for the table built so far:

```
(define (make-table (env : Env) (sto : Store) (entries : (listof Entry)) (acc : Table))
  : Result
  (cond
    [(empty? entries) (res (tableV acc) sto)]
    [else
     (type-case Entry (first entries)
       [entry (k e)
              (type-case Result (eval-env env sto e)
                [res (v sto-1)
                     (make-table env sto-1 (rest entries) (hash-set acc k v))])])]))
```

- The recursive call gets `sto-1`, not `sto`. Passing `sto` would quietly drop the first entry's side effects; the `set-box!` test in Section 6.5.6 catches that.
- "The later entry wins" for a repeated key comes for free: `hash-set` on an existing key replaces it, and later entries are added later.
- It returns a `Result`, not a `Table`, because the caller needs the final store too.

Exercise 105 (`get-table`, `set-table`, `remove-table`)

Write `(get-table t k)`, which returns the value for `k` in `t` or signals `(string-append "no such key: " (symbol->string k))`; `(set-table t k v)`, a new table with `k` mapped to `v`; and `(remove-table t k)`, a new table without `k`, which signals the same error as `get-table` if `t` has no key `k`. Hint: which helper already signals that error?

Solution. These three are pure: no store, no evaluation, just hash operations. `hash-ref` returns an `optionof`, taken apart with `type-case` as in Section 5.5:

```
(define (get-table (t : Table) (k : symbol)) : Value
  (type-case (optionof Value) (hash-ref t k)
    [some (v) v]
    [none () (error 'get-table
                    (string-append "no such key: " (symbol->string k)))]))

(define (set-table (t : Table) (k : symbol) (v : Value)) : Table
  (hash-set t k v))

(define (remove-table (t : Table) (k : symbol)) : Table
  (begin (get-table t k)
         (hash-remove t k)))
```

`hash-remove` on a missing key silently returns the same hash, but `remove-table` must raise an error. `get-table`

already signals it, so `remove-table` calls it only for its error and throws the value away with `begin`.

```
(test (expect-table (tableV (hash empty))) (hash empty))
(test/exn (expect-table (numV 1)) "expected a table")

(define t-ab (hash (list (values 'a (numV 1)) (values 'b (numV 2)))))
(test (get-table t-ab 'b) (numV 2))
(test/exn (get-table t-ab 'c) "no such key: c")
(test (get-table (set-table t-ab 'c (numV 3)) 'c) (numV 3))
(test (get-table (set-table t-ab 'a (numV 3)) 'a) (numV 3))
(test (get-table t-ab 'a) (numV 1))
(test/exn (get-table (remove-table t-ab 'a) 'a) "no such key: a")
(test/exn (remove-table t-ab 'c) "no such key: c")
```

The fifth test is the immutability promise at the Racket level: `set-table` made a new hash, and `t-ab` still maps `a` to 1.

6.5.6 Tables in the Interpreter

Exercise 106 (The four table cases)

Fill in the four table cases of `eval-env`. `maketableC` uses `make-table`, starting from an empty hash. `gettableC`, `settableC`, and `removetableC` evaluate `tbl` and use `expect-table` on the result. In `settableC`, evaluate `tbl` *before* `val`, and pass the store from one to the other.

Solution. Each case evaluates its subexpressions while threading the store, then calls a helper:

```
[maketableC (entries) (make-table env sto entries (hash empty))]
[gettableC (tbl k)
           (type-case Result (eval-env env sto tbl)
             [res (v sto-1) (res (get-table (expect-table v) k) sto-1)])]
[settableC (tbl k val)
           (type-case Result (eval-env env sto tbl)
             [res (v1 sto-1)
                  (let ([t (expect-table v1)])
                    (type-case Result (eval-env env sto-1 val)
                      [res (v2 sto-2) (res (tableV (set-table t k v2)) sto-2)]))])]
[removetableC (tbl k)
              (type-case Result (eval-env env sto tbl)
                [res (v sto-1) (res (tableV (remove-table (expect-table v) k)) sto-1)])]
```

`settableC` is the only case with two subexpressions, so it is the only one where order matters. It evaluates `tbl` first, then `val` in `sto-1`. Here `expect-table` is called *before* `val` is evaluated, so `(set-table 5 a (set-box! b 1))` stops before `val` has changed the store. Checking after `val` would also pass every test below; the difference is only whether the side effects of `val` happen before the error.

```
(test (run '(get-table (make-table (a 1) (b 2)) b)) (numV 2))
(test (run '(get-table (make-table (a (+ 1 2))) a)) (numV 3))
(test (run '(make-table)) (tableV (hash empty)))
(test (run '(make-table (a 1) (b 2))) (tableV t-ab))
(test (run '(get-table (make-table (a 1) (a 2)) a)) (numV 2))   ; the later entry wins
(test/exn (run '(get-table (make-table (a 1)) b)) "no such key: b")

(test (run '(get-table (set-table (make-table (a 1)) b 2) b)) (numV 2))
(test (run '(get-table (set-table (make-table (a 1)) a 2) a)) (numV 2))
(test/exn (run '(get-table (remove-table (make-table (a 1)) a) a)) "no such key: a")
(test/exn (run '(remove-table (make-table (a 1)) b)) "no such key: b")
```

Entries are evaluated in the current environment, and can hold any value, here a table and a function:

```
(test (run '(let y 5 (get-table (make-table (a y)) a))) (numV 5))
(test (run '(get-table (get-table (make-table (inner (make-table (x 7)))) inner) x))
      (numV 7))
(test (run '(let t (make-table (inc (lambda x (+ x 1))))
              ((get-table t inc) 41)))
      (numV 42))
```

The `inc` test shows a table holding a closure, and why `parse`'s `else` branch must handle a non-symbol head: `((get-table t inc) 41)` is an application whose function part is a `get-table` expression. `make-table` passes the store from one entry to the next, so the second entry sees the first entry's `set-box!`; and using something that is not a table as a table is an error:

```
(test (run '(let b (box 0)
              (get-table (make-table (x (set-box! b 1)) (y (unbox b))) y)))
      (numV 1))

(test/exn (run '(get-table 5 a)) "expected a table")
(test/exn (run '(set-table (box 1) a 2)) "expected a table")
(test/exn (run '(remove-table (lambda x x) a)) "expected a table")
```

6.5.7 Tables vs. Boxes

Exercise 107 (What can change?)

Predict what each program evaluates to before running it.

```
;; a) set-table returns a new table. What happens to t?
(define ex-a
  '(let t (make-table (a 1))
     (begin
       (set-table t a 2)
       (get-table t a))))

;; b) This time we use the table that set-table returns.
(define ex-b
  '(let t (make-table (a 1))
     (get-table (set-table t a 2) a)))

;; c) set! can change what the *variable* t refers to.
(define ex-c
  '(let t (make-table (a 1))
     (begin
       (set! t (set-table t a 2))
       (get-table t a))))

;; d) Two variables, one table. Does u see the change?
(define ex-d
  '(let t (make-table (a 1))
     (let u t
       (begin
         (set! t (set-table t a 2))
         (get-table u a)))))

;; e) A box inside a table.
(define ex-e
  '(let b (box 1)
     (let t (make-table (a b))
       (begin
         (set-box! b 5)
         (unbox (get-table t a))))))

;; f) Which runs first: the table or the value?
(define ex-f
  '(let b (box 0)
     (get-table (set-table (begin (set-box! b 1) (make-table))
                           a
                           (unbox b))
                a)))
```

Solution.

```
(test (run ex-a) (numV 1))
(test (run ex-b) (numV 2))
(test (run ex-c) (numV 2))
(test (run ex-d) (numV 1))
(test (run ex-e) (numV 5))
(test (run ex-f) (numV 1))
```

ex	result	why
a	1	the new table is thrown away by `begin`; `t` never changed
b	2	reads the new table directly
c	2	`set!` puts the new table in `t`'s cell
d	1	`u` has its own cell, holding the old table
e	5	the table holds a *box*; the box's contents changed
f	1	the table expression runs first, so `(unbox b)` sees 1

The way to keep these straight is to ask, for each program, which of the two maps changes.

```
ex-d, just before (get-table u a):

  env:   t -> L0        u -> L1
  store: L0 -> T2       (overridden by set!)
         L1 -> T1       (copied from t when u was bound; set! never touched it)

  T1 = {a: 1}    T2 = {a: 2}      two separate, unchanging tables
```

- `set!` changes the *store*, at the one location its variable names. In `ex-c` that is the location we read from; in `ex-d` it is not. `(let u t ...)` copied the table *value* into `u`'s new cell, and that value cannot change.
- A table can still *contain* something mutable. In `ex-e` the table maps `a` to `(boxV L)`, and that does not change. What changes is the contents of `L` in the store, which is exactly what `unbox` reads.
- `ex-f` is the evaluation-order test of Section 6.5.4, for `settableC`. Right to left would give 0.

Exercise 108 (Tables of locations)

Another design: a table maps each key to a *location* in the store, and the store holds the value. Would that change any of the answers above? When would it matter?

Solution. It depends on what `set-table` does with that design.

- If `set-table` *allocates a fresh location* for `k` and copies the other key-to-location pairs, nothing ever writes to an existing table's cell, so all six answers stay the same. The difference is hidden sharing: the old and new tables now point to the *same* cells for every other key. As soon as we add an operation that writes to a key's cell (an in-place `update-table!`), a write through one table shows up in the other.
- If `set-table` *overrides the existing location* for a key that is already present, that is in-place mutation, the case of the next exercise.

Exercise 109 (Mutable tables)

Which of `ex-a` to `ex-f` would change if tables could be changed in place, like Racket's mutable hash tables or JavaScript objects?

Solution. `ex-a` and `ex-d`. In place, `set-table` changes the one table that both the old and new names refer to:

ex	immutable	in place	why it changes
a	1	2	the "thrown away" result was `t` itself
d	1	2	`t` and `u` alias one table
b, c, e, f	same	same	they read the result, or never use `set-table` on a shared table

`ex-d` is the aliasing problem in its usual form. In JavaScript, `let u = t; t.a = 2; u.a` gives 2. With immutable tables a copy and a reference behave the same, so aliasing cannot be observed. The table above can be checked by changing Table to a mutable hash (`make-hash`, and `hash-set!` in `make-table` and `set-table`).

6.5.8 Challenge: Passing the Store with Monads

Every multi-subexpression case above has the same shape: evaluate, take the result apart with `type-case Result`, pass the new store on, and repeat. That shape can be factored out into two functions, `mret` and `mbind`. With Result fixed to hold a Value:

```
;; A Stateful is a computation that needs a store to run, and gives back a Result.
(define-type-alias Stateful (Store -> Result))

;; A computation that just returns v, without touching the store.
(define (mret (v : Value)) : Stateful
  (lambda (sto) (res v sto)))

;; Run m, then pass its value to f, and run the computation f returns,
;; using the store m returned.
(define (mbind (m : Stateful) (f : (Value -> Stateful))) : Stateful
  (lambda (sto)
    (type-case Result (m sto)
      [res (v sto-1) ((f v) sto-1)])))

;; eval-env isn't written this way, but a call to it can be turned into a Stateful.
(define (meval (env : Env) (e : Expr)) : Stateful
  (lambda (sto) (eval-env env sto e)))
```

A Stateful is a computation that has not run yet: give it a store and it gives back a Result. `mbind` is the only place `sto-1` appears. It runs `m`, hands the value to `f`, and runs whatever `f` returns *in the store* `m` *left behind*. Threading the store correctly is now `mbind`'s job, not each case's.

Exercise 110 (`make-table/m`)

Rewrite `make-table` using `mret`, `mbind`, and `meval`: no `sto`, and no `type-case Result`.

Solution.

```
(define (make-table/m (env : Env) (entries : (listof Entry)) (acc : Table)) : Stateful
  (cond
    [(empty? entries) (mret (tableV acc))]
    [else
     (type-case Entry (first entries)
       [entry (k e)
              (mbind (meval env e)
                     (lambda (v)
                       (make-table/m env (rest entries) (hash-set acc k v))))])]))
```

Put it next to `make-table`: `(res (tableV acc) sto)` became `(mret (tableV acc))`, and the `type-case Result` plus explicit `sto-1` became one `mbind`. The parameter `sto` is gone: `make-table/m` returns a computation, and whoever runs it supplies the store.

Exercise 111 (`set-table/m`)

Rewrite the `settableC` case of `eval-env` the same way, as `(set-table/m env tbl k val)`.

Solution.

```
(define (set-table/m (env : Env) (tbl : Expr) (k : symbol) (val : Expr)) : Stateful
  (mbind (meval env tbl)
         (lambda (tv)
           (let ([t (expect-table tv)])
             (mbind (meval env val)
                    (lambda (v) (mret (tableV (set-table t k v)))))))))
```

The nesting of the `mbind`s *is* the evaluation order: `tbl` is outermost, so it runs first. To evaluate right to left, swap the two `mbind`s; no store variable would need renaming.

To test these we need an environment and a store to run them in. `env-b` and `sto-b` build by hand the state just after `(let b (box 0) ...)`: `b` lives at -1, and that cell holds a box pointing at -2, which holds 0. Negative

locations cannot clash with the ones `new-loc` hands out.

```
(define env-b (extend-env (bind 'b -1) empty-env))
(define sto-b (override-store (cell -1 (boxV -2))
                              (override-store (cell -2 (numV 0)) empty-store)))

;; The second entry sees the first entry's set-box!.
(test (get-table (expect-table
                  (res-v ((make-table/m env-b
                                        (list (entry 'x (parse '(set-box! b 1)))
                                              (entry 'y (parse '(unbox b))))
                                        (hash empty))
                          sto-b)))
                 'y)
      (numV 1))

;; set-table/m evaluates the table before the value, and returns the final store.
(define set-table/m-result
  ((set-table/m env-b
                (parse '(begin (set-box! b 1) (make-table)))
                'a
                (parse '(unbox b)))
   sto-b))
(test (get-table (expect-table (res-v set-table/m-result)) 'a) (numV 1))
(test (fetch -2 (res-s set-table/m-result)) (numV 1))
```

Exercise 112 (Using them in `eval-env`)

Use `make-table/m` and `set-table/m` in the `maketableC` and `settableC` cases of `eval-env`, and check that all the tests still pass.

Solution. A Stateful becomes a Result by applying it to the current store, so the two cases become one line each:

```
    [maketableC (entries) ((make-table/m env entries (hash empty)) sto)]
    [settableC (tbl k val) ((set-table/m env tbl k val) sto)]
```

All the table tests pass. One thing to watch: the tests that call `run` must come after the `define`s of `make-table/m` and `set-table/m`. Module-level code runs top to bottom, so an earlier test fails with "cannot reference an identifier before its definition".

6.5.9 Summary

idea	where it shows up
env: name to location; store: location to value	`lookup` then `fetch`
every binding gets a fresh cell	`appC` calls `new-loc`; `set!` needs it
the store is threaded, so order is a choice	`ex-order`, `ex-f`, the C program
tables are values, boxes are references	`ex-a` to `ex-e`
mutable vs. immutable is about aliasing	`ex-a` and `ex-d` flip with in-place tables
`mret`/`mbind` hide the store-passing	`make-table/m`, `set-table/m`

Chapter 7

Week 7: Recursion and Objects

Lecture 7: October 20, 2026. Recitation 7: October 23, 2026.

Assigned reading and resources (course page):

- [1] Chapter 9 (Recursion and Cycles: Procedures and Data), which builds recursive functions out of a placeholder and mutation, as this chapter does.
- [1] Chapter 10 (Objects), whose objects-by-desugaring sections follow the same path as Section 7.4: messages, constructors, state, static members, `self`, dynamic dispatch, and inheritance through a parent object.
- Lecture code: `lecture07.zip`. According to the course page, its interpreter that encodes recursion with boxes (Section 7.3) is a bit cleaner than the version written in lecture.

Further reading:

- [5] Section 5.2 (Programming in the Lambda-Calculus), whose last part on recursion defines `fix`, the call-by-value fixed-point combinator, which is `Z` of Section 7.5; optional.

Under lexical scope, a function cannot call itself by the name it is being defined with: when the closure is created, that name is not bound yet. This chapter adds a form `rec` for recursive functions and implements it twice. The first version teaches application to rebind the function's own name at every call. The second version uses no new kind of value at all. It builds the cycle that recursion needs out of a box, by first creating a placeholder and then filling it in, a technique called *backpatching*, packaged as a function `fix`. The same ideas then build objects: an object is a closure that dispatches on messages, its fields are mutable variables or boxes, it finds `self` either by backpatching or by taking itself as an argument, and it inherits by delegating to a parent object. The recitation gets recursion with no mutation at all, from the Y and Z combinators. This chapter covers:

- Section 7.1: `if0` and `rec`, why `let` cannot define a recursive function, and a first implementation with a new value `recV`;
- Section 7.2: cyclic data built from Racket boxes, backpatching a recursive function in Racket, and `fix`;
- Section 7.3: `rec` in the store-passing interpreter, desugared into a call to `fix`, or inlined with a fresh name, and why both need a forwarding function;

- Section 7.4: objects as closures over messages, constructors, private, shared, and static state, two ways to get `self`, trees built from objects, and inheritance by delegation;
- Section 7.5: Recitation 7, circular lists and trees with boxes, fixed points, the Y and Z combinators, and `rec` implemented with `Z`.

7.1 The Problem of Recursion

7.1.1 Two New Forms

We start from the closure interpreter of Section 5.4 and add two forms. `(if0 e1 e2 e3)` evaluates `e1`. If the result is 0, it evaluates `e2`; otherwise it evaluates `e3`. This is enough branching to write a base case. `(rec f x e)` is a function with parameter `x` and body `e`, whose body may call the function itself under the name `f`.

```
(define-type Expr
  [numC (n : number)]
  [plusC (e1 : Expr) (e2 : Expr)]
  [timesC (e1 : Expr) (e2 : Expr)]
  [letC (x : symbol) (e1 : Expr) (e2 : Expr)]
  [lambdaC (x : symbol) (e : Expr)]
  [appC (e1 : Expr) (e2 : Expr)]
  [idC (x : symbol)]
  [if0C (guard : Expr) (e1 : Expr) (e2 : Expr)]
  [recC (f : symbol) (x : symbol) (e : Expr)])
```

The parser gets one line for each:

```
           [(rec) (recC (s-exp->symbol (second l)) (s-exp->symbol (third l)) (parse (fourth l)))]
           [(if0) (if0C (parse (second l)) (parse (third l)) (parse (fourth l)))]
```

`if0` only evaluates the branch it chooses, just like `if` in Racket:

```
    [if0C (guard e1 e2)
          (let [(v (eval-env env guard))]
            (if (equal? v (numV 0))
                (eval-env env e1)
                (eval-env env e2)))]
```

`(if0 0 1 2)` is 1 and `(if0 1 1 2)` is 2. The guard's value is compared with `(numV 0)` as a whole, so a guard that is not a number, such as a closure, is simply "not 0" and selects `e3` rather than raising an error. With both forms, factorial is written

```
(rec fact n (if0 n 1 (* n (fact (+ n -1)))))
```

7.1.2 Why `let` Is Not Enough

Without `rec`, the natural attempt is to bind the function with `let`:

```
(let fact (lambda n (if0 n 1 (* n (fact (+ n -1)))))
  (fact 5))
```

Under lexical scope this fails with `No binding found`. The `lambda` is evaluated *before* `let` binds `fact`, so the closure saves an environment without `fact`. When the body later calls `fact`, it looks the name up in that saved environment and does not find it.

Under dynamic scope, the same program works. The body is evaluated in the environment of the *call* `(fact 5)`, and by then `let` has bound `fact`. This is one of the few things dynamic scope makes easier, and it is no

reason to go back to it (Section 5.3). We want recursion and lexical scope together.

7.1.3 A First Solution: `recV`

The function needs an environment that contains the function itself. That is a chicken-and-egg problem. To build the closure we need its environment first, and the environment is supposed to contain the closure.

The first solution avoids the cycle by *waiting*. A new kind of value records the function's own name next to the usual closure fields:

```
(define-type Value
  [numV (n : number)]
  [closV (env : Env) (x : symbol) (e : Expr)]
  [recV (env : Env) (f : symbol) (x : symbol) (e : Expr)])
```

`rec` only packages these fields, without binding `f`:

```
    [recC (f x e) (recV env f x e)]
```

The binding happens at each call. Application now has to tell the kinds of function apart:

```
    [appC (e1 e2)
          (let [(v1 (eval-env env e1))
                (v2 (eval-env env e2))]
            (type-case Value v1
              [closV (env x e)
                     (eval-env (extend-env (bind x v2) env) e)]
              [recV (env f x e)
                    (eval-env (extend-env (bind f v1) (extend-env (bind x v2) env)) e)]
              [numV (n)
                    (error 'eval-env "Error: applied number as a function")]))]
```

In the `recV` case, `v1` *is* the function being called. So binding `f` to `v1` makes the function's own name visible in its body. By the time of the call, `v1` already exists, so there is no cycle to build. Every recursive call goes through `appC` again and adds the binding again, so recursion can go as deep as needed.

For `((rec fact n ...) 2)`, the body runs in

```
[fact = recV(...), n = 2, the environment where rec was evaluated]
```

and the call `(fact (+ n -1))` in the body finds the same `recV`.

The order of the two bindings matters only when `f` and `x` are the same name. Here `f` is added last, so in `(rec g g ...)` the name `g` in the body means the function, not the argument. Binding `x` last instead would let the parameter shadow the function name, which is what Racket does for `(letrec ([g (lambda (g) ...)]) ...)`.

The lecture's examples:

```
((rec fact x (if0 x 1 (* x (fact (+ x -1))))) 5)           ; 120

(let fact (rec f x (if0 x 1 (* x (f (+ x -1)))))
  (fact 6))                                                ; 720

(let fibb (rec f x (if0 x
                        0
                        (if0 (+ x -1) 1
                             (+ (f (+ x -1)) (f (+ x -2))))))
  (fibb 6))                                                ; 8
```

In the last two, the name the body uses for itself, `f`, is not the name the rest of the program uses, `fact` or `fibb`. The `let` only names the finished `recV` for the outside. The recursive calls go through `f`, which `appC` binds at every call.

7.2 Cycles from Boxes

The `recV` solution needs a new kind of value and a special case in application. Racket's own `letrec` works differently: it builds the cycle directly, with mutation. This section shows the idea in plain `#lang racket`.

7.2.1 A Box That Contains Itself

The direct attempt at a value that refers to itself is

```
(let ([b b])
  b)
```

and it fails with `b: unbound identifier`, for the reason of Section 7.1: the right-hand side is evaluated before `b` is bound. A box gets around this:

```
(let ([b (box 'dummy)])
  (begin
    (set-box! b b)
    b))
```

The program first creates a box holding a placeholder, `'dummy`. It then replaces the contents with the box itself. The result is a cycle of length one: unboxing `b` gives `b` again, however many times we unbox. Racket prints it as

```
#0='#&#0#
```

Here `#&` marks a box, `#0=` labels a value as number 0, and `#0#` refers back to that value. So the output reads "a box whose contents are value 0", which is the box itself.

A cycle of length two works the same way:

```
(let* ([b1 (box 'dummy)]
       [b2 (box b1)])
  (set-box! b1 b2)
  b1)
```

`b2` is built holding `b1`, which already exists. Then `b1` is changed to hold `b2`, which closes the cycle. Racket prints `#0='#&#&#0#`, a box holding a box holding the first box. `let*` is needed because the initial value of `b2` refers to `b1`.

Remark 113. These examples only run in `#lang racket`. In `plai-typed`, the body of a `let*` must be a single expression, so the second example is a syntax error without a `begin`. Worse, neither example can be typed: a box that contains itself would need the infinite type "box of box of box of . . . ". The checker reports `(boxof symbol) vs. symbol` for the first one.

7.2.2 Backpatching

Both examples follow the same three steps, and so does every cycle built this way:

1. *Placeholder*: create a box holding a dummy value.

2. *Build*: create the things that refer to the box. This is possible because the box already exists.
3. *Patch*: use `set-box!` to put the real value into the box, which closes the cycle.

This technique is called *backpatching*. It is exactly what a recursive function needs. The function's environment must refer to the function. With backpatching, the environment refers to a box instead, and the box is filled with the function once the function exists.

7.2.3 A Recursive Function in Racket

```
(let* ([fact (box 'dummy)]
       [fact-fun (lambda (n)
                   (if (equal? n 0)
                       1
                       (* n ((unbox fact) (+ n -1)))))])
  (begin
    (set-box! fact fact-fun)
    ((unbox fact) 5)))
```

The three steps are visible. `fact` is the placeholder box. `fact-fun` is a closure whose saved environment contains `fact`. `set-box!` puts the closure into the box. From then on, the body finds the box through `fact`, unboxes it, and gets `fact-fun` itself. The result is 120.

Pitfall 114 (`let` instead of `let*`)

With `let`, the same program is rejected with `fact: unbound identifier`. In a `let`, every right-hand side is evaluated in the outer scope, so the `lambda` cannot see the `fact` bound next to it (Section 5.1). This is the problem of Section 7.1 once more: the function's own name must be in scope when its closure is created.

Note that the body has to write `((unbox fact) (+ n -1))`, not `(fact (+ n -1))`. The programmer has to know that `fact` is a box.

7.2.4 `fix`: Backpatching Once and for All

The three steps do not depend on what the function computes, so they can be written once, as a function `fix`. Its argument `f` is the function to make recursive, written so that it receives "the recursive call" as an extra first

argument `rec`:

```
(define fix
  (lambda (f)
    (let* ([b (box 'dummy)]
           [frec (lambda (x)
                   (f (unbox b) x))])
      (begin
        (set-box! b frec)
        frec))))

(define fact-alt
  (fix (lambda (rec n)
         (if (equal? 0 n)
             1
             (* n (rec (- n 1)))))))
```

`(fact-alt 5)` is 120. The steps are as before, with one change: the box holds `frec`, not `f`. `frec` takes the real argument `x`, fetches itself from the box, and calls `f` with itself as `rec`. So when `f`'s body calls `(rec (- n 1))`, it calls `frec` again, which goes through the box again. The function passed to `fix` never mentions a box: it calls `rec` like any other function. Section 7.3 does the same inside our interpreter, with `fix` written in the interpreted language.

7.3 rec with Boxes

7.3.1 The Starting Point

To use backpatching inside our own language, the language needs boxes. So the lecture starts again from the box interpreter of Section 6.2 and adds `if0C` and `recC` to it. Expr has `boxC`, `unboxC`, `setboxC`, and `seqC`; Value has `boxV` but no `recV`; and `eval-env` takes a store and returns a Result:

```
(define (eval-env (env : Env) (sto : Store) (e : Expr)) : Result
  ...)
```

Every case threads the store: the store returned by one subexpression is passed to the next one. As in Section 6.2, `let` is not a case of its own but is rewritten into an application:

```
   [letC (x e1 e2) (eval-env env sto (appC (lambdaC x e2) e1))]
```

Threading the store through `if0`. `if0` changes in the same way as `plusC`:

```
   [if0C (guard e1 e2)
         (type-case Result (eval-env env sto guard)
           (res (v sto-1)
                (if (equal? v (numV 0))
                    (eval-env env sto-1 e1)
                    (eval-env env sto-1 e2))))]
```

There are three reasons for this shape. First, `eval-env` now returns a Result, so the value `v` has to be taken out before it can be compared with 0. Second, the guard may have side effects, and the chosen branch must see them. So the branch runs in `sto-1`, not `sto`:

```
(let b (box 5)
  (if0 (begin (set-box! b 0) (unbox b))
       (unbox b)    ; must see 0, the value the guard stored
       99))
```

With `sto`, the branch would still see 5. Third, the branch's own Result is returned as it is, so the store after the branch reaches the rest of the program.

7.3.2 Desugaring rec into fix

The Racket `fix` of Section 7.2 can be written in our language too. Our functions take exactly one argument, so the function passed to `fix` takes the recursive call first and returns a function of `x`: it is `(lambda f (lambda x e))` instead of `(lambda (rec n) ...)`. Then

```
(rec f x e)  =  (fix (lambda f (lambda x e)))
```

and fix is an ordinary program in our language, parsed once:

```
(define fix
  (parse
   '(lambda f
      (let b (box 0)
        (let frec (lambda x ((f (unbox b)) x))
          (begin
            (set-box! b frec)
            frec))))))
```

The recC case builds the application and evaluates it:

```
    [recC (f x e)
          (eval-env env sto (appC fix (lambdaC f (lambdaC x e))))]
```

fix is an Expr, not a Racket function: the interpreter splices it into the program as if the programmer had written it. The pieces around it are Expr constructors too, (lambdaC f (lambdaC x e)) and not a closV, because eval-env evaluates them into closures itself. With this version, recC produces an ordinary closV, so the recV value of Section 7.1 is no longer needed. Both examples of Section 7.1 still give 720 and 8.

What fix does. The steps are those of the Racket fix. b is the placeholder box, holding 0 because the language has no symbols. frec is created inside the scope of b. set-box! puts frec into b, and fix returns frec. When the program calls frec with an argument v:

1. (unbox b) fetches frec itself from the store;
2. (f frec) returns the real function (lambda x e), closed over an environment in which f is frec;
3. that function is applied to v, and every call to f in e is a call to frec, which repeats from step 1.

So the real function is rebuilt at every call; only frec exists once. The store after the factorial example shows the backpatch. It holds two cells for location 0, the newer one on top:

```
(list (cell 0 (closV [b = (boxV 0), f = (closV '() 'f (lambdaC 'x ...))]
                     'x
                     ((f (unbox b)) x)))      ; frec, written by set-box!
      (cell 0 (numV 0)))                      ; the placeholder, written by box
```

frec's environment contains (boxV 0), and location 0 contains frec: this is the cycle, running through the store.

No fresh names needed. The names b and frec are bound inside fix's body, and e is not written inside that body. e is part of the argument (lambda f (lambda x e)), which is evaluated in the environment of the rec, before fix binds anything. By lexical scope, fix's names are invisible to e, even when e uses the same names from outside:

```
(let b 100 ((rec g n (if0 n b (g (+ n -1)))) 3))
---> 100
```

The b in the body is the outer b, not fix's box.

7.3.3 Inlining `fix`

Instead of calling `fix`, `recC` can write the backpatching out in place. Writing `z` for a fresh name,

```
(rec f x e)
  =  (let z (box 0)
       (let f (lambda x ((unbox z) x))
         (begin (set-box! z (lambda x e))
                f)))
```

The roles are swapped compared with `fix`. Here `f` is the forwarding function, and the box holds the real function `(lambda x e)`, which is created inside `(let f ...)` so its closure can see `f`. As a `recC` case:

```
    [recC (f x e)
          (let ([z (gensym)])
            (eval-env env sto
                      (letC z (boxC (numC 0))
                            (letC f (lambdaC x (appC (unboxC (idC z)) (idC x)))
                                  (seqC (setboxC (idC z) (lambdaC x e))
                                        (idC f))))))]
```

Now `e` *is* written inside the scope of `z`, so the box's name must be fresh. It comes from `gensym`, imported with `typed-in` as in Section 4.3. A fixed name such as `'z` would shadow any `z` that `e` uses from outside:

```
(let z 100 ((rec g n (if0 n z (g (+ n -1)))) 3))
---> 100        with gensym
---> a box      with the fixed name 'z
```

Key Point 115 (A closed helper needs no fresh names)

A desugaring that puts the user's code inside new binders has to invent fresh names for them. Passing the user's code as an *argument* to a closed function such as `fix` avoids the problem: lexical scope keeps the helper's names and the user's names apart.

7.3.4 Why a Forwarding Function?

In `fix`, `frec` is not the box, and not the result of `(f (unbox b))` either. It is a small function that does the unboxing when it is called. The two simpler choices both fail.

Return the box. If `fix` returned `b` itself, the program would get a `boxV`, and a call such as `(fact 6)` would apply a box, which is an error. The body would have to write `((unbox f) (+ x -1))`, as the Racket version of Section 7.2 does. But the point of `rec` is that the programmer calls `f` like any other function.

Unbox right away. Write `(let frec (f (unbox b)) ...)` instead. Our `let` is eager, so `(unbox b)` runs at once, before `set-box!`. `f` receives the placeholder 0 as its recursive call, and the first recursive call in `e` applies the number 0. Patching the box later changes nothing, since 0 has already been passed.

The forwarding function. `(lambda x ((f (unbox b)) x))` is a function, so the program can call it directly. Creating it does not run `(unbox b)`. The unbox runs only when `frec` is called, and by then the box has been

patched. The inlined version's `(lambda x ((unbox z) x))` works for the same reason.

Key Point 116 (Delaying with a `lambda`)

Replacing an expression `g` by `(lambda x (g x))` is called *η-expansion*. Both behave the same when called, but the second one evaluates `g` only at the call. This is the tool for any value that is not ready now but will be ready by the time it is used. The Z combinator of Section 7.5 uses it to get recursion with no boxes at all.

Remark 117. Two other designs avoid the forwarding function. One substitutes `(unbox z)` for `f` in `e`, which needs the capture-avoiding substitution of Section 4.3. The other uses mutable variables, where the environment maps names to locations (Section 6.5): bind `f` to a placeholder, create the closure, and then `set! f` to it. That is how Racket's `letrec` behaves.

7.4 Objects from Closures

This section asks whether objects need to be built into a language, or whether the features we already have are enough. The answer is that closures and mutable state go a long way. The code in this section is plain `#lang racket`.

7.4.1 An Object Is a Function on Messages

An object answers *messages*. In a language like Java, `o.add(x, y)` sends the message `add` with arguments `x` and `y` to `o`. We can model `o` as a function that takes a message name and returns the method for it:

```
(define o-1
  (lambda (m)
    (case m
      [(add) (lambda (x y) (+ x y))]
      [(add1) (lambda (x) (+ x 1))]
      [(sub1) (lambda (x) (- x 1))])))
```

`((o-1 'add1) 5)` is 6 and `((o-1 'add) 5 6)` is 11. The `case` on the message is the object's *dispatch*: it picks the method that the message names.

Writing two calls each time is clumsy, so a helper sends a message with any number of arguments:

```
(define (msg o m . a)
  (apply (o m) a))
```

The dot in the parameter list makes `a` a *rest parameter*: it collects all remaining arguments into a list. `apply` then calls the method with the elements of that list as its arguments. So `(msg o-1 'add 5 6)` is `((o-1 'add) 5 6)`, which is 11.

Pitfall 118 `((lambda m ...)` versus `(lambda (m) ...))`

Without parentheses, `(lambda m body)` also takes any number of arguments and binds `m` to the *list* of them. In an object written that way, `(o 'first)` binds `m` to `'(first)`, no `case` clause matches, and the call quietly returns nothing.

7.4.2 Constructors and Fields

A *constructor* is a function that returns an object. Its parameters become the object's fields, because the methods are closures that capture them:

```
(define (o-constr-1 x)
  (lambda (m)
    (case m
      ((addX) (lambda (y) (+ x y))))))
```

`(msg (o-constr-1 10) 'addX 5)` is 15. Each call of `o-constr-1` creates a new closure with its own `x`, so each object has its own field.

7.4.3 State

Fields captured this way cannot change. For mutable fields, the constructor puts the value in a box, as in Section 2.4:

```
(define (obj-counter count)
  (let [(b (box count))]
    (lambda (m)
      (case m
        [(inc) (lambda () (set-box! b (+ (unbox b) 1)))]
        [(dec) (lambda () (set-box! b (- (unbox b) 1)))]
        [(get) (lambda () (unbox b))]))))
```

Every call of the constructor creates a new box, so two counters do not affect each other:

```
(define cntr (obj-counter 0))
(define cntr2 (obj-counter 10))
(msg cntr 'inc)
(msg cntr 'inc)
(msg cntr 'dec)
(msg cntr 'get)     ; 1
(msg cntr2 'inc)
(msg cntr2 'get)    ; 11
(msg cntr 'get)     ; 1, unchanged by cntr2
```

The box is also *private*. Only the methods can reach it, since nothing outside the closure has a name for `b`. The only way to change a counter is to send it `inc` or `dec`.

Where the box is created decides who shares it. The next version moves the box out of the object:

```
(define (obj-counter-share count)
  (let ([b (box count)])
    (lambda (count)
      (begin (set-box! b count)
             (lambda (m)
               (case m
                 [(inc) (lambda () (set-box! b (+ (unbox b) 1)))]
                 [(dec) (lambda () (set-box! b (- (unbox b) 1)))]
                 [(get) (lambda () (unbox b))]))))))
```

`obj-counter-share` now returns a *constructor*, and all objects made by that constructor share one box:

```
(define mk (obj-counter-share 0))
(define a (mk 1))
(define b (mk 10))      ; resets the shared box to 10
(msg a 'inc)
(list (msg a 'get) (msg b 'get))   ; '(11 11)
```

Two details matter. Objects share the box only if they come from the *same* `mk`; a second call of `obj-counter-share` creates a second box. And every new object runs `(set-box! b count)`, which overwrites the value seen by all existing objects.

Static fields. The usual way to share state among all objects of a class is to create the shared state once, when the constructor itself is defined. The lecture writes this version with Racket's `set!` instead of boxes:

```
(define o-static-1
  (let ([numobj 0])
    (lambda (amount)
      (begin
        (set! numobj (+ 1 numobj))
        (lambda (m)
          (case m
            [(inc) (lambda (n) (set! amount (+ amount n)))]
            [(dec) (lambda (n) (set! amount (- amount n)))]
            [(get) (lambda () amount)]
            [(count) (lambda () numobj)])))))))
```

The `let` runs once, when `o-static-1` is defined, so `numobj` is shared by every object: it is a *static field*, and here it counts the objects created. The parameter `amount` is bound anew on every constructor call, so it belongs to one object: it is an *instance field*.

```
(let ([o (o-static-1 1000)])
  (msg o 'count))         ; 1
(let ([o (o-static-1 0)])
  (msg o 'count))         ; 2
```

`set!` changes a variable, while `set-box!` changes a box that a variable names. The behavior is the same as with `(box 0)` for `numobj` and `(box amount)` for the counter: a Racket variable that is assigned is in effect a box that the language creates for us. In the terms of Section 6.5, the environment maps the name to a location, and `set!` overrides that location.

Key Point 119 (Lifetime of a binding = how widely it is shared)

State created outside the constructor is shared by all objects; state created inside it, including the constructor's own parameters, belongs to one object. Static versus instance fields is just the question of which `let` or `lambda` the variable is bound by.

7.4.4 `self`

A method often needs to send a message to its own object, as `this.second(x)` does in Java. The object therefore needs a name for itself. This is the problem of Section 7.1 again: the object must refer to the object, but it does not exist until it is built. There are two solutions.

`self` through a box. The first uses the backpatching of Section 7.2:

```
(define o-self!
  (let ([self (box 'dummy)])
    (begin
      (set-box! self
            (lambda (m)
              (case m
                [(first) (lambda (x) (msg (unbox self) 'second (+ x 1)))]
                [(second) (lambda (x) (+ x 1))])))
      (unbox self))))
```

The three steps are the same as before. `self` is a placeholder box. The object is built inside its scope, so the method `first` can reach the box. `set-box!` puts the object into the box. When `first` runs, `(unbox self)` is the object itself. `(msg o-self! 'first 5)` computes `5 + 1`, sends `second`, and returns 7. The name `self` plays exactly the role that `f` played for `rec`.

`self` as an argument. The second solution needs no mutation. Every method takes the object as an extra first argument:

```
(define o-self
  (lambda (m)
    (case m
      [(first) (lambda (self x) (msg/self self 'second (+ x 1)))]
      [(second) (lambda (self x) (+ x 1))])))
```

The sender passes it, through a variant of `msg` that supplies the object twice, once to find the method and once as its first argument:

```
(define (msg/self o m . a)
  (apply (o m) o a))
```

`(msg/self o-self 'first 5)` is 7, as before. This is the explicit `self` parameter of Python methods: `o.f(x)` means `f(o, x)`. Every call between methods must then go through `msg/self`; a plain `msg` would pass one argument too few.

	`o-self!`	`o-self`
how a method finds `self`	through a box in its closure	the caller passes it
needs mutation	yes (backpatching)	no
send with	`msg`	`msg/self`

Messages are values. Since a message is just a symbol, the program can compute which message to send:

```
(define test
  (lambda (x)
    (msg o-self! (if (equal? x 1) 'first 'second) 5)))
```

`(test 1)` sends `first` and gives 7; `(test 2)` sends `second` and gives 6. The method that runs is chosen at run time from a value. An ordinary method call such as `o.first(5)` fixes the method name in the program

text; with closures, the name is just data.

7.4.5 Objects with Structure: Trees

Objects can refer to other objects. A binary tree is a node object whose fields are a value and two subtrees, and an empty-tree object `mt` with no fields:

```
(define (mt)
  (lambda (m)
    (case m
      [(add) (lambda () 0)])))

(define (node v l r)
  (lambda (m)
    (case m
      [(add) (lambda () (+ v
                           (msg l 'add)
                           (msg r 'add)))])))
```

`add` sums the values in the tree. A node answers it by sending `add` to its two subtrees and combining the answers; the empty tree answers 0. `mt` is a constructor with no arguments, so an empty tree is written `(mt)`.

```
(define a-tree
  (node 10
        (node 5 (mt) (mt))
        (node 15 (node 6 (mt) (mt)) (mt))))
```

`(msg a-tree 'add)` is $10 + 5 + 15 + 6 = 36$.

The node never asks whether a subtree is a node or an empty tree. It sends the message, and each subtree runs its own method. Which code runs is decided by the object that receives the message, not by the sender. This is *dynamic dispatch*.

7.4.6 Inheritance by Delegation

Now add a `size` message, which counts the nodes, without editing `node` and `mt`. A new constructor takes the old one as an argument, the *parent*, and builds a parent object with the same fields. It answers `size` itself and

passes every other message on to the parent object:

```
(define (node/size parent-maker v l r)
  (let ([parent-object (parent-maker v l r)])
    (lambda (m)
          (case m
            [(size) (lambda () (+ 1
                                  (msg l 'size)
                                  (msg r 'size)))]
            [else (parent-object m)]))))

(define (mt/size parent-maker)
  (let ([parent-object (parent-maker)])
    (lambda (m)
      (case m
        [(size) (lambda () 0)]
        [else (parent-object m)]))))
```

The `else` clause is the whole mechanism: a message the child does not know is *delegated* to the parent, so the child inherits `add`. Every node of the tree has to be built with the new constructors, with `node` and `mt` passed as the parents:

```
(define a-tree/size
  (node/size node
             10
             (node/size node 5 (mt/size mt) (mt/size mt))
             (node/size node 15
                        (node/size node 6 (mt/size mt) (mt/size mt))
                        (mt/size mt))))
```

`(msg a-tree/size 'size)` is 4, and `(msg a-tree/size 'add)` is still 36. For `add`, the outer `node/size` object delegates to its parent `node` object. That object sends `add` to `l` and `r`, which are `node/size` objects, and they delegate in turn. All subtrees must be `/size` objects: if one were a plain `node`, `size` would reach it and match no `case` clause. The dispatch would return `#<void>` instead of a method, and `msg` would fail with `application: not a procedure`.

Remark 120. The parent object is built from `v`, `l`, and `r` alone; it never learns that a child exists. So delegation passes on the message but not the object. If a parent method sent a message to itself through a `self` of its own, as `o-self!` does, the message would reach the parent, and a method that the child overrides would not be used. Inheritance in the usual sense makes `self` mean the child, for example by passing the child as `self` in the style of `o-self`.

7.5 Recitation 7: Recursion

This recitation asks two closely related questions: how to build a *circular* data structure, and how to let something refer to *itself* in its own definition. A plain `let` answers neither, for the reason of Section 7.1: the right-hand side is evaluated before the name is bound. The recitation shows two ways around this. The first ties the knot with boxes, as in Section 7.2. The second uses *fixed-point combinators*, which give recursion with no mutation at all. A challenge at the end uses one of them to implement `rec` in the interpreter. The code is plain `#lang racket`, with tests written with `check-expect` from `test-engine/racket-tests`, except for the challenge, which is `plai-typed`.

7.5.1 Tying the Knot with Boxes

`self-box` is a box that contains itself, and `two-cycle` is two boxes that point to each other, `b1` → `b2` → `b1`:

```
(define self-box
  (let ([b (box 'dummy)])
    (begin
      (set-box! b b)
      b)))
(define two-cycle
  (let* ([b1 (box 'dummy)]
         [b2 (box b1)])
    (begin
      (set-box! b1 b2)
      b1)))

(check-expect (eq? (unbox self-box) self-box) #t)
(check-expect (eq? (unbox (unbox two-cycle)) two-cycle) #t)
(check-expect (eq? (unbox two-cycle) two-cycle) #f)
```

The tests use `eq?`, which asks whether two values are the *same* object in memory. Unboxing `self-box` once gets back to it; `two-cycle` needs two steps, and one step gets to the other box. Racket prints the two values as `#0='#&#0#` and `#0='#&#&#0#` (Section 7.2).

Exercise 121 (Recursive functions with boxes)

Implement factorial as `fact-box` by putting the function in a box, without letting `fact-box` call itself by name.

Solution. This is the factorial of Section 7.2, with the call `((unbox fact) n)` at the end:

```
(define (fact-box n)
  (let* ([fact (box 'dummy)]
         [fact-fun (lambda (k)
                     (if (equal? k 0)
                         1
                         (* k ((unbox fact) (- k 1)))))])
    (begin
      (set-box! fact fact-fun)
      ((unbox fact) n))))

(check-expect (fact-box 0) 1)
(check-expect (fact-box 5) 120)
```

Pitfall 122 (`((unbox fact)` is the function, not the call)

`(* k (unbox fact) (- k 1))` is also syntactically fine, but it multiplies three things: `k`, the function itself, and `k - 1`. It fails with `*: contract violation, expected: number?, given: #<procedure:fact-fun>`. The test `(fact-box 0)` still passes, because the base case never reaches the multiplication, so only `(fact-box 5)` shows the bug. The call needs its own parentheses: `((unbox fact) (- k 1))`.

A loop with no recursive definition. What does this program do?

```
(let ([nonterminating (box 'dummy)])
  (begin
    (set-box! nonterminating (lambda (x) ((unbox nonterminating) x)))
    ((unbox nonterminating) 5)))
```

It runs forever, although nothing in it is defined in terms of itself. The loop is in the store, not in the program text. The `lambda` reaches the box through its environment, and after `set-box!` the box holds that same `lambda`. So each call unboxes the function and calls it again. `(define (loop x) (loop x))` loops for the same reason: a recursive `define` also ties a knot, and Racket ties it for us (Section 7.3). The call is in tail position, so Racket does not grow the stack; memory use stays flat and the program simply never stops.

Exercise 123 (Circular linked lists)

Write `circ-list`, which takes $n \geq 1$ and returns a circular list of n boxes. Each box holds a pair `(cons value next)`. The head holds n, the next box $n-1$, and so on down to 1, and the box holding 1 points back to the head:

$$\text{head } (n) \to (n-1) \to \cdots \to (1) \to \text{head}$$

For $n < 1$, signal `(error 'circ-list "n must be at least 1")`. Use a recursive helper stored in a box, which takes a number `k` and a box `b`, fills `b` with `k` and the next box, and builds the rest of the list.

Solution. Two knots are tied here. The head is allocated first as a placeholder, so the last box can point to it.

And the helper `build` is recursive, so it lives in a box too:

```
(define (circ-list n)
  (if (< n 1)
      (error 'circ-list "n must be at least 1")
      (let* ([head (box 'dummy)]
             [build (box 'dummy)]
             [build-fun
              (lambda (k b)
                (if (equal? k 1)
                    (set-box! b (cons 1 head))
                    (let ([next-box (box 'dummy)])
                      (begin
                        (set-box! b (cons k next-box))
                        ((unbox build) (- k 1) next-box)))))])
        (begin
          (set-box! build build-fun)
          ((unbox build) n head)
          head))))
```

`build` fills the box it is given, which is the head on the first call, and allocates the next box only when $k > 1$. The box for 1 is filled with `head` instead of a new box, which closes the cycle. With the helpers

```
(define (node-value b) (car (unbox b)))
(define (node-next b) (cdr (unbox b)))
```

the expected results are

```
(define circ-1 (circ-list 1))
(define circ-3 (circ-list 3))
(check-expect (node-value circ-3) 3)
(check-expect (node-value (node-next circ-3)) 2)
(check-expect (eq? (node-next circ-1) circ-1) #t)
(check-expect (eq? (node-next circ-3) circ-3) #f)
(check-expect (eq? (node-next (node-next (node-next circ-3))) circ-3) #t)
(check-error (circ-list 0) "circ-list: n must be at least 1")
```

`circ-1` is a one-box cycle, like `self-box`, so its next box is itself. Racket prints `(circ-list 3)` as `#0='#&(3 . #&(2 . #&(1 . #0#)))`.

Exercise 124 (Recursive trees)

A tree is `(leaf` v`)` or `(node` t_1 t_2`)`, written as an s-expression `'(leaf v)` or `'(node left right)`. Write `tree-box`, which turns a nested list into a tree: a number becomes a leaf, `(list l r)` becomes a node, and anything else is an error `"tree: not a tree"`. Tie the knot with a box; `tree-box` must not call itself by name.

input	output
`5`	`'(leaf 5)`
`(list 5 (list 6 7))`	`'(node (leaf 5) (node (leaf 6) (leaf 7)))`
`(list 1 2 3)`	error

Solution. The pattern `(? number? n)` matches any number and binds it to `n`. A node needs two recursive calls, one for each subtree:

```
(define (tree-box input)
  (let* ([tree (box 'dummy)]
         [tree-fun
          (lambda (t)
            (match t
              [(? number? n) (list 'leaf n)]
              [(list left right) (list 'node ((unbox tree) left) ((unbox tree) right))]
              [_ (error 'tree "not a tree")]))])
    (begin
      (set-box! tree tree-fun)
      ((unbox tree) input))))

(check-expect (tree-box 5) '(leaf 5))
(check-expect (tree-box (list 5 (list 6 7)))
              '(node (leaf 5) (node (leaf 6) (leaf 7))))
(check-error (tree-box (list 1 2 3)) "tree: not a tree")
(check-error (tree-box (list 5 (list 6))) "tree: not a tree")
```

`(list 5 (list 6))` is an error because the inner list has one element, which matches neither a number nor a two-element list. The pitfall above applies here as well: `(list 'node ((unbox tree) left) (unbox tree) right)` builds a list of four elements, with the function itself as the third one.

7.5.2 Recursion without Mutation

Can we get recursion without boxes? The idea is that a function does not refer to itself. Instead, it takes "the recursive call" as an argument, as the function passed to `fix` in Section 7.2 does, but one argument at a time:

```
(define fact-gen
  (lambda (rec)
    (lambda (n)
      (if (equal? 0 n)
          1
          (* n (rec (- n 1)))))))
```

`fact-gen` is not factorial. Given a function `rec` that computes factorial, it returns a function that computes factorial too, by doing one step and leaving the rest to `rec`. What we need is a function that is not changed by `fact-gen`.

Definition 125 (Fixed point)

A *fixed point* of a function f is a value v with $f(v) = v$. A *fixed-point combinator* is a function Y such that, for every f,

$$Y\ f = f\ (Y\ f),$$

that is, $Y\ f$ is a fixed point of f.

If such a Y exists, `(Y fact-gen)` is factorial, because unfolding the equation once gives exactly the factorial step:

```
Y fact-gen
=  fact-gen (Y fact-gen)
=  (lambda (n) (if (equal? 0 n) 1 (* n ((Y fact-gen) (- n 1)))))
```

Here = means the two sides are equal, but not by a single evaluation step; => below is one step.

The Y combinator. One such Y is

```
(define Y
  (lambda (h)
    ((lambda (x) (h (x x)))
     (lambda (x) (h (x x))))))
```

The trick is *self-application*, `(x x)`. The same function is written twice, and the first copy is applied to the second. So inside its body, `x` is that function itself, and `(x x)` rebuilds the whole application. `Y` satisfies the equation:

```
Y f
=> (lambda (x) (f (x x))) (lambda (x) (f (x x)))
=> f ((lambda (x) (f (x x))) (lambda (x) (f (x x))))
=  f (Y f)
```

The last step is =, not =>: the argument of `f` is what `(Y f)` stepped to on the first line, but nothing steps back to `(Y f)`.

Why `Y` loops in Racket. `(define fact-y (Y fact-gen))` never finishes. Write W for `(lambda (x) (fact-gen (x x)))`. Then

```
(Y fact-gen)
=> (W W)
=> (fact-gen (W W))
=> (fact-gen (fact-gen (W W)))
=> ...
```

Racket evaluates arguments eagerly, so before it can call `fact-gen`, it must evaluate the argument `(W W)`. That is the expression it started from, one step later. `fact-gen` is never called, although it would not even use `rec` right away: it would return a `lambda` at once.

In a lazy language, `Y` works. The argument `(W W)` is passed unevaluated, as in the lazy substitution of Section 4.2. `fact-gen` returns its `(lambda (n) ...)`, and `(W W)` is evaluated only when the body calls `rec`, which unfolds one more level. A base case that does not call `rec` stops the unfolding.

Exercise 126 (The Z combinator)

Write `Z`, the version of `Y` for eager evaluation. For every `f` and value `v` it satisfies

$$Z\ f\ v = f\ (Z\ f)\ v.$$

Hint: start from `Y`, and delay the self-application by wrapping it in a `lambda`: `(x x)` becomes `(lambda (v) ((x x) v))`.

Solution. This is the η-expansion of Section 7.3, applied to both copies of `(x x)`:

```
(define Z
  (lambda (h)
    ((lambda (x) (h (lambda (v) ((x x) v))))
     (lambda (x) (h (lambda (v) ((x x) v)))))))

(define fact-z
  (lambda (n) ((Z fact-gen) n)))

(check-expect (fact-z 0) 1)
(check-expect (fact-z 5) 120)
```

Write W' for `(lambda (x) (fact-gen (lambda (v) ((x x) v))))`. Now

```
(Z fact-gen)
=> (W' W')
=> (fact-gen (lambda (v) ((W' W') v)))
=> (lambda (n) (if (equal? 0 n) 1 (* n (rec (- n 1)))))
       where rec = (lambda (v) ((W' W') v))
```

The argument of `fact-gen` is a `lambda`, which is already a value, so `fact-gen` is called, and the result is a function. Evaluation stops there. `(W' W')` is evaluated only when `rec` is called, and then it produces the same function again, one level down:

```
((Z fact-gen) 3)
=> (* 3 (rec 2))
=> (* 3 ((W' W') 2))
=> (* 3 (* 2 (rec 1)))
=> (* 3 (* 2 (* 1 (rec 0))))
=> (* 3 (* 2 (* 1 1)))
=> 6
```

Each line skips the steps that unfold `(W' W')` into the factorial function again.

Exercise 127 (Putting it together)

Rewrite `tree-box` with `Z` instead of a box. First write `tree-gen`, which takes the recursive call as an argument, like `fact-gen`.

Solution. The body of `tree-fun` is unchanged, except that the recursive call is the argument `rec` instead of `(unbox tree)`:

```
(define tree-gen
  (lambda (rec)
    (lambda (t)
      (match t
        [(? number? n) (list 'leaf n)]
        [(list left right) (list 'node (rec left) (rec right))]
        [_ (error 'tree "not a tree")]))))
(define tree-z
  (lambda (t) ((Z tree-gen) t)))
```

The four tests of `tree-box` pass for `tree-z` as well.

`Z` versus `fix`. Both turn a function like `fact-gen` into a recursive function. `fix` ties the knot with a box: there is one `frec`, and it finds itself in the store. `Z` uses nothing but functions: the function never finds "itself", it rebuilds a copy with `(x x)` at every recursive call. (The Racket `fix` of Section 7.2 takes `rec` and `n` together, `(lambda (rec n) ...)`; `Z` takes them one at a time, like the `fix` of Section 7.3.)

7.5.3 Challenge: `rec` via `Z`

The file `recitation7-interp.rkt` contains the interpreter of Section 7.1 with `if0C` and `recC`, but with no `recV` and no boxes: Value has only `numV` and `closV`. The task is to implement `rec` by rewriting it into a call to `Z`, as Section 7.3 rewrote it into a call to `fix`. A helper `run` parses and evaluates:

```
(define (run (s : s-expression)) : Value
  (eval (parse s)))
```

Exercise 128 (Z in our language)

Write `Z` as a program in our language. `lambda` takes exactly one argument, written `(lambda x body)`, and application is `(e1 e2)`.

Solution. The Racket `Z` already uses one-argument functions only, so only the syntax changes:

```
(define Z
  (parse '(lambda h
            ((lambda x (h (lambda v ((x x) v))))
             (lambda x (h (lambda v ((x x) v))))))))

(define fact-gen
  (parse '(lambda self
            (lambda n
              (if0 n 1 (* n (self (+ n -1))))))))

(test (eval (appC (appC Z fact-gen) (numC 0))) (numV 1))
(test (eval (appC (appC Z fact-gen) (numC 5))) (numV 120))
```

In `fact-gen`, which the handout provides, the parameter is called `self` and not `rec`, because `rec` is a keyword of our language: `(rec (+ n -1))` would parse as a `rec` expression.

Exercise 129 (`rec`)

Fill in the `recC` case of `eval-env`.

Solution. Exactly as with `fix`, `(rec f x e)` becomes `(Z (lambda f (lambda x e)))`:

```
    [recC (f x e) (eval-env env (appC Z (lambdaC f (lambdaC x e))))]
```

`(lambda f (lambda x e))` plays the role of `fact-gen`. All four tests pass:

```
(test (run '((rec fact x (if0 x 1 (* x (fact (+ x -1))))) 5)) (numV 120))
(test (run '(let fact (rec f x (if0 x 1 (* x (f (+ x -1)))))
              (fact 6)))
      (numV 720))
(test (run '(let fibb (rec f x (if0 x
                                   0
                                   (if0 (+ x -1)
                                        1
                                        (+ (f (+ x -1)) (f (+ x -2))))))
              (fibb 6)))
      (numV 8))
(test (run '(let k 10
              (let f (rec g x (if0 x k (g (+ x -1))))
                (f 3))))
      (numV 10))
```

The last test checks that the body of a `rec` can still use variables from outside it. It passes for the reason of Section 7.3: `Z` is a closed program, and the user's `e` is passed to it as an argument, so `Z`'s names `h`, `x`, and `v` never capture anything in `e`. That holds even though the tests use `x` as their own parameter name.

Exercise 130 (Y instead of Z)

Replace `Z` by `Y` in the `recC` case and run the file again. What happens, and why?

Solution. The `Z` tests still pass, but the first `rec` test never finishes, and memory use grows steadily until DrRacket is stopped. This is the loop of the Racket `Y`, one level down. Our language is eager too: `appC` evaluates the argument `e2` to a value before it runs the body. So `(h (x x))` evaluates `(x x)`, which evaluates `(h (x x))` again, without ever calling `h`. Unlike the Racket loop above, this one also grows the Racket stack: each `appC` evaluates its argument with a nested call to `eval-env`, which is not a tail call.

Exercise 131 (Three ways to implement `rec`)

For each of `recV`, `fix`, and `Z`: does it need a new kind of Value? Does it need boxes (a store)?

Solution.

	`recV` (Section 7.1)	`fix` (Section 7.3)	`Z` (this section)
new kind of Value	yes, `recV`	no	no
boxes / a store	no	yes	no
interpreter changes	`appC` gets a case	only `recC`	only `recC`
where the self-reference is	environment, rebound by each call	a cycle through the store	rebuilt by `(x x)` at each call

`recV` changes the interpreter the most, but it is the most direct: the function's own name is bound to the function, at each call. `fix` and `Z` change nothing but `recC`, because both turn `rec` into a program the interpreter can already run. `fix` needs the language to have boxes. `Z` needs only `lambda` and application, which is why recursion needs no special support in an untyped language with first-class functions.

Remark 132. `Z` relies on `(x x)`, and this self-application cannot be typed directly in a language like `plai-typed`: `x` would need a type of functions whose argument has that same type. This is the same obstacle as the box that contains itself in Section 7.2. (A recursive `define-type` that wraps `x` gets around it.) Typed languages therefore usually build recursion in, as `define` and `letrec` are built into Racket.

Chapter 8

Week 8: Macros

Lecture 8: October 27, 2026. Recitation 8: October 30, 2026.

Assigned reading and resources (course page):

- [1] Chapter 13 (Desugaring as a Language Feature).
- [4] Chapter 16 (Macros), and the further resources it links to.
- [10], optional: the history of how hygienic macro systems were developed (Section 8.6).
- Lecture code: `lecture08.rkt`.

Further reading:

- [11], a tutorial on `syntax-case`, `with-syntax`, and `datum->syntax`, recommended by the Racket Guide; it covers deliberately unhygienic macros like `aif` (Section 8.10).
- [1] Chapter 4 (A First Taste of Desugaring), the desugaring inside an interpreter that macros hand to the programmer.
- [5] Section 5.2 (Programming in the Lambda-Calculus): Church booleans and numerals, and recursion, as in Recitation 8 (Section 8.11); optional.

Some constructs cannot be functions. A function call evaluates every argument first, once, so a function cannot skip an argument (`or`, `if`), repeat one (`while`), look at an argument's text (`assert`), or assign to the caller's variables (`swap`). A *macro* can: it is a function from syntax to syntax that Racket runs before evaluation, rewriting each use into other code. This is the desugaring our interpreters have done all along (`let` into `lambda`, `rec` into boxes), now available to the programmer. Hygiene can also be broken on purpose, one name at a time. The recitation turns away from macros to the lambda calculus, a language with nothing but functions, and encodes booleans, numbers, and recursion in it. This chapter covers:

- Section 8.1: why `or` cannot be a function under call-by-value;
- Section 8.2: `define-syntax`, `syntax-case`, syntax objects and `#'` templates, with `my-or`, `my-if`, `define-syntax-rule`, and `unless`;

- Section 8.3: a recursive macro that never stops expanding, and a `while` loop that expands into a recursive function instead;
- Section 8.4: a macro that uses its argument both as code to run and as data to print;
- Section 8.5: ellipsis patterns, recursion that terminates by shrinking the arguments, and binding an argument once to avoid evaluating it twice;
- Section 8.6: variable capture, hygienic expansion, and hygiene as lexical scope for macros;
- Section 8.7: keywords in patterns with the literal list;
- Section 8.8: `let`, `let*`, and multi-binding `let` as macros into `lambda`;
- Section 8.9: `swap` as a function, with boxes, and as a macro that mutates the caller's variables;
- Section 8.10: an anaphoric `aif` that deliberately captures `it` with `datum->syntax` (not covered in lecture);
- Section 8.11: Recitation 8, the lambda calculus: substitution and alpha-equivalence, normal-order evaluation, Church booleans and numerals, and `Y` under normal order.

8.1 Why or Is Not a Function

Racket's or stops at the first argument that is true. In

```
(or #t (print "Hello World!"))
```

the first argument is already #t, so the result is #t and the print never runs: nothing is printed.

The obvious attempt to write our own or is a function:

```
(define (my-or1 a b)
  (if a #t b))
```

Its body looks right: if a is true, answer #t without looking at b. But the call prints:

```
> (my-or1 #t (print "hello world"))
"hello world"#t
```

print writes the string with its quotes and no newline, and the REPL then prints the result #t right after it. Writing the body out by hand, with the argument expression in place of b, does not print:

```
> (if #t #t (print "hello world"))
#t
```

So the function and its inlined body behave differently. Racket evaluates every argument of a function call before the body runs (eager, call-by-value evaluation, as in our interpreters since Section 5.4). By the time the if inside my-or1 looks at a, the side effect in b has already happened. The body cannot undo that.

Key Point 133

A function receives values, not expressions. or has to decide *whether* to evaluate its second argument, so it cannot be a function under call-by-value. It has to be a rewrite of the program text, done before evaluation: (or a b) becomes something like (if a #t b), where b sits in a branch of the if and is evaluated only if that branch is taken.

The extra evaluation shows up through effects: printing, mutation, errors, and non-termination. For example, (my-or1 #t (car '())) raises an error where (or #t (car '())) returns #t.

Remark 134. my-or1 also differs from or in the value it returns: Racket's or returns the first true value itself, so (or 5 #f) is 5, while (my-or1 5 #f) is #t.

8.2 A First Macro: `my-or`

Section 8.1 showed that `or` must rewrite the program before it runs. Racket lets us write such a rewrite ourselves, as a *macro*:

```
(define-syntax (my-or stx)
  (syntax-case stx ()
    [(my-or a b)
     #'(if a #t b)]))
```

Piece by piece:

- `define-syntax` binds `my-or` as a macro instead of a function. Whenever the expander meets a form `(my-or . . . )`, it calls the macro's *transformer* on that form, before the program runs.
- The transformer is a function. Its argument `stx` is a *syntax object*: the program text of the whole use, such as `(my-or #t (print "hello world"))`, together with information like source location and scope. It returns another syntax object, which replaces the use.
- `syntax-case` takes `stx` apart by pattern matching. The `()` is the list of literals, keywords that must match exactly; there are none here. The pattern `(my-or a b)` matches a use with exactly two subforms and binds the *pattern variables* `a` and `b` to them, unevaluated. The first position, `my-or`, is a pattern variable too: it matches the macro's name as written at the use, and is usually ignored (it is often written `_`).
- `#'(if a #t b)` is short for `(syntax (if a #t b))`: a *template*. It builds a syntax object in which every pattern variable is replaced by the piece of syntax it matched. `#'` is to syntax objects what `'` is to lists.

So the use

```
(my-or #t (print "hello world"))
```

is rewritten into

```
(if #t #t (print "hello world"))
```

which is exactly the expression from Section 8.1 that returns `#t` without printing. The argument `(print "hello world")` is moved into a branch of the `if` as program text, never as a value, so it is evaluated only if that branch is taken.

Key Point 135

A function maps values to a value at run time. A macro maps syntax to syntax at expansion time, before evaluation starts. This is the same desugaring we did inside the interpreter, `let` into an application of a `lambda` (Section 5.1) and `rec` into boxes (Section 7.3), now done by the language for the programmer.

8.2.1 my-if

The same recipe gives a conditional of our own, written in terms of cond:

```
(define-syntax (my-if stx)
  (syntax-case stx ()
    [(my-if a b c)
     #'(cond [a b] [else c])]))
```

(my-if e1 e2 e3) expands to (cond [e1 e2] [else e3]), so only one of e2 and e3 is evaluated. As a function, (define (my-if a b c) (cond [a b] [else c])) would evaluate both branches before choosing one. Every effect in the branch not taken would happen anyway, and a recursive function whose base case is guarded by such an if, like

```
(define (fact n)
  (my-if (= n 0) 1 (* n (fact (- n 1)))))
```

would never terminate: the recursive call in the third argument is evaluated even when n is 0. With the macro, fact works, because the recursive call ends up in the else branch of the cond.

Remark 136. In the template, else is not a pattern variable: it is not in the pattern (my-if a b c), so it is copied into the output as is, and cond sees it as its own else keyword. Only a, b, and c are replaced.

8.2.2 define-syntax-rule

When a macro has a single pattern, define-syntax-rule says the same thing more briefly:

```
(define-syntax-rule (my-or2 a b)
  (if a #t b))
```

The first part is the pattern and the second is the template. There is no stx, no syntax-case, and no #': the template is implicitly a syntax template. my-or2 behaves exactly like my-or. The longer syntax-case form is needed when a macro has several patterns, or when its transformer must run code of its own before producing the output.

8.2.3 unless

(unless b e) evaluates e only when b is false. It is if with the branches swapped and nothing in the true branch:

```
(define-syntax-rule (unless b e)
  (if b (void) e))
```

(void) returns Racket's "no useful value" value, which the REPL does not print. As with or and if, unless must be a macro: as a function, e would be evaluated before the test, so (unless #t (print "hi")) would print.

Remark 137. Racket already has an unless; a definition at the top of a module shadows the imported one. Racket's version also allows several body expressions, (unless b e1 e2 . . .), which needs an ellipsis pattern, (unless b e ...).

8.3 A Loop: my-while

(my-while test body) should evaluate body again and again as long as test is true. Like or, it cannot be a function: test and body must be evaluated many times, and a function receives each argument once, already evaluated.

8.3.1 A Macro That Never Stops Expanding

The first attempt mirrors the definition of a while loop: if the test holds, run the body and then the whole loop again.

```
(define-syntax-rule (my-while test body)
  (cond [test (begin body (my-while test body))] [else (void)]))
```

The definition itself is accepted. The problem appears at the first use. The expander replaces (my-while test body) by the template, and the template contains (my-while test body) again, which the expander must also replace, and so on. Expansion happens before the program runs, so test has no value yet and nothing can stop the unfolding: expansion never terminates, and the program never starts.

Key Point 138

A macro that uses itself in its template must make the recursive use *smaller* (fewer arguments, a shorter list), so that expansion reaches a base case. Repetition that depends on run-time values, such as a loop condition, cannot be done by expansion. It has to be done by run-time code that the macro generates.

8.3.2 Generating a Recursive Function

The fix moves the recursion from expansion time to run time. The macro expands, once, into a recursive function and a call to it:

```
(define-syntax-rule (my-while test body)
  (begin (define (loop)
           (if test (begin body (loop)) (void)))
         (loop)))
```

Here (loop) inside the template is an ordinary function call, not a macro use, so the expander leaves it alone. At run time, each call of loop evaluates test afresh; if it is true, it runs body and calls loop again, and if it is false, it returns (void). The test and body expressions are pasted into the body of loop, which is exactly what makes them run once per iteration.

Using it:

```
(define b (box 0))

(my-while (< (unbox b) 3)
          (begin (print "Hello!")
                 (set-box! b (+ 1 (unbox b)))))
```

This prints "Hello!" three times, and b ends up holding 3.

Remark 139. The template defines a function named `loop`, and the user's `test` or `body` might mention a `loop` of their own. By hygiene (Section 8.6), the `loop` written in the template and a `loop` written by the user are different variables, so the macro's helper cannot capture the user's name.

8.4 Macros See the Code: `my-assert`

The macros so far needed control over *when* an argument is evaluated. A macro can also use the argument's *text*:

```
(define-syntax-rule (my-assert e)
  (unless e (printf "assertion failed: ~s\n" 'e)))
```

In the template, `e` appears twice. The first `e` is evaluated as the condition. The second is inside a quote: `'e` is `(quote e)`, and since `e` is a pattern variable it is replaced by the user's expression first, so the quote produces that expression as data. An assertion that holds, such as

```
(my-assert (= (+ 1 1) 2))
```

prints nothing: the condition is true, so `unless` returns `(void)`. A failing one,

```
(my-assert (= (+ 1 1) 3))
```

expands to

```
(unless (= (+ 1 1) 3) (printf "assertion failed: ~s\n" '(= (+ 1 1) 3)))
```

and prints `assertion failed: (= (+ 1 1) 3)`: the expression as written, not its value `#f` and not a simplified form. The `~s` directive of `printf` prints a value the way `write` does.

Key Point 140

A function `(define (my-assert v) . . . )` would receive only `#f` and could not report which assertion failed. A macro receives the expression itself, so it can both evaluate it and print it.

Remark 141. In this file, `unless` is the macro defined earlier, which takes exactly one body expression. The template uses it with one, so it works with either version.

8.5 Any Number of Arguments: multi-or

Racket's or takes any number of arguments. A macro accepts that with an *ellipsis* pattern, and handles it by recursion on the number of arguments:

```
(define-syntax (multi-or stx)
  (syntax-case stx ()
    [(multi-or e0 e1 ...)
     #'(let ([b e0])
         (if b b (multi-or e1 ...)))]
    [(multi-or)
     #'#f]))
```

8.5.1 Ellipses

In a pattern, e1 ... matches zero or more subforms, and binds e1 to the whole sequence. So (multi-or e0 e1 ...) matches any use with at least one argument: e0 is the first, e1 the rest. In a template, e1 ... puts that sequence back, in order. The second clause, (multi-or), matches the use with no arguments, and its template #'#f is the answer of an empty or. syntax-case tries the clauses in order, and the two patterns do not overlap.

8.5.2 Recursion That Terminates

The template uses multi-or again, as my-while did in Section 8.3, but here the recursive use has one argument fewer. Every expansion step removes one argument, so expansion reaches (multi-or) and stops. For example:

```
(multi-or #f #f 3)
---> (let ([b #f]) (if b b (multi-or #f 3)))
---> (let ([b #f]) (if b b (let ([b #f]) (if b b (multi-or 3)))))
---> ... (let ([b 3]) (if b b (multi-or))) ...
---> ... (let ([b 3]) (if b b #f)) ...
```

The result is 3. Like Racket's or, and unlike my-or of Section 8.2, multi-or returns the first true value itself, not #t.

8.5.3 Why the let

The lecture code also defines the version without the let, and calls it multi-or-bad:

```
(define-syntax (multi-or-bad stx)
  (syntax-case stx ()
    [(multi-or-bad e0 e1 ...)
     #'(if e0 e0 (multi-or-bad e1 ...))]
    [(multi-or-bad)
     #'#f]))
```

Its template copies e0 twice, so e0 is evaluated twice when it is true. With effects that is visible: (multi-or-bad (begin (print "hi") 1)) prints "hi" twice, once for the test and once for the result. multi-or binds e0 once to b and then tests and returns b, so it evaluates e0 exactly once.

The same issue arises in any macro that uses an argument twice. A `square` macro written as `(* x x)` would evaluate its argument twice, so it binds it first:

```
(define-syntax-rule (square x)
  (let ([v x])
    (* v v)))
```

Key Point 142

A pattern variable in a template stands for a piece of code, not a value. Using it twice in the template copies the code twice, and the code then runs twice. When a macro needs an argument's value more than once, it should bind the argument to a variable once.

8.5.4 A Name Introduced by the Macro

The `let` introduces a name, `b`, that the user never wrote. What happens when the user's own arguments mention a `b` is the subject of Section 8.6.

8.6 Hygiene

8.6.1 The Problem: Capture

In lecture, `multi-or` of Section 8.5 was used under a binding of the user's own named `b`:

```
(let ([b #t])
  (multi-or #f b))
```

If expansion just pasted text, as the C preprocessor does (Section 5.5), the result would be the expansion written out in a comment in the lecture code:

```
(let ([b #t])
  (let ([b #f])                ; b written in the template
    (if b b (multi-or b))))    ; b written by the user
```

The user's `b`, which referred to the outer `#t`, now sits inside the macro's `(let ([b #f]) . . . )` and refers to `#f`. The program answers `#f` although one argument of the `or` is true. The user's variable has been *captured* by a binding the user cannot see.

8.6.2 Hygienic Expansion

Definition 143 (Hygienic macro system)

A macro system is *hygienic* if expansion preserves lexical scope: every name in the expanded program refers to the binding that was visible where the name was *written*. A name written in a macro's template is resolved where the macro is defined; a name written by the user in an argument is resolved where the macro is used.

Racket's expander is hygienic. It records, for every identifier, whether it came from the template or from the use, so the two `b`s above are different variables even though they print the same. The effect is as if the template's `b` had been renamed to a fresh name before pasting,

```
(let ([b #t])
  (let ([b1 #f])
    (if b1 b1 (multi-or b))))
```

and the program answers `#t`, which is what DrRacket printed in lecture.

8.6.3 Two Directions of Capture

Hygiene rules out capture in both directions.

1. *The macro captures the user's name.* A binding introduced by the template, like `b` above, must not capture a variable the user wrote in an argument.
2. *The user captures the macro's name.* A free name in the template, like `if` or `let`, must not be captured

by a binding at the use site. In

```
(let ([if 42])
  (multi-or #f 1))
```

the user has rebound `if` locally, but the `if` in the template still means Racket's `if`, the one visible where `multi-or` was defined. The answer is `1`.

The `loop` helper of `my-while` (Section 8.3) is protected by the first rule: a user variable named `loop` in `test` or `body` is not the macro's `loop`.

8.6.4 Connections

Key Point 144

Hygiene is lexical scope for macros. A text macro resolves every name after pasting, at the use site, which is the macro analogue of dynamic scope (Section 5.3), with the same symptom: renaming a variable changes what the program does. A hygienic expander avoids capture the way capture-avoiding substitution did in Section 4.3, by giving the introduced names fresh identities.

Remark 145. DrRacket's *Macro Stepper* shows each expansion step. On the example above, it displays the template's `b` and the user's `b` in different colors, which is a quick way to see which name refers to which binding.

8.7 Keywords in Patterns: my-if2

Every syntax-case so far had an empty list () after stx. That list names *literals*: identifiers that a pattern matches exactly, instead of binding them as pattern variables. They let a macro have keywords of its own:

```
(define-syntax (my-if2 stx)
  (syntax-case stx (then else)
    [(my-if2 e1 then e2 else e3)
     #'(if e1 e2 e3)]))
```

In the pattern, e1, e2, and e3 are pattern variables, but then and else are listed as literals, so the use must contain those words in those positions:

```
(my-if2 (> 2 1) then "yes" else "no")    ; expands to (if (> 2 1) "yes" "no")
```

A use without the keywords, such as (my-if2 (> 2 1) "yes" "no"), matches no clause, and the expander reports a syntax error for my-if2 before the program runs.

Pitfall 146 (Forgetting the literal list)

Without (then else), the pattern (my-if2 e1 then e2 else e3) would treat then and else as two more pattern variables. Every use with five subforms would match, whatever words stood in those positions, and (my-if2 c 1 2 3 4) would expand to (if c 2 4) without complaint.

Remark 147. A literal is matched as an identifier, by binding, not only by spelling: the else in the use must refer to the same else as the one in the macro's literal list. This is hygiene again (Section 8.6), applied to keywords. cond's else works the same way.

8.8 let as a Macro

In Section 5.1 we observed that let is an application of a lambda in disguise, and our interpreters used that equation to desugar letC. A macro states the same equation in Racket itself:

```
(define-syntax (my-let1 stx)
  (syntax-case stx ()
    [(my-let1 ([var val]) body)
     #'((lambda (var) body) val)]))
```

The pattern matches a let with exactly one binding, written with Racket's usual double parentheses. For example,

```
(my-let1 ([x 3]) (+ x x))
---> ((lambda (x) (+ x x)) 3)
```

which evaluates to 6. Square brackets and parentheses are interchangeable in patterns, as everywhere in Racket; the brackets only make the binding easier to read.

The pattern variable var appears in the template in a *binding* position, as the parameter of the lambda. So the user's x is bound by a lambda the macro wrote. This is fine under hygiene: the user wrote both the binding x and its uses in body, so they agree, and they are both resolved at the use site.

8.8.1 my-let*: Recursion over the Bindings

let* binds its variables one at a time, so each right-hand side can use the variables before it. Desugared into lambdas, each binding becomes one application, nested inside the previous one:

```
(my-let* ([x1 e1] [x2 e2]) e3)
---> ((lambda (x1) ((lambda (x2) e3) e2)) e1)
```

Since e2 sits inside (lambda (x1) . . .), it is in the scope of x1. The macro produces this nesting by recursion on the list of bindings, the same shape as multi-or (Section 8.5):

```
(define-syntax (my-let* stx)
  (syntax-case stx ()
    [(my-let* ([var val]) body)
     #'((lambda (var) body) val)]
    [(my-let* ([var val] . rest) body)
     #'((lambda (var) (my-let* rest body)) val)]))
```

The first clause is the base case, a single binding, which is my-let1. The second peels off the first binding. Its pattern ([var val] . rest) is a dotted pair, as in Chapter 1: rest matches the *list* of the remaining bindings, so the template can pass it on unchanged, (my-let* rest body), to a recursive use with one binding fewer. Expansion therefore terminates. An equivalent pattern is ([var val] rest ...), where rest matches the remaining bindings as a sequence and the template must wrap them back into a list, (my-let* (rest ...) body). The order of the clauses matters: a single binding also matches the second pattern, with rest empty, and would expand to (my-let* () body), which no clause matches. Listing the one-binding clause first catches that case before it reaches the second.

For example, `(my-let* ([x 1] [y (+ x 1)]) (* x y))` expands to `((lambda (x) ((lambda (y) (* x y)) (+ x 1))) 1)` and evaluates to 2. With `let` instead of `let*`, the `x` in `(+ x 1)` would be unbound, because all right-hand sides of a `let` are evaluated outside the new bindings.

8.8.2 `my-let`: Many Bindings at Once

A `let` with several bindings is a single `lambda` with several parameters, applied to all the right-hand sides at once:

```
(let ([x1 e1] [x2 e2] [x3 e3]) body)
---> ((lambda (x1 x2 x3) body) e1 e2 e3)
```

No recursion is needed; an ellipsis does it in one step:

```
(define-syntax (my-let stx)
  (syntax-case stx ()
    [(my-let ([var val] ...) body)
     #'((lambda (var ...) body) val ...)]))
```

In the pattern, the ellipsis follows the sub-pattern `[var val]`, so it matches a sequence of pairs, and `var` and `val` each stand for a sequence: all the names, and all the right-hand sides, in order. The template uses the two sequences separately, `var ...` as the parameter list and `val ...` as the arguments. The ellipsis thus "unzips" the list of pairs into two lists.

Key Point 148

`let` and `let*` desugar differently. In `my-let`, every `val` is an argument of the one application, outside the `lambda`, so no right-hand side can see any of the new variables. In `my-let*`, each `val` is inside the `lambda`s of the earlier bindings, so it can see them.

8.9 swap: A Macro That Mutates Variables

The last example needs to change the caller's variables:

```
(define r 100)
(define s 20)
(swap r s)        ; afterwards r should be 20 and s 100
```

8.9.1 As a Function: It Does Nothing

```
(define (swap x y)
  (let ([t y])
    (set! y x)
    (set! x t)))
```

`(swap r s)` runs without error, but `r` and `s` keep their values. Racket passes the *values* 100 and 20. The parameters `x` and `y` are new variables, with their own locations in the store in the sense of Chapter 6, initialized with copies of those values. The two `set!`s update the parameters' locations; `r`'s and `s`'s locations are never touched.

8.9.2 With Boxes: Pass the Locations

If the caller hands over boxes instead of numbers, the function can mutate what the boxes contain:

```
(define (swap-box b1 b2)
  (let ([t (unbox b2)])
    (set-box! b2 (unbox b1))
    (set-box! b1 t)))

(define r-box (box 10))
(define s-box (box 20))
```

After `(swap-box r-box s-box)`, `(unbox r-box)` is 20 and `(unbox s-box)` is 10. The parameters are still copies, but copies of box values, and a box value is a location: `b1` and `r-box` refer to the same box. This works, but only for data that the caller has put in boxes beforehand.

8.9.3 As a Macro: Rewrite the Caller's Code

```
(define-syntax (swap stx)
  (syntax-case stx ()
    [(swap x y)
     #'(let ([t y])
         (set! y x)
         (set! x t))]))
```

Now `(swap r s)` expands, at the place where it is used, into

```
(let ([t s])
  (set! s r)
  (set! r t))
```

The `set!`s name `r` and `s` themselves, so they update the caller's variables. No values are passed: the macro inserts code that refers to the variables directly.

Key Point 149

A function receives values, so it can mutate only what it can reach from those values (the contents of a box, the fields of a mutable struct). A macro receives the caller's code, so it can expand into `set!` on the caller's own variables. This is the closest Racket comes to call-by-reference.

Remark 150. The template introduces `t`. If the user's variable is also named `t`, as in `(swap t s)`, a text macro would expand to `(let ([t s]) (set! s t) (set! t t))`, which swaps nothing. Racket's hygienic expander keeps the two `t`s apart (Section 8.6), and the swap works. This is the `SWAP` example of Recitation 5 (Section 5.5).

8.10 Breaking Hygiene on Purpose: `aif`

This section was not covered in lecture. It comes from the end of the posted lecture code.

Hygiene (Section 8.6) is almost always what we want, but occasionally a macro is *meant* to bind a name that the user then writes. The standard example is the *anaphoric if*. A common pattern is to compute a value, test it, and use it in a branch. With a division that returns `#f` instead of failing,

```
(define (safe-div x y)
  (if (= y 0) #f (/ x y)))

(let ([q (safe-div 10 2)])
  (if q (+ q 1) 'undefined))
```

the `let` is boilerplate. An anaphoric if, `(aif test then-e else-e)`, binds the value of `test` to the fixed name `it`, so that the branches can refer to it: `(aif (safe-div 10 2) (+ it 1) 'undefined)`.

8.10.1 The Hygienic Version Fails

The obvious definition is

```
(define-syntax (aif-broken stx)
  (syntax-case stx ()
    [(aif-broken test then-e else-e)
     #'(let ([it test])
         (if it then-e else-e))]))
```

and `(aif-broken (safe-div 10 2) (+ it 1) 'undefined)` fails with `it: unbound identifier`. This is hygiene doing its job, in the first direction of Section 8.6: the `it` bound in the template comes from the macro, the `it` in `(+ it 1)` comes from the user, and a binding introduced by the template does not capture a name the user wrote. The user's `it` is looked up where it was written, where nothing named `it` is bound.

8.10.2 Choosing the Context of a Name

To make the binding visible to the user's code, the macro has to create an `it` that counts as written by the user:

```
(define-syntax (aif stx)
  (syntax-case stx ()
    [(aif test then-e else-e)
     (with-syntax ([it (datum->syntax stx 'it)])
       #'(let ([it test])
           (if it then-e else-e)))]))
```

- `(datum->syntax stx 'it)` turns the symbol `'it` into a syntax object, taking its lexical context from `stx`, the use of `aif` that the user wrote. The new identifier therefore behaves as if the user had written `it` at the use site.
- `with-syntax` binds the pattern variable `it` to that syntax object, as if it had been matched by a pattern. In the template, both occurrences of `it` are pattern-variable references, so both are replaced by the user-context `it`, and the `let` now binds the same `it` that the branches refer to.

The transformer now runs code of its own before building the result, which is the case where `define-syntax-rule` is not enough and the full `syntax-case` form is needed (Section 8.2). As intended,

```
(aif (safe-div 10 2) (+ it 1) 'undefined)    ; 6
(aif (safe-div 10 0) (+ it 1) 'undefined)    ; 'undefined
```

8.10.3 The Price

The capture is now real, and it is not limited to the branches the macro author had in mind. It also hides any `it` the user already has:

```
(let ([it "user's it"])
  (aif #f 3 it))
```

The `else` branch looks as if it uses the user's `it`, but it is inside the binding made by `aif`, so it gets the test value, and the result is `#f`, not `"user's it"`. This is the capture problem of the C preprocessor (Section 5.5), now created on purpose.

Key Point 151

In a hygienic system, capture is the exception and has to be asked for explicitly, one name at a time, with `datum->syntax`. A macro that does so makes that name part of its interface: users must know that `aif` binds `it`, just as they know that `lambda` binds its parameters.

8.11 Recitation 8: The Lambda Calculus

Our interpreters have kept growing: numbers, booleans, `let`, boxes, `rec`. This recitation goes the other way. It starts from a language with *only* functions, the *lambda calculus* (LC), and builds booleans, numbers, and recursion out of functions alone. The handout `recitation8.rkt` is written in `#lang scribble/lp2`, a literate-programming language: the text and the code chunks render as a document, and the chunks run as a program with `check-expect` tests.

8.11.1 Syntax

LC has three kinds of terms:

$$LC ::= x \mid \lambda x.\, e \mid e_1\ e_2$$

written as s-expressions `(var x)`, `(lam x e)`, and `(app e1 e2)`. The identity function $\lambda x.\, x$ is `'(lam x (var x))`, and applying it to y is `'(app (lam x (var x)) (var y))`.

Terms are built and taken apart with *quasiquote*. The backquote quotes a list, and each comma plugs a Racket value into it:

```
`(app ,e1 ,e2)          ; = (list 'app e1 e2)
```

Inside a `match` pattern, the same notation takes a list apart: `` `(app ,e1 ,e2) `` matches any three-element list whose first element is `'app`, and binds `e1` and `e2`. A pattern `,(? symbol?)` matches any symbol without binding it.

8.11.2 Given: Renaming, Alpha-Equivalence, Substitution

The handout provides three functions on terms, which the exercises build on.

`is-lc?`. checks that an s-expression is a term, so `'(lam x x)` (body not a term) and `'(var 5)` (not a symbol) are rejected.

`(rename-var lc x x-new)`. renames the *free* occurrences of `x` to `x-new`. It stops at a `lam` that binds `x` again, since the occurrences under it refer to that inner binder:

```
(define (rename-var lc x x-new)
  (match lc
    [`(var ,v) (if (eq? v x) `(var ,x-new) lc)]
    [`(lam ,v ,body)
     (if (eq? v x)
         lc
         `(lam ,v ,(rename-var body x x-new)))]
    [`(app ,e1 ,e2) `(app ,(rename-var e1 x x-new) ,(rename-var e2 x x-new))]))
```

`x-new` must be *fresh*, that is, appear nowhere in `lc`. Otherwise it can be captured: renaming `x` to `y` in `'(lam y (var x))` would give `'(lam y (var y))`, a different function. The handout only calls `rename-var` with names from `gensym`, which are always fresh.

Alpha-equivalence. `'(lam x (var x))` and `'(lam y (var y))` are the same function; they differ only in the name of the bound variable.

Definition 152 (Alpha-equivalence)

Two terms are *alpha-equivalent* if they are equal up to a consistent renaming of bound variables.

`equiv-LC` checks this by renaming the binders of two `lam`s to one fresh name and comparing the bodies:

```
(define (equiv-LC lc1 lc2)
  (match* (lc1 lc2)
    [(`(var ,x1) `(var ,x2)) (eq? x1 x2)]
    [(`(lam ,x1 ,body1) `(lam ,x2 ,body2))
     (let ([x-new (gensym)])
       (equiv-LC (rename-var body1 x1 x-new) (rename-var body2 x2 x-new)))]
    [(`(app ,e11 ,e12) `(app ,e21 ,e22))
     (and (equiv-LC e11 e21) (equiv-LC e12 e22))]
    [(_ _) #f]))
```

`match*` matches several values at once. Free variables must be literally equal: `'(lam x (var y))` is not equivalent to `'(lam y (var y))`, because the first body mentions a free `y` and the second the bound one. Nested binders may also swap names, so $\lambda x.\, \lambda y.\, x\ y$ and $\lambda y.\, \lambda x.\, y\ x$ are equivalent.

Substitution. `(subst lc x arg)` replaces the free occurrences of `x` in `lc` by `arg`, avoiding capture:

```
(define (subst lc x arg)
  (match lc
    [`(var ,v) (if (eq? v x) arg lc)]
    [`(lam ,v ,body)
     (if (eq? v x)
         lc
         (let ([v-new (gensym v)])
           `(lam ,v-new ,(subst (rename-var body v v-new) x arg))))]
    [`(app ,e1 ,e2) `(app ,(subst e1 x arg) ,(subst e2 x arg))]))
```

This is the capture-avoiding substitution of Section 4.3, with a simplification: it renames the binder of *every* `lam` it enters to a fresh name, whether or not capture could happen. Then no binder can capture a free variable of `arg`, and the code never has to compute which variables are free in `arg`. The cost is that results contain names like `y1825`, so tests compare them with `equiv-LC` instead of `equal?`. The capture case is

```
(subst '(lam y (var x)) 'x '(var y))     ; alpha-equivalent to '(lam w (var y))
```

which must not become the identity function `'(lam y (var y))`.

8.11.3 Evaluation: Normal Order

Definition 153 (Redex, normal form)

A *redex* is a function applied to an argument, `` `(app (lam ,x ,body) ,arg) ``. *Reducing* it means substituting `arg` for `x` in `body`. A term with no redex anywhere in it is in *normal form*, and the normal form is the answer.

The interpreter of this recitation evaluates in *normal order*, which differs from our earlier interpreters in three ways:

- It reduces the *outermost* redex first. `(app (lam x (var z)) (app (lam y (var y)) (var b)))` reduces at once to `(var z)`, without touching the argument. Racket and our interpreters would evaluate the argument first.
- Among subterms, it goes *left to right*.
- It also reduces *under* `lam`. Our interpreters treated a function as a value and never looked inside it; here `(lam x (app (lam y (var y)) (var x)))` steps to `(lam x (var x))`.

Exercise 154 (One step)

Write `(step-LC lc)`, which finds the redex that normal order picks, reduces it, and returns the new term, or returns `#f` if `lc` has no redex. Before writing code, fill in the expected results of the handout's tests:

```
(step-LC '(app (lam x (var x)) (var y)))
(step-LC '(var x))
(step-LC '(app (var f) (var a)))
(step-LC '(app (lam x (var z)) (app (lam y (var y)) (var b))))
(step-LC '(app (app (lam x (var x)) (var a)) (app (lam y (var y)) (var b))))
(step-LC '(lam x (app (lam y (var y)) (var x))))
```

Hints from the handout: for an `app` that is not a redex, try to step `e1`, and if that returns `#f`, try `e2`. The redex case must come before the general `app` case in the `match`, since every redex also matches `` `(app ,e1 ,e2) ``.

Solution. The expected results, in order:

```
'(var y)                                    ; the redex itself
#f                                          ; a variable has no redex
#f                                          ; f is not a lam
'(var z)                                    ; outermost first; argument dropped
'(app (var a) (app (lam y (var y)) (var b)))   ; left subterm first
'(lam x (var x))                            ; under the lam
```

The fifth term is not itself a redex, since its function part is an `app`, not a `lam`. So the left subterm steps, and the right one is left for later. In the last test, `subst` enters no `lam`, so the result has no gensym names and `equal?` works.

`step-LC` has one case per form, with the redex first:

```
(define (step-LC lc)
  (match lc
    [`(app (lam ,x ,body) ,arg) (subst body x arg)]
    [`(app ,e1 ,e2)
     (let ([s1 (step-LC e1)])
       (if s1
           `(app ,s1 ,e2)
           (let ([s2 (step-LC e2)])
             (and s2 `(app ,e1 ,s2)))))]
    [`(lam ,x ,body)
     (let ([s (step-LC body)])
       (and s `(lam ,x ,s)))]
    [`(var ,_) #f]))
```

`(and s2 . . . )` is `#f` when `s2` is `#f`, and the rebuilt term otherwise. Each case encodes one rule of normal order. The redex case reduces the outermost redex before looking inside the argument. The `app` case goes left to right, trying `e2` only when `e1` has no redex. The `lam` case steps under the binder.

Exercise 155 (Normal form)

Write `(interp-LC lc)`, which steps until no redex is left and returns the normal form.

Solution.

```
(define (interp-LC lc)
  (let ([next (step-LC lc)])
    (if next
        (interp-LC next)
        lc)))
```

`step-LC` returns `#f` exactly when `lc` is in normal form, and that is the answer. The call is in tail position, so a long reduction does not grow the stack.

Not every term has a normal form. The classic example is

$$\Omega = (\lambda w.\, w\ w)\ (\lambda w.\, w\ w),$$

`omega` in the handout. Its only redex substitutes the second copy for `w` in `(app (var w) (var w))`, which gives back Ω itself, so `(interp-LC omega)` runs forever. The test `(check-expect (step-LC omega) omega)` can use `equal?`: the body `(app (var w) (var w))` contains no `lam`, so `subst` renames nothing and the result is literally `omega` again.

The handout's tests for `interp-LC` include

```
(interp-LC `(app (lam x (var z)) ,omega))      ; should be '(var z)
```

Exercise 156 (Discussion)

What would an interpreter that evaluates arguments first, like Racket, do with `` `(app (lam x (var z)) ,omega) ``?

Solution. It runs forever. Before calling `(lam x (var z))`, it evaluates the argument Ω, which steps only to itself. Normal order calls the function first. The function never uses `x`, so Ω is thrown away without being evaluated. In general, if a term has a normal form, normal order is guaranteed to find it (the *standardization theorem*). Evaluating arguments first can loop on such a term, as here.

8.11.4 Church Booleans

LC has no booleans, but they can be *encoded* as functions. A boolean is a function that chooses between two arguments, *true* picking the first and *false* the second:

$$true = \lambda x.\, \lambda y.\, x \qquad false = \lambda x.\, \lambda y.\, y$$

```
(define LC-true  '(lam x (lam y (var x))))
(define LC-false '(lam x (lam y (var y))))
```

`NF-to-bool` reads a normal form back as a Racket boolean, comparing with `equiv-LC`, and signals an error for anything else.

Exercise 157 (If)

Define `LC-if`, a term that takes a boolean `c` and two terms `t` and `e`, one argument at a time, and returns `t` if `c` is true and `e` otherwise. The tests apply it as `` `(app (app (app ,LC-if ,LC-true) (var yes)) (var no)) ``.

Hint: a boolean already *is* a choice between two arguments. What is left for `LC-if` to do?

Solution. Nothing but passing the branches to the boolean: $if = \lambda c.\, \lambda t.\, \lambda e.\, c\ t\ e$.

```
(define LC-if
  '(lam c (lam t (lam e (app (app (var c) (var t)) (var e))))))
```

With $c = true$, $c\ t\ e$ reduces to t. Under normal order the branch not taken is never evaluated, so `LC-if` needs no special treatment. In Racket it does: there, `if` must be a special form, and a function `my-if` fails (Section 8.2).

Exercise 158 (Not, And)

Define `LC-not` and `LC-and` in the same style. `LC-true` and `LC-false` can be plugged in with quasiquote.

Hint: write each one as "if ... then ... else ..." first, and then use that a boolean is its own `if`.

Solution. $not\ b$ is "if b then false else true", and $and\ a\ b$ is "if a then b else false". Since a boolean applied to two terms is already an `if`:

```
(define LC-not
  `(lam b (app (app (var b) ,LC-false) ,LC-true)))
(define LC-and
  `(lam a (lam b (app (app (var a) (var b)) ,LC-false))))
```

The commas plug the terms `LC-false` and `LC-true` in. Without the quasiquote, `'(. . . LC-false . . .)` would contain the symbol `LC-false`, which is not a term. `LC-and` returns b itself when a is true, just as `and` in Racket returns its last argument.

8.11.5 Church Numerals

The natural number n is encoded as the function that applies f to x, n times:

$$\bar{0} = \lambda f.\, \lambda x.\, x \qquad \bar{1} = \lambda f.\, \lambda x.\, f\ x \qquad \bar{2} = \lambda f.\, \lambda x.\, f\ (f\ x) \qquad \bar{3} = \lambda f.\, \lambda x.\, f\ (f\ (f\ x))$$

These are *Church numerals*. `(LC-nat n)` builds the numeral for a Racket number, and `NF-to-nat` reads a normal form back by counting the `f`s. It works for any names of the two binders, so the gensym names in a result do not matter.

Exercise 159 (Arithmetic)

Define `LC-succ`, `LC-add`, and `LC-mul`. Each takes numerals and returns a numeral, a term of the form $\lambda f.\, \lambda x.\, \ldots$. Hints from the handout:

- $n\ f\ x$ applies f to x, n times. Successor applies it once more.
- $m + n$: apply f n times, then m more times.
- $m \times n$: $n\ f$ is a function that applies f n times. Apply *that* m times.

The tests include 0×7 and 3×0.

Solution. Each definition takes its numerals, then f and x, and builds the body from the hint:

$$\begin{aligned} succ &= \lambda n.\, \lambda f.\, \lambda x.\, f\ (n\ f\ x) \\ add &= \lambda m.\, \lambda n.\, \lambda f.\, \lambda x.\, m\ f\ (n\ f\ x) \\ mul &= \lambda m.\, \lambda n.\, \lambda f.\, \lambda x.\, m\ (n\ f)\ x \end{aligned}$$

```
(define LC-succ
  '(lam n (lam f (lam x (app (var f) (app (app (var n) (var f)) (var x)))))))
(define LC-add
  '(lam m (lam n (lam f (lam x (app (app (var m) (var f))
                                    (app (app (var n) (var f)) (var x))))))))
(define LC-mul
  '(lam m (lam n (lam f (lam x (app (app (var m) (app (var n) (var f)))
                                    (var x)))))))
```

In `LC-add`, $n\ f\ x$ is f applied n times to x, and $m\ f$ applies f m more times to that. In `LC-mul`, $n\ f$ is "apply f n times", and m repeats that m times, $m \cdot n$ applications in total. The zero cases need no special handling. For 0×7, $\bar{0}$ ignores its function and returns x. For 3×0, $\bar{0}\ f$ is the identity, and three identities applied to x give x.

Because normal order reduces under `lam`, the result is a numeral in normal form, which is what `NF-to-nat` expects. The shorter $mul = \lambda m.\, \lambda n.\, \lambda f.\, m\ (n\ f)$ also passes the tests: m is a numeral, so $m\ (n\ f)$ reduces to a $\lambda x.\ \ldots$ itself.

Given: zero test and predecessor. `LC-is-zero` applies the constant function $\lambda u.\, false$ to $true$, n times. Zero applications leave $true$, and one or more give $false$:

```
(define LC-is-zero
  `(lam n (app (app (var n) (lam u ,LC-false)) ,LC-true)))
```

`LC-pred` is famously tricky, and it is given:

```
(define LC-pred
  '(lam n
     (lam f
       (lam x
         (app (app (app (var n)
                        (lam g (lam h (app (var h)
                                           (app (var g) (var f))))))
                    (lam u (var x)))
               (lam u (var u)))))))
```

It builds the result one f at a time, but its starting value $\lambda u.\, x$ throws away the first f, so n copies of f become $n - 1$. The predecessor of $\bar{0}$ is $\bar{0}$. Wikipedia's "Church encoding" page has the details.

8.11.6 Challenge: Recursion with `Y`

In Recitation 7 (Section 7.5), `Y` looped forever in Racket and `Z` was needed instead. This part tries `Y` again, in LC with normal order.

The handout provides Racket helpers that build bigger terms, so that not every `app` has to be written by hand. Their arguments are LC terms, so write `(SLC-zero? '(var n))`, not `(SLC-zero? 'n)`:

```
(define (SLC-if c t e)  `(app (app (app ,LC-if ,c) ,t) ,e))
(define (SLC-zero? e)   `(app ,LC-is-zero ,e))
(define (SLC-pred e)    `(app ,LC-pred ,e))
(define (SLC-mul e1 e2) `(app (app ,LC-mul ,e1) ,e2))
```

`SLC-if` and `SLC-mul` use your `LC-if` and `LC-mul`, so if the factorial tests fail, check those first.

Exercise 160 (Y and factorial)

Write `LC-Y`, the Y combinator of Section 7.5,

$$Y = \lambda h.\,(\lambda x.\,h\;(x\;x))\;(\lambda x.\,h\;(x\;x)),$$

as an LC term. Then write `LC-fact-gen`, the LC version of `fact-gen`: it takes the recursive call `fact` and returns a function of `n`. The tests check $0! = 1$, $3! = 6$, and $4! = 24$ via `` `(app (app ,LC-Y ,LC-fact-gen) ,(LC-nat 3)) ``.

Solution. `LC-Y` is the formula transcribed:

```
(define LC-Y
  '(lam h (app (lam x (app (var h) (app (var x) (var x))))
               (lam x (app (var h) (app (var x) (var x)))))))
```

`LC-fact-gen` is `fact-gen` of Section 7.5, built with the helpers:

```
(define LC-fact-gen
  `(lam fact
     (lam n
       ,(SLC-if (SLC-zero? '(var n))
                (LC-nat 1)
                (SLC-mul '(var n)
                         `(app (var fact) ,(SLC-pred '(var n))))))))
```

The comma before `(SLC-if . . . )` switches back to Racket, so the helpers run and build the body of the `lam`. The inner backquote starts a new quasiquote for the recursive call `(fact (pred n))`. All three tests pass. In the Python port used to check these solutions, $4!$ takes about 4100 steps.

Exercise 161 (Discussion)

1. Why does `Y` work here, when it looped in Racket last week?
2. Does `(interp-LC `(app ,LC-Y ,LC-fact-gen))`, without a number, finish?
3. Write `Z` as an LC term. Does it compute factorial in LC? What about `Z` applied to `LC-fact-gen` alone, without a number?

Solution.

1. In Racket, `(Y fact-gen)` loops because the argument `(W W)` must be evaluated before `fact-gen` is called, and `(W W)` only unfolds into itself (Section 7.5). Normal order reduces the outermost redex first, so `fact-gen` is called with `(W W)` *unevaluated*, as in the lazy evaluation of Section 4.2. `(W W)` unfolds one more level only when it reaches function position, that is, when factorial actually makes a recursive call. At $n = 0$, the `if` picks the base case, and the remaining copy of `(W W)` is discarded.
2. No. $Y\ \mathit{fact\text{-}gen}$ reduces to $\lambda n.\ \mathit{if}\ (\mathit{iszero}\ n)\ \bar{1}\ (n \times (W\ W)\ (\mathit{pred}\ n))$. Here n is a variable, so the test

cannot decide between the branches. But normal order reduces under λ, and so inside both branches. In the second branch, $(W\ W)$ unfolds into another copy of the factorial body, which contains another $(W\ W)$, and so on. The term has no normal form, and `interp-LC` runs forever. With a number, the test decides, and the unfolding stops.

3. `Z` is `Y` with each `(x x)` wrapped in `(lam v . . . )`:

```
(define LC-Z
  '(lam h (app (lam x (app (var h) (lam v (app (app (var x) (var x)) (var v)))))
               (lam x (app (var h) (lam v (app (app (var x) (var x)) (var v))))))))
```

With a number, it computes factorial as well: $0!$ and $3!$ come out right, and $3!$ takes about the same number of steps as with `Y`. Without a number, it also runs forever. The `(lam v . . . )` delayed `(x x)` in Racket because Racket never evaluates inside a `lambda`. Normal order does reduce under `lam`, so the wrapper delays nothing, and `(x x)` unfolds forever as before.

The lesson: `Y` versus `Z` is not a property of the combinators alone. It depends on the evaluation strategy, namely whether arguments are evaluated before the call and whether evaluation goes under λ.

Bibliography

[1] S. Krishnamurthi. *Programming Languages: Application and Interpretation*, 2nd edition. 2017. https://cs.brown.edu/~sk/Publications/Books/ProgLangs/

[2] M. Felleisen, R. B. Findler, M. Flatt, and S. Krishnamurthi. *How to Design Programs*, 2nd edition. MIT Press, 2018. https://htdp.org

[3] H. Abelson and G. J. Sussman. *Structure and Interpretation of Computer Programs*, 2nd edition. MIT Press, 1996.

[4] M. Flatt, R. B. Findler, and PLT. *The Racket Guide*. https://docs.racket-lang.org/guide/

[5] B. C. Pierce. *Types and Programming Languages*. MIT Press, 2002.

[6] J. C. Reynolds. Definitional interpreters for higher-order programming languages. In *Proceedings of the ACM Annual Conference*, 1972.

[7] M. L. Scott. *Programming Language Pragmatics*, 4th edition. Morgan Kaufmann, 2015.

[8] P. Ragde. *Teach Yourself Racket*. https://cs.uwaterloo.ca/~plragde/flaneries/TYR/

[9] S. J. Gould. What, if anything, is a zebra? Reprinted in *Hen's Teeth and Horse's Toes*. Norton, 1983.

[10] W. Clinger and M. Wand. Hygienic macro technology. *Proceedings of the ACM on Programming Languages* 4 (HOPL), 2020. https://doi.org/10.1145/3386330

[11] G. Hendershott. *Fear of Macros*. https://www.greghendershott.com/fear-of-macros/